FeladatRadar

VIZSGAREMEK

BGÉSZC GANZ ÁBRAHÁM KÉT TANÍTÁSI NYELVŰ TECHNIKUM

KÉSZÍTETTE: Tarcsányi Csongor Márk,
Le Tuan Anh (Róbert),
Lőrincz Antal

SZAKMA MEGNEVEZÉSE: Szoftverfejlesztő és -tesztelő

TÉMAVEZETŐ: Szalainé Török Edit,
Méhes József,
Csukárdi Rajmund

Tartalomjegyzék

BEVEZETÉS....7
1.1 A projekt háttere és célja....7
1.2 Témaválasztás indoklása....8
1.3 A program funkciói röviden....8
1.4 Célközönség....9
FEJLESZTŐI DOKUMENTÁCIÓ....10
2.1 Hasonló meglévő megoldások és miben más a FeladatRadar....10
2.2 Telepítési útmutató részletesen....10
2.3 Projekt menedzsment és csapatmunka dokumentációja....13
2.3.1 Csapatösszetétel és feladatkiosztás....13
Tarcsányi Csongor Márk feladatai....13
Lőrincz Antal feladatai....13
Közös fejlesztések....14
2.3.2 Kommunikáció és koordináció....14
2.3.3 Kihívások és megoldásaik....15
2.3.4 Verziókövetés és Git munkafolyamat....17
2.4 A fejlesztőkörnyezet....19
2.4.1 Hardverkövetelmények....19
2.4.2 Felhasznált szoftverek....19
2.4.3 Programnyelv és keretrendszer választása....20
2.5 Rendszerarchitektúra....22
2.5.1 A rendszer felépítése....22
2.5.2 Projekt- és mappastruktúra....23
2.5.3 Rétegek közötti kommunikáció....28
2.6 Adatbázis-struktúra....28
2.6.1 Az adatbázis ismertetése....28
2.6.2 Táblák részletes leírása....29
2.6.3 Táblák kapcsolatai....39
2.6.4 Tárolt eljárások....40
2.7 Autentikáció és jogosultságkezelés....43
2.7.1 JWT-token alapú autentikáció....43
2.7.2 Szerepkör-alapú hozzáférés-szabályozás (RBAC)....43
2.7.3 Jelszókezelés: BCrypt....44
2.7.4 SQL Injection védelem....44
2.7.5 CORS konfiguráció....45

2.7.6 Biztonsági megoldások részletes ismertetése *46*
2.7.6.1 Jelszótárolás: BCrypt 46
2.7.6.2 JWT token biztonság 46
2.7.6.3 Szerep-alapú hozzáférés-szabályozás (RBAC) 47
2.8 Főbb algoritmusok 47
2.8.1 Ismétlődő feladatok kezelése *47*
2.8.2 Határidő-közelség kiszámítása *49*
2.8.3 E-mail alapú meghívás csoportba *50*
2.8.4 FocusTimerService *50*
2.8.5 E-mail uniqueness ellenőrzése *51*
2.8.6 Szavazási algoritmus (Upsert) *51*
2.8.7 Órarend ütközés-felismerés *52*
2.9 Backend felépítése 52
2.9.1 Service osztályok bemutatása *53*
2.9.1.1 AuthService (Backend) 53
2.9.1.2 TaskService (Backend) 55
2.9.1.3 ScheduleService (Backend) 56
2.9.1.4 GroupService (Backend) 57
2.9.1.5 TeacherService (Backend) 58
2.9.2 API végpontok dokumentációja *58*
2.9.2.1 Autentikáció: /api/auth 58
2.9.2.2 Feladatok: /api/task 59
2.9.2.3 Órarend: /api/schedule 59
2.9.2.4 Csoportok: /api/group 59
2.9.2.5 Tanári funkciók: /api/teacher 60
2.9.2.6 Tantárgyak: /api/subject 60
2.9.3 DTO-k és adatátviteli modellek *61*
2.9.3.1 Autentikációs DTO-k 61
2.9.3.2 Feladat DTO-k 62
2.9.3.3 Csoport DTO-k 62
2.9.3.4 Szavazási DTO-k 62
2.9.3.5 Tanári dolgozat DTO-k 63
2.9.3.6 Ismétlődési típusok: RecurrenceKind static osztály 63
2.9.4 Dapper használati minták *63*
2.9.4.1 Típusos lekérdezés (QueryAsync<T>) 63
2.9.4.2 Dinamikus lekérdezés (QueryAsync<dynamic>) 64
2.9.4.3 DynamicParameters használata 64
2.9.5 Admin funkciók *65*
2.9.5.1 AdminController és AdminService 65
2.9.5.2 AdminService (Frontend) 66
2.9.5.3 Kaszkád törlés algoritmusa 67

2.9.5.4 Soft-delete mechanizmus 68
2.9.6 Kódminőség és konvenciók *68*
2.9.7 Swagger API dokumentáció *70*
2.10 Frontend felépítése 70
2.10.1 Oldal-architektúra és navigáció *70*
2.10.2 Komponensek bemutatása *71*
2.10.2.1 Dashboard.razor 71
2.10.2.2 Login.razor és Register.razor 72
2.10.2.3 Naptar.razor 72
2.10.2.4 Feladatok.razor 80
2.10.2.5 Focus.razor 82
2.10.2.6 Tantargyak.razor 83
2.10.2.7 Csoport.razor 83
2.10.2.8 Home.razor 87
2.10.3 Frontend Service réteg *87*
2.10.3.1 AuthService (Frontend) 87
2.10.3.2 TaskService (Frontend) 90
2.10.3.3 SubjectService (Frontend) 92
2.10.3.4 ScheduleService (Frontend) 93
2.10.3.5 GroupService (Frontend) 94
2.10.3.6 TeacherService (Frontend) 95
2.10.3.7 FocusTimerService 96
2.10.4 Blazor életciklus és állapotkezelés *97*
2.10.4.1 OnInitializedAsync 97
2.10.4.2 StateHasChanged és InvokeAsync 97
2.10.4.3 IDisposable implementáció 98
2.10.5 CSS architektúra és témarendszer *98*
2.11 Fejlesztési lehetőségek 99
2.11.1 Értesítési rendszer *99*
2.11.2 Mobilalkalmazás *99*
2.11.3 Fájlfeltöltés feladatokhoz *99*
2.11.4 Valós idejű csevegés csoportokban *99*
FELHASZNÁLÓI DOKUMENTÁCIÓ 100
3.1 Rendszerkövetelmények és hozzáférés 100
3.1.1 Hardver- és szoftverkövetelmények *100*
3.1.2 Hálózati követelmények *101*
3.1.3 Az alkalmazás elérése *101*
3.2 Fiók létrehozása és bejelentkezés 102
3.2.1 A kezdőoldal *102*
3.2.2 Regisztráció *103*

3.2.3 Bejelentkezés ... *106*
3.3 Az alkalmazás felülete: navigáció és elrendezés ... 109
3.3.1 A navigációs sáv (sidebar) ... *109*
3.3.2 A felső sáv (topbar) ... *109*
3.3.3 Sötét és világos mód ... *110*
3.3.4 Mobil nézet ... *111*
3.4 Főoldal ... 111
3.4.1 Az üdvözlő sáv ... *112*
3.4.2 Statisztikai kártyák ... *112*
3.4.3 Közelgő feladatok panel ... *113*
3.4.4 Gyors navigáció panel ... *113*
3.5 Naptár ... 114
3.5.1 A naptár fejléce ... *114*
3.5.2 Nézetek ... *115*
3.5.3 Navigáció a naptárban ... *115*
3.5.4 Új órarend-bejegyzés hozzáadása ... *116*
3.5.5 Bejegyzés törlése ... *116*
3.5.6 Csoport mód ... *117*
3.6 Feladatok ... 118
3.6.1 A feladatlista ... *118*
3.6.2 Szűrés ... *119*
3.6.3 Keresés ... *120*
3.6.4 Rendezés ... *120*
3.6.5 Új feladat hozzáadása ... *120*
3.6.6 Feladat teljesítése és visszaállítása ... *121*
3.6.7 Feladat törlése ... *121*
3.7 Fókusz Időzítő: Pomodoro ... 121
3.7.1 Az időzítő főablaka ... *123*
3.7.2 Módok ... *123*
3.7.3 Az időzítő vezérlése ... *123*
3.7.4 Automatikus váltás (AutoSwitch) ... *123*
3.7.5 Beállítások panel ... *124*
3.7.6 Munkamenet-napló ... *124*
3.8 Tantárgyak ... 124
3.8.1 A tantárgyak megjelenítése ... *125*
3.8.2 Tantárgy felvétele listából ... *126*
3.8.3 Egyedi (privát) tantárgy hozzáadása ... *126*
3.8.4 Tantárgy leadása ... *126*

3.9 Csoportok 126
3.9.1 Az oldal felépítése 127
3.9.2 Csoport létrehozása 127
3.9.3 Tagok meghívása 128
3.9.4 A csoport fülei 128
3.9.5 Kilépés a csoportból 130
3.10 Admin felület 130
3.10.1 Az Admin felület fülei 131
3.10.2 Áttekintés fül 131
3.10.3 Felhasználók fül 132
3.10.4 Csoportok fül 133
3.10.5 Napló fül (Audit Log) 133
3.11 Hibaelhárítás: Gyakori problémák és megoldások 133
3.11.1 Bejelentkezési problémák 133
3.11.2 Naptár problémák 134
3.11.3 Feladatok problémák 134
3.11.4 Csoport problémák 135
3.11.5 Általános problémák 135
3.12 Tippek és trükkök a hatékony használathoz 136
3.12.1 Tippek diákoknak 136
3.12.2 Tippek tanároknak 136
3.12.3 Tippek adminoknak 137
3.13 Kapcsolat és elérhetőség 137
ÖSSZEFOGLALÁS ÉS KÖSZÖNETNYILVÁNÍTÁS 138
4.1 A projekt értékelése 138
4.2 A program jövőbeli hasznosulása 138
4.3 A tanulságok 139
4.4 Köszönetnyilvánítás 140
4.5 Záró gondolatok 141
4.5.1 Le Tuan Anh (Róbert) visszatekintése 141
4.5.2 Tarcsányi Csongor Márk visszatekintése 142
4.5.3 Lőrincz Antal visszatekintése 142
4.5.4 Végszó 143
IRODALOMJEGYZÉK 144
5.1 Felhasznált online források 144
5.2 Felhasznált könyvek 145

Bevezetés

1.1 A projekt háttere és célja

A FeladatRadar egy modern, webalapú iskolai feladatkezelő alkalmazás, amelyet középiskolás és felsőoktatásban tanuló diákok számára fejlesztettünk. Az ötlet egy mindennapi problémából született: a tanulók számos különböző csatornán (üzenetküldő alkalmazások, e-mail, szóbeli közlés) kapják a feladatokkal, dolgozatokkal és határidőkkel kapcsolatos értesítéseket, amelyeket nehéz egy helyen nyomon követni. A legtöbb diák füzetbe vagy telefonjának naptárjába írja fel a teendőket, de ezek könnyen elvesznek, és nem teszik lehetővé a csoportos együttműködést.

A FeladatRadar célja, hogy egyetlen, könnyen kezelhető felületen egyesítse az egyéni feladatkezelést, az órarend-nyilvántartást, a fókuszált tanulást segítő időzítőt, valamint a csoportos koordinációt, mindezt egy letisztult, modern kinézetű webalkalmazás formájában.

1.2 Témaválasztás indoklása

A projekt ötlete egy hétköznapi helyzetből született: egyik este, amikor egyikünk betegen feküdt otthon, szembesültünk azzal, hogy senki sem tudta pontosan, mi vár ránk a következő héten. Nem volt egyetlen megbízható felület, ahol mindent egyben láthattunk volna. Ebből a személyes frusztrációból nőtt ki a FeladatRadar ötlete. A mai elérhető rendszerek mint a Kréta vagy a Google Classroom merev, intézményi logika szerint működnek: előbbi közösségi és rugalmas feladatkezelés nélküli adminisztrációs eszköz, utóbbi inkább tananyag-megosztásra koncentrál. A FeladatRadar ezzel szemben tudatosan nem kizárólag iskolai célra készült. Minden felhasználó a saját életritmusához igazíthatja: valaki iskolai teendőit követi benne nyomon, valaki sportversenyeit és edzésmenetrendjét szervezi általa, megint más csapatban zajló szabadidős tevékenységeit kezeli benne. A témaválasztást három tényező motiválta: a személyesen megélt probléma, a webes technológiák különösen az ASP.NET Core és a Blazor elmélyült megismerésének szakmai ambíciója, valamint az a szándék, hogy olyan platformot építsünk, amely az iskolai közösségünkön túl is valódi hasznosulással bírhat. A fejlesztés során nemcsak programozási tudásunkat mélyítettük el, hanem megtapasztaltuk a valódi csapatmunka kihívásait is, a feladatok szétosztását, a verziókövetést, és a közös döntéshozatal folyamatát.

1.3 A program funkciói röviden

A FeladatRadar a következő főbb funkciókat kínálja felhasználóinak:

- Regisztráció és bejelentkezés: kétféle szerepkörrel (Diák, Tanár)
- Személyes órarend kezelése: heti, napi és havi nézet, ismétlődő bejegyzésekkel
- Feladatlista: határidős emlékeztetőkkel, típusonkénti szűréssel
- Fókusz időzítő: Pomodoro-technikán alapuló tanulássegítő eszköz
- Tantárgy-nyilvántartás: tantárgy felvétele listából vagy egyedi bevitellel
- Csoportkezelés: meghívásos rendszer, csoportos órarend és feladatmegosztás
- Szavazás csoporton belül: dolgozatok időpontjáról, feladatokról
- Tanári modul: vizsgák és dolgozatok hozzáadása csoportok számára
- Admin felület: felhasználók kezelése (aktiválás, szerepkör módosítás, törlés), rendszerstatisztikák megtekintése és auditnapló nyomon követése

1.4 Célközönség

Az alkalmazás elsősorban olyan felhasználók számára készült, akik szeretnék szervezettebben kezelni mindennapi feladataikat, legyen szó tanulmányokról, munkáról, sportról vagy bármilyen közösségi tevékenységről. A tanári szerepkör lehetővé teszi, hogy csoportvezetők, edzők vagy szervezők is bekapcsolódjanak: feladatokat és eseményeket hirdessenek meg a csoportjuknak, és szavazásokat indítsanak közös döntések meghozatalához. A felhasználói felület kialakításánál fontos szempont volt a mobil-kompatibilitás: az alkalmazás reszponzív, kisebb képernyőn is jól használható, mivel a legtöbb felhasználó okostelefonról is szívesen követi nyomon a feladatait és eseményeit.

Fejlesztői dokumentáció

2.1 Hasonló meglévő megoldások és miben más a FeladatRadar

Számos feladatkezelő alkalmazás létezik a piacon: a legismertebbek a Todoist, a Microsoft To Do, a Notion és az iskolai célra kialakított Google Classroom. Ezek közül azonban egyik sem kombinálja egyetlen, könnyen kezelhető felületen az alábbiakat:

- Személyre szabható heti/napi/havi nézet, amely igazodik bármilyen rutinhoz: legyen az tanulmányi, munkahelyi, sport- vagy versenynapirend
- Csoportos együttműködés, amellyel csapattagok, Diákok és Tanárok egyaránt összehangolhatják teendőiket
- Beépített Pomodoro-fókusz időzítő munkamenet-naplózással
- Csoporton belüli szavazás funkció (pl. dolgozat időpontjáról)

A FeladatRadar tehát nem egy általános célú feladatkezelő, de nem is csupán iskolai eszköz, olyan integrált platform, amely személyre szabható bármilyen közösség vagy egyéni felhasználó igényeihez, és ezeket a specifikus funkciókat egyetlen, átlátható felületen egyesíti.

2.2 Telepítési útmutató részletesen

Lépésről lépésre telepítés (lokális fejlesztés)

1. Repository klónozása

Nyisson egy terminált (pl. PowerShell vagy Git Bash), navigáljon a kívánt mappába, majd futtassa: git clone https://github.com/RobiGanz/FeladatRadar.git

A parancs létrehozza a FeladatRadar nevű mappát a gépén, és letölti a teljes forráskódot. Ha nincs Git telepítve, előbb töltse le a git-scm.com oldalról.

2. Solution megnyitása a Visual Studio-ban

FeladatRadar.sln → Visual Studio 2022. Ha a NuGet-visszaállítás nem fut le automatikusan: jobb klikk a Solution-ön → Restore NuGet Packages.

3. Adatbázis inicializálása

SSMS-ben nyissa meg és futtassa a FeladatRadar/database/dump/ures.sql szkriptet – ez létrehozza az adatbázist, a táblákat (üresen) és a tárolt eljárásokat. Amennyiben tesztadatokra is szükség van, futtassa a feltoltes.sql szkriptet is, amely feltölti az adatbázist adatokkal.

4. appsettings.json beállítása (backend)

FeladatRadar.backend/appsettings.json → ConnectionStrings.Default felülírása:

```
// SQL Auth

"Server=GEPNEV\SQLEXPRESS; Database=FeladatRadar; user=sa;
password=JELSZO; TrustServerCertificate=true"

// Windows Auth

"Server=.\SQLEXPRESS; Database=FeladatRadar; Integrated
Security=true; TrustServerCertificate=true"
```

5. Multiple Startup Projects beállítása

Solution → Set Startup Projects → mindkét projekthez Action: Start.

A fejlesztői környezet felállításához az alábbi előre konfigurált felhasználók állnak rendelkezésre helyi tesztelés és fejlesztés céljából:

Szerepkör	Felhasználónév	Jelszó	Leírás
Student (Diák)	tesztdiak	Teszt123!	Alapszintű felhasználó, feladatok, órarend, csoportok
Teacher (Tanár)	tesztanar	Teszt123!	Dolgozat hozzáadása, tanári csoportkezelés
Admin	tesztadmin	Teszt123!	Rendszerkezelés, felhasználó-kezelés, audit napló

jelszavak BCrypt hash-ként tárolódnak az adatbázis PasswordHash mezőjében.

Adatbázis inicializálása

1. Nyissa meg az SSMS-t, és csatlakozzon a helyi SQL Server példányhoz (localhost\SQLEXPRESS vagy egyenértékű).
2. Nyissa meg és futtassa az ures.sql szkriptet (database\dump\ures.sql) ez létrehozza az adatbázissémát és a tárolt eljárásokat.
3. Nyissa meg és futtassa a tablafeltoltes.sql szkriptet – ez feltölti a tesztfelhasználókat és a mintaadatokat.
4. Ellenőrzés: a Users táblában meg kell jelennie a tesztfelhasználóknak.

Alternatíva: a FeladatRadar.bacpac fájl teljes adatbázis-mentésként is visszaállítható az SSMS *Import Data-tier Application* funkciójával.

2.3 Projekt menedzsment és csapatmunka dokumentációja

2.3.1 Csapatösszetétel és feladatkiosztás

A FeladatRadar-t háromfős csapat készítette. A munkamegosztás az alábbiakban részletezett módon alakult. Minden csapattag rendelkezett saját felelősségi körrel, emellett bizonyos modulokat, különösen a tanári és adminisztrátori felületet közösen valósítottuk meg.

Tarcsányi Csongor Márk feladatai

Elsősorban a felhasználói azonosítási rendszer és az autentikációhoz kapcsolódó felületek fejlesztéséért felelt:

- Felhasználói fiókok teljes kezelése: regisztráció, bejelentkezés, jelszókezelés és szerepkör-hozzárendelés
- Bejelentkezési és regisztrációs felület létrehozása (Login.razor, Register.razor): vizuális kialakítás, jelszóerősség-ellenőrzés, szerepkör-választó kártyák
- Főoldal felület megvalósítása (Home.razor)
- Adatbázis-kezelés: felhasználói adatok tárolása, jogosultságok és szerepkörök kezelése
- Tanári funkciók: szavazás-modul és dolgozat-kezelés (TeacherService.cs)
- Admin felület létrehozása (Admin.razor)
- Audit napló megjelenítése és kezelése (AdminService.cs)
- Fiók felület megvalósítása (Fiok.razor): felhasználónév és jelszó módosításának biztosítása, biztonságos adatkezeléssel és validációval.

Lőrincz Antal feladatai

A feladatkezelő és fókusz időzítő modulok fejlesztéséért, valamint a tanári és adminisztrátori backend API-rétegéért felelt:

- Feladatok oldal teljes megvalósítása: típusválasztó, szűrők, keresés, rendezés és mobilnézet (Feladatok.razor, TaskService.cs)
- Fókusz idő funkció: időzítők, munkamenet-napló, beállítások panel és SVG timer ring (FocusTimerService.cs)
- Tanári felületi műveletek API-hívásainak kezelése (TeacherController.cs)
- Felhasználók, csoportok, rendszerstatisztikák és audit naplók adatmodelljei, valamint tanári szavazások, dolgozatok és meghívók adatstruktúrái (TeacherModels.cs)
- Admin és tanári CRUD műveletek megvalósítása (AdminService.cs, TeacherService.cs): lehetőséget biztosít az adatok Létrehozására (Create), Lekérdezésére (Read), Módosítására (Update) és Törlésére (Delete)

Le Tuan Anh (Róbert) feladatai
A tantárgy- és órarendkezelő modul, a naptár frontend és a backend adatbázis-koordináció felelőse volt:

- Tantárgyak és órarendek kezelése: felvétel, törlés, Student és Teacher szerepkörre (Tantargyak.razor, SubjectService.cs)
- Naptár frontend teljes megvalósítása: havi, heti és napi nézet, navigáció, közelgő feladatok panel, új bejegyzés modal és CSS (Naptar.razor)
- Naptár adatok lekérése és összekapcsolása a bejelentkezett felhasználóval (ScheduleService.cs)
- Események kezelése: órák és házi feladatok időbélyegzése és ismétlődési logikája
- Adatbázis-tárolt eljárások koordinálása és SQL szkriptek karbantartása
- Adminisztrátori adatmodellek és statisztikák (AdminModels.cs)
- Szavazások és dolgozatok adatmodelljei (TeacherModels.cs)
- Admin backend API végpontok kezelése (Controllers/AdminController.cs, backend)

Közös fejlesztések

Az alábbi modulokat és funkciókat a csapat közösen tervezte, fejlesztette és tesztelte:

- Főoldal (Dashboard.razor) és Csoportok modul: az adatbázis-kezelést közösen terveztük, hogy minden részadat konzisztensen összekapcsolódjon
- Naptár, feladatkezelés és csoportfunkciók integrációja: a modulok együttműködése lehetővé teszi, hogy diákok és tanárok egyidejűleg használják az összes funkciót
- Felhasználói jogosultságok, eseménymegjelenítés és órarend-ütközések kezelése: folyamatos közös tesztelés és hibajavítás
- Tanári felület (Teacher) és Admin felület teljes körű fejlesztése: a szavazás-modul, dolgozatkezelés és adminisztrátori műveletek tervezése és megvalósítása közösen történt
- Sötét/világos téma (dark mode): a témakezelés közösen lett megtervezve és megvalósítva; a theme.js fájl biztosítja a témaállapot tárolását (localStorage), amelyet minden oldal (Csoport.razor, Tantárgyak.razor, Naptár.razor stb.) egységesen használ a themeHelper.getTheme / themeHelper.setTheme JavaScript-hívásokon keresztül

2.3.2 Kommunikáció és koordináció

A csapat napi szintű kommunikációja Discord szerveren zajlott, ahol külön csatornák foglalkoztak a frontend fejlesztéssel, a backend fejlesztéssel, az adatbázis témájával és a dokumentációval. A kódot kizárólag GitHub-on keresztül osztottuk meg, közvetlenül egymásnak fájlokat nem küldtünk, mert ez verzió-ütközésekhez vezet.

A branch stratégia biztosította, hogy mindenki a saját ágán dolgozhasson, és az összeolvasztás (merge) csak ellenőrzés után történt meg a dev ágba, majd onnan a main-be.

2.3.3 Kihívások és megoldásaik

Blazor Server + JWT autentikáció

Az egyik legnagyobb technikai kihívás a Blazor Server és a saját JWT API közötti autentikáció megvalósítása volt. A Blazor Server alapból ASP.NET Core Identity-t vagy cookie-alapú autentikációt preferál, de mi API tokenes megoldást akartunk, ez nem bevett minta Blazor Server esetén. A megoldás az volt, hogy az AuthService a Blazored.LocalStorage-ba mentette a tokent, és minden Service osztály az InitializeAsync() híváskor betölti és csatolja a HttpClient-hez. Ez "kézzel" kezeli az autentikációs állapotot.

SignalR kapcsolat megszakadása

Blazor Server esetén a böngésző és a szerver között folyamatos WebSocket (SignalR) kapcsolat szükséges. Tesztelés során észrevettük, hogy ha a laptop alvó állapotba megy, majd feléled, a kapcsolat megszakad és az oldal " befagyott" állapotba kerül ("Reconnecting..." üzenettel). Ez Blazor Server ismert korlátja, a felhasználónak ilyenkor frissíteni kell az oldalt. A megoldás hosszabb távon a Blazor WASM-ra való átállás lenne, de a jelenlegi projekt keretein belül ez elfogadható viselkedés.

Dapper és aszinkronitás

Néhány service metódusnál előfordult, hogy a Dapper queryAsync visszatérési értéke null volt, pedig az tárolt eljárás lefutott. Kiderült, hogy bizonyos tárolt eljárások nem SET NOCOUNT ON-nal indultak, így a Dapper a row count üzenetet próbálta eredményként feldolgozni. A javítás minden tárolt eljárás elejéhez hozzáadta a SET NOCOUNT ON utasítást.

CSS theming és Blazor

A sötét/világos mód implementálása trükkös volt Blazor Server esetén, mert a CSS változók data-theme attribútumon alapulnak, de az attribútumot a C# @code blokkból kell módosítani. A lokálisan tárolt témabeállítást az oldal inicializáláskor JS interop segítségével olvassuk be a localStorage-ból, majd a _theme C# változó frissítésével a Razor template automatikusan alkalmazza a helyes CSS-t.

GitHub Actions: Felesleges workflow futtatások és push problémák

A fejlesztés során előfordult, hogy egy push művelet összeakadt, a feltöltés nem ment át rendesen, ami után a csapat elkezdte törölni, visszatölteni a fájlokat, újra pusholni. Ez láncreakcióként több felesleges workflow futtatást indított el ugyanarra a kódállapotra. Emellett előfordultak commit üzenet elírások is, amelyek szintén hozzájárultak a history kissé kaotikus megjelenéséhez.

A felesleges workflow futtatásokat utólag töröltük a GitHub Actions felületről, ez csak a logokat érinti, a kódot vagy a commitokat nem. Néhány régebbi commithoz ezért nem tartozik workflow futás, de ez nem jelent funkcionális problémát, mivel a legutóbbi zöld workflow az összes korábbi commit kódját is tartalmazza és teszteli.

Tanulság: push problémák esetén érdemes először megvizsgálni a hibaüzenetet (git status, git log), és csak szükség esetén beavatkozni, a kapkodó törlés-visszatöltés általában több problémát okoz, mint amennyit megold. A jövőben feature branch-ek használata is segít elkerülni az ilyen helyzeteket, mivel a main ág csak tesztelt, végleges kódot tartalmaz.

SQL Server Management Studio: Szervernév változás

Róbert gépét a fejlesztés során újra kellett telepíteni. Az újratelepítés következtében a Windows gépnév megváltozott, ami érintette az SSMS kapcsolati beállításait is, az SSMS alapértelmezett szervernévként a gépnevet használja Windows Authentication esetén. Ezért az újratelepítés után frissíteni kellett a ConnectionString-et az appsettings.json fájlban.

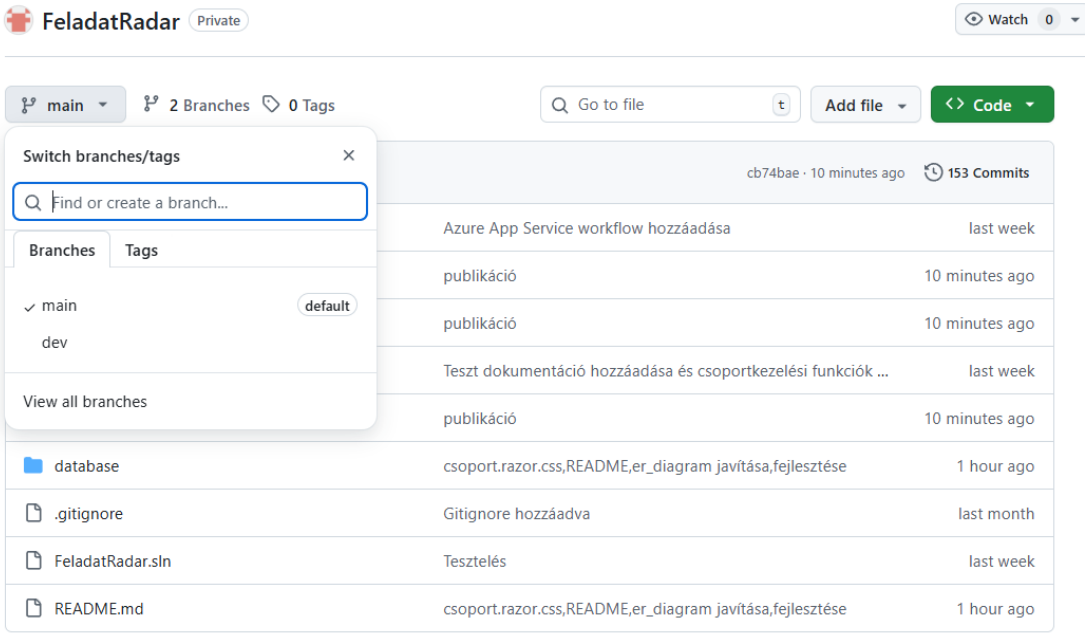
Javaslat a jövőre: az appsettings.json-ban mindig a localhost kulcsszót használjuk a konkrét gépnév helyett, hogy eltérő fejlesztői gépeken is működjön a projekt módosítás nélkül.

2.3.4 Verziókövetés és Git munkafolyamat

Branch stratégia

A projekt fejlesztése branch-alapú Git munkafolyamattal zajlik. A fő ágak:

Branch neve	Szerep
main	Stabil, mindig futásképes kód. Csak merge-elt, tesztelt változtatások kerülnek ide.
dev	Aktív fejlesztési ág. Az új funkciók itt kerülnek integrálásra, mielőtt main-be kerülnek.



A commit üzenetben pontosabban is leírjuk, mit változtattunk és miért, ez segíti a visszakövethetőséget és a code review folyamatot.

GitHub Actions: CI pipeline

A repóban a .github/workflows/dotnet.yml fájl egy automatikus CI (Continuous Integration) pipeline-t definiál, amely minden main ágba érkező push és pull request esetén lefut. A workflow lépései:

1. Kód letöltése (actions/checkout@v4)
2. .NET 8.0 SDK telepítése (actions/setup-dotnet@v4)
3. NuGet függőségek visszaállítása (dotnet restore)
4. Projekt fordítása (dotnet build --no-restore)
5. Tesztek futtatása (dotnet test --no-build --verbosity normal)

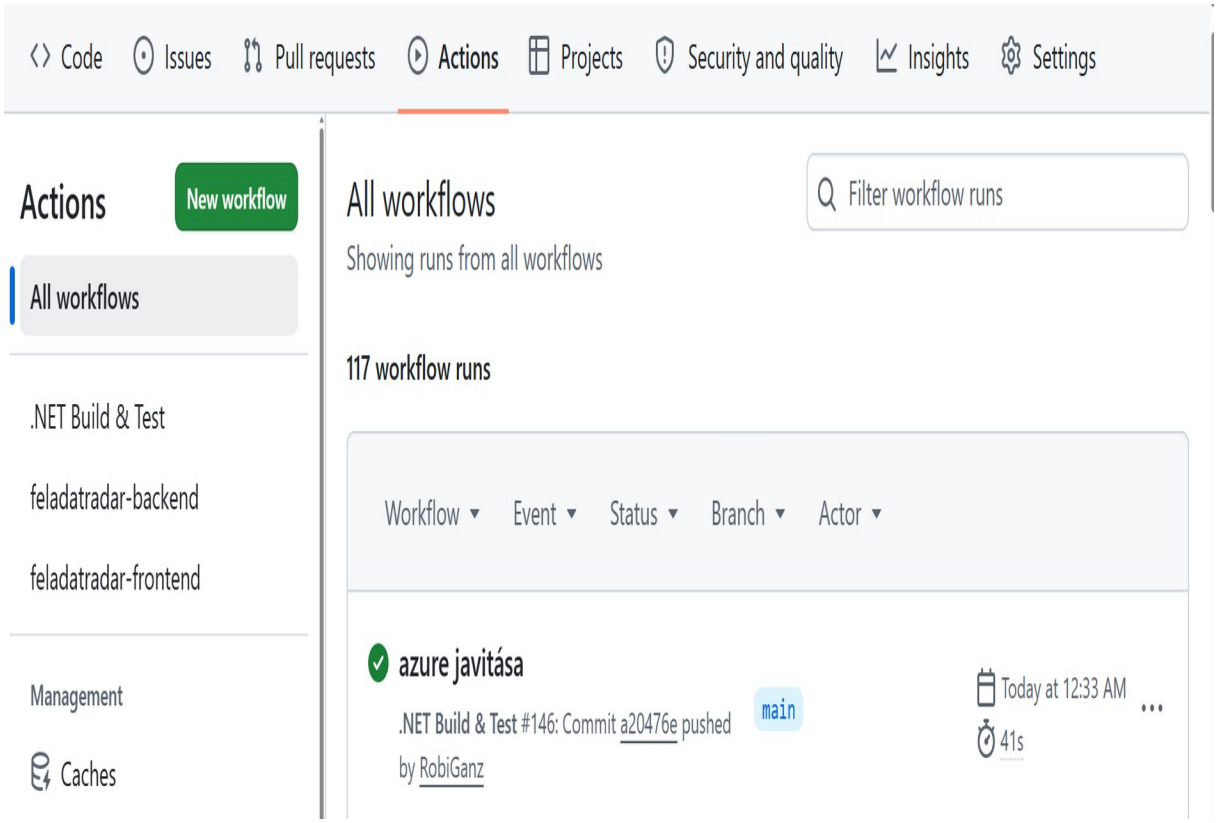
A dotnet test lépés a FeladatRadar.teszt projektben lévő xUnit egységteszteket futtatja. Ez a projekt a FeladatRadarTests.cs fájlban tartalmazza az összes tesztesetet.

Ez biztosítja, hogy minden merge előtt a kód lefordul és az esetleges tesztek lefutnak, így a main ág mindig buildelhető állapotban marad.

GitHub Actions: CD pipeline (Azure deployment)

A main_feladatradar-backend.yml és main_feladatradar-frontend.yml workflow-k manuális CD (Continuous Deployment) pipeline-t valósítanak meg. Az alkalmazás kizárólag manuálisan telepíthető az Azure App Service-re, a GitHub Actions felületén a „Run workflow" gomb segítségével (workflow_dispatch). Automatikus deploy main ágba érkező push esetén nem történik.Az éles alkalmazás az alábbi URL-eken érhető el:

- **Frontend:** https://feladatradar.hu/
- **Backend API:** https://api.feladatradar.hu/



2.4 A fejlesztőkörnyezet

2.4.1 Hardverkövetelmények

A fejlesztés három különböző gépen folyt párhuzamosan. Az alábbiakban a fejlesztői környezeteket ismertetjük:

Hardver	Anti	Csongi	Robi
CPU	Intel Core i5-10400f	Intel Core i5-1135G7	Intel Core i9-12900kf
RAM	32Gb DDR4	24Gb DDR4	32Gb DDR5
GPU	GTX 1050Ti	Intel iris Xe	RTX 3070Ti
Képernyőfelbontás	1920x1080	1920x1080	1920x1080

2.4.2 Felhasznált szoftverek

Szoftver	Verzió	Szerep a projektben
Microsoft Visual Studio 2022	17.x	Fő IDE, backend és frontend fejlesztés
SQL Server 2022 (Developer)	16.x	Adatbázis-szerver
SQL Server Management Studio 21	21.x	Adatbázis-kezelés, tárolt eljárás írás és tesztelés
GitHub	2.51.0	Közös kódtároló, branch-alapú munka
.NET SDK	net8.0	Build és futtatókörnyezet

2.4.3 Programnyelv és keretrendszer választása

A projekt fő programnyelve a C#, keretrendszere az ASP.NET Core 8.0 és a Blazor Server. A választást a következő szempontok indokolták:

- C# és .NET: erős típusosság, kiváló IDE(Integrált Fejlesztői Környezet)-támogatás (Visual Studio), a csapat tagjai az iskolai képzés során megismerték ezt a technológiát, így a tanulási görbe minimális volt.
- ASP.NET Core Web API: iparági standard RESTful API-k fejlesztéséhez, kitűnő JWT-autentikációs és Swagger-támogatással.
- Blazor Server: lehetővé teszi, hogy a frontend C# nyelven készüljön, nem szükséges külön JavaScript keretrendszer (React, Angular) tanulása. A valós idejű SignalR kapcsolatnak köszönhetően az UI interaktív és reszponzív.
- Dapper: könnyű, teljesítményközpontú ORM-könyvtár SQL Server felett. Az Entity Framework Core-hoz képest finomabb SQL-kontrollt biztosít, ami a tárolt eljárásokkal való munkához ideális.
- SQL Server: az iskolai szerveren elérhető, a csapat tagjai ismerik, és a tárolt eljárások (stored procedures) révén az adatlogika egy részét adatbázis-szinten tudjuk kezelni.

A döntés tudatos volt: ez a tech stack lehetővé teszi, hogy az egész alkalmazás egyetlen programnyelven (C#) íródjon, amit a csapat minden tagja ismert és alkalmazott korábban.

Miért Blazor Server és nem WebAssembly?

A választás során mérlegeltük a Blazor Server és a Blazor WebAssembly közötti különbségeket. A Blazor WebAssembly esetén a teljes alkalmazás a böngészőben fut, míg Blazor Server esetén a renderelés a szerveren zajlik és SignalR WebSocket-kapcsolaton keresztül kommunikál a böngészővel. A Blazor Server fő előnyei számunkra:

- A szerver-oldali C# kód közvetlenül elérheti az adatbázis service-eket a Dependency Injection rendszeren keresztül, nincs szükség külön API rétegre a frontend részéhez (bár mi egyébként is szeparált API-t használunk).
- Kisebb kezdeti letöltési méret a felhasználó számára: a Blazor WASM futtatókörnyezet sok megabájt, ami Blazor Server esetén a szerveren van.
- Az összetett állapotkezelés (pl. FocusTimerService singleton) egyszerűbb, mivel a singleton service valóban a szerveren fut, nem minden felhasználónál külön WASM példányban.

A Blazor Server hátránya a folyamatos szerver-kapcsolat igénye, ha a SignalR kapcsolat megszakad, az oldal "elveszíti" az állapotát.

Miért Dapper és nem Entity Framework Core?

Az Entity Framework Core ugyan kényelmes ORM, amely automatikusan generál SQL-t a C# modellekből, de a FeladatRadar esetén az alábbi szempontok a Dapper mellett szóltak:

- Teljes kontroll az SQL felett: a komplex lekérdezések (pl. sp_GetMyTasks határidő-számítással, sp_GetGroupPolls JOIN-okkal) EF Core esetén bonyolultan vagy nem kellő teljesítménnyel oldhatók meg.
- Tárolt eljárások preferálása: a projektben kizárólag tárolt eljárásokat szerettünk volna használni az adateléréshez. Dapper ezeket egyszerűen és hatékonyan kezeli, EF Core esetén ugyanez verbose konfiguráció.

Miért SQL Server és nem PostgreSQL?

Az iskolai teremben SQL Server volt elérhető (Developer Edition ingyenesen). Az SSMS (SQL Server Management Studio) kiváló GUI-t biztosít a tárolt eljárások írásához és debuggolásához. Bár a PostgreSQL nyílt forrású és ingyenes, a váltás szükségtelen pluszmunkát jelentett volna, és a T-SQL szintaxis (amelyet tanultunk) néhány ponton eltér a PL/pgSQL-től.

2.5 Rendszerarchitektúra

2.5.1 A rendszer felépítése

A FeladatRadar háromrétegű architektúrán alapul, amelynek rétegei a következők:

- Adatbázis réteg (SQL Server): Az összes adat tárolása tárolt eljárásokon (Stored Procedures) keresztül érhető el. Ez az architektúra biztosítja az adatlogika centralizálását és a biztonságos adatelérést.
- Üzleti logika réteg (ASP.NET Core Web API): A „FeladatRadar.backend" nevű backend projekt felelős az autentikációért, az üzleti szabályok betartatásáért, az adatok validálásáért és a JWT-tokenek kezeléséért. Ez a réteg RESTful HTTP API-n keresztül kommunikál a frontenddel.
- Prezentációs réteg (Blazor Server): A „FeladatRadar.frontend" projekt a felhasználói felületet kezeli. Blazor Server esetén a renderelés a szerveren zajlik, és a változások SignalR WebSocket-kapcsolaton keresztül jutnak el a böngészőhöz. A frontend saját Service osztályokon keresztül hívja a backend API-t HttpClient segítségével.

A két projekt (FeladatRadar.backend és FeladatRadar.frontend) egymástól teljesen független, külön portszámon futnak:

https://localhost:44359/swagger/index.html

https://localhost:7189/

Ez a szétválasztás lehetővé teszi, hogy a jövőben a backend API-t mobilalkalmazás vagy más kliens is fogyaszthassa.

2.5.2 Projekt- és mappastruktúra

A megoldás (Solution) több, egymással együttműködő projektet és kiegészítő mappát tartalmaz. Ezek közé tartozik a **FeladatRadar.backend** és **FeladatRadar.frontend** alkalmazás, valamint az adatbázis-kezeléshez, dokumentációhoz és egyéb kiegészítő funkciókhoz szükséges állományok.

Az alábbiakban a projekt teljes mappastruktúrája kerül bemutatásra, az egyes elemek rövid leírásával.

FeladatRadar.backend:

Mappa / Fájl	Szerep
Controllers/	HTTP kérések fogadása, válasz visszaküldése
Controllers/BaseController.cs	JWT helper metódusok közös ősosztálya
Controllers/AuthController.cs	Regisztráció és bejelentkezés végpontjai (/api/auth)
Controllers/TaskController.cs	Feladatok CRUD végpontjai (/api/task)
Controllers/ScheduleController.cs	Órarend végpontjai (/api/schedule)
Controllers/SubjectController.cs	Tantárgy végpontjai (/api/subject)
Controllers/GroupController.cs	Csoport végpontjai (/api/group)
Controllers/TeacherController.cs	Tanári funkciók végpontjai (/api/teacher)
Controllers/AdminController.cs	Admin funkciók végpontjai (/api/Admin)
Services/	Üzleti logika, adatbázis-hívások (Dapper)
Services/AuthService.cs	Regisztráció, bejelentkezés, JWT generálás
Services/ScheduleService.cs	Órarend kezelése, ütközés-ellenőrzés
Services/SubjectService.cs	Tantárgy felvétel, leadás, listázás
Services/GroupService.cs	Csoport létrehozás, meghívás, tagkezelés
Services/TeacherService.cs	Szavazás és dolgozat kezelése
Services/AdminService.cs	Felhasználó-kezelés, audit napló, statisztikák
Services/I*.cs	Interfészek a DI regisztrációhoz (IAuthService stb.)
Models/	Adatmodellek és DTO osztályok
Models/User.cs	User, UserDto, LoginRequest, RegisterRequest, LoginResponse
Models/Task.cs	TaskItem, AddTaskRequest
Models/ScheduleEntry.cs	ScheduleEntry, AddScheduleRequest, RecurrenceKind
Models/Subject.cs	Subject, SubjectResponse, EnrollmentItem
Models/GroupModels.cs	Group, GroupMember, GroupInvite, GroupScheduleEntry stb.
Models/TeacherModels.cs	Poll, PollOption, TeacherExam, CreatePollRequest stb.
Models/AdminModels.cs	AdminUserDto, SystemStatsDto, AdminGroupDto, AuditLogEntry
Program.cs	Alkalmazás belépési pontja: DI regisztráció, JWT konfiguráció, CORS, Swagger
appsettings.json	Kapcsolati sztring (ConnectionStrings:Default), JWT beállítások

FeladatRadar.frontend:

Mappa / Fájl	Szerep
Components/Pages/	Blazor oldalak (.razor fájlok)
Components/Pages/Home.razor	Nyitóoldal (nem hitelesített felhasználóknak)
Components/Pages/Login.razor	Bejelentkezési felület
Components/Pages/Register.razor	Regisztrációs felület szerepkör-választóval
Components/Pages/Dashboard.razor	Főoldal: statisztikák, közelgő feladatok
Components/Pages/Naptar.razor	Naptár: havi/heti/napi nézet, órarend és feladatok
Components/Pages/Feladatok.razor	Feladatlista szűréssel, rendezéssel, keresővel
Components/Pages/Focus.razor	Pomodoro időzítő, munkamenet-napló
Components/Pages/Tantargyak.razor	Tantárgy felvétel, leadás, egyedi tantárgy
Components/Pages/Csoport.razor	Csoportkezelés: tagok, feladatok, szavazás, dolgozatok
Components/Pages/Admin.razor	Admin felület: felhasználók, csoportok, audit napló
Components/Pages/Fiok.razor	profil-legördülő komponens: felhasználónév/jelszó módosítás, kijelentkezés
Components/Layout/MainLayout.razor	Közös oldal-elrendezés (sidebar + fő tartalom)
Components/App.razor	Alkalmazás gyökérkomponens, szkript betöltések
Components/Routes.razor	Blazor routing konfiguráció
Services/AuthService.cs	JWT token kezelés, bejelentkezés, kijelentkezés, LocalStorage
Services/TaskService.cs	Feladatok API hívásai, FilteredTasks computed property
Services/ScheduleService.cs	Órarend API hívásai
Services/SubjectService.cs	Tantárgy API hívásai
Services/GroupService.cs	Csoport API hívásai (17 metódus)
Services/TeacherService.cs	Szavazás és dolgozat API hívásai
Services/AdminService.cs	Admin API hívásai
Services/FocusTimerService.cs	Pomodoro időzítő singleton állapotgép
Models/	Frontend DTO osztályok (tükrözik a backend modelleket)
Models/AuthModel.cs	LoginRequest, RegisterRequest, LoginResponse, UserDto
Models/TaskModels.cs	TaskDto (IsOverdue, IsDueSoon, DaysLeft computed property-kkel)

Models/ScheduleModels.cs	ScheduleEntryDto, AddScheduleRequest, RecurrenceLabel
Models/SubjectModels.cs	SubjectDto, EnrollmentDto, SubjectResponse
Models/GroupModels.cs	GroupDto, GroupMemberDto, GroupInviteDto stb.
Models/TeacherModels.cs	PollDto, PollOptionDto, TeacherExamDto stb.
Models/AdminModels.cs	AdminUserDto, SystemStatsDto, AuditLogEntry
wwwroot/app.css	Globális CSS: CSS változók, dark mode, sidebar, layout
wwwroot/theme.js	localStorage-alapú témaváltó és Pomodoro beállítások
wwwroot/bootstrap/	Bootstrap 5 minimális CSS (csak grid és reset)
appsettings.json	Backend BaseAddress beállítása
Program.cs	DI regisztráció: Service-ek, HttpClient, LocalStorage, AuthState

database: adatbázis szkriptek és segédanyagok

Mappa / Fájl	Szerep
er_diagram/	Az ER diagramot tartalmazó almappa.
er_diagram/er_full.html	Az adatbázis ER diagramja HTML formátumban. Böngészőben megnyitható, nagyítható és mozgatható; vizuálisan ábrázolja az összes táblát, mezőt és a köztük lévő kapcsolatokat (PK, FK).
dump/	Adatbázis-szkripteket és mentési fájlt tartalmazó almappa.
dump/ures.sql	Üres adatbázis-szkript: lefuttatva létrehozza a FeladatRadar adatbázist az összes táblával és tárolt eljárással együtt. A telepítés első lépése, SSMS-ben kell megnyitni és végrehajtani.
dump/tablafeltoltes.sql	Tesztadat-feltöltő szkript: felhasználókat, tantárgyakat, feladatokat és egyéb mintaadatokat szúr be az adatbázisba. Fontos: admin fiókot a regisztrációs felületen nem lehet létrehozni (nincs Admin opció a frontenden, és az AuthService.Register szerver oldalon is visszautasítja az Admin szerepkörű regisztrációt), az admin felhasználó csak ezzel a szkripttel, közvetlen SQL-beszúrással, vagy egy meglévő Admin által a szerepkör-módosító felületen hozható létre.
dump/FeladatRadar.bacpac	SQL Server adatbázis-mentés (backup). A teljes adatbázis visszaállítható belőle SSMS-ben a Restore Database funkcióval, gyors alternatíva a dump/ures.sql + dump/tablafeltoltes.sql páros helyett.

Dokumentumok:

Mappa / Fájl	Szerep
Le_Tuan_Anh_Dokumentáció.pdf Tarcsányi_Csongor_Márk_Dokumentáció.pdf	Az egyes csapattagokhoz tartozó egyéni feladatok és felelősségi körök dokumentációját tartalmazza.
FeladatRadar_Dokumentáció.pdf FeladatRadar_Dokumentáció.docx	A dokumentum a projekt működését és funkcióit részletesen ismerteti, átfogó képet adva a rendszer teljes funkcionalitásáról.
Teszt_dokumentáció.pdf	A projekt tesztelési folyamatát, a teszteseteket és az eredményeket részletesen ismerteti.
Vizsgaremek.pptx	A projekt bemutatásához készített prezentáció
Nyilatkozatok.pdf	A csapat tagjainak nyilatkozatai

FeladatRadar.teszt:

Mappa / Fájl	Szerep
FeladatRadarTests.cs	Tesztesetekhez végzett kód

Published:

Mappa / Fájl	Szerep
INSTRUCTIONS.md	Telepítési és futtatási útmutató

2.5.3 Rétegek közötti kommunikáció

A frontend service osztályok (pl. TaskService.cs a FeladatRadar.frontend /Service mappában) HttpClient példányon keresztül küldnek HTTP-kéréseket a backend API végpontjaira. Minden kéréshez hozzáadják az autentikációs tokent:

```
try{
    _token = await _localStorage.GetItemAsync<string>("authToken");
    _currentUser = await _localStorage.GetItemAsync<UserDto>("currentUser");
    if (!string.IsNullOrEmpty(_token))
    {
       _httpClient.DefaultRequestHeaders.Authorization =
          new AuthenticationHeaderValue("Bearer", _token);
    }
    _isInitialized = true;
                    Console.WriteLine($"[AuthService]    InitializeAsync    -    IsAuthenticated:
{IsAuthenticated}");
 }
```

A backend minden védett végponton ellenőrzi a JWT-token érvényességét az [Authorize] attribútummal. A token tartalmaz felhasználói azonosítót (sub claim), felhasználónevet, e-mail-t és szerepkört (role claim), amelyeket a Controller az autentikált HttpContext.User objektumból olvas ki.

2.6 Adatbázis-struktúra

2.6.1 Az adatbázis ismertetése

Az adatbázis neve: FeladatRadar. SQL Server 2022-es (Developer Edition) szerveren fut, helyi (localhost) telepítésben. Az adatmanipuláció tárolt eljárásokon (Stored Procedures) keresztül történik.

A Dapper mini-ORM segítségével a Service osztályok DynamicParameters objektumba csomagolják a paramétereket, majd a connection.QueryAsync<T> vagy connection.QueryFirstOrDefaultAsync<dynamic> metódusok hívják meg a tárolt eljárásokat.

2.6.2 Táblák részletes leírása

Az alábbiakban az összes tábla részletes leírása következik, az ures.sql exportált séma alapján. A mezőtípusok és korlátok a tényleges CREATE TABLE utasításokból származnak.

Users tábla

A rendszer összes felhasználóját tárolja. Három szerepkör létezik: Student, Teacher és Admin. A UserRole mezőre CHECK constraint biztosítja, hogy csak ezek az értékek kerülhessenek be. Az IsActive mező a soft-delete mechanizmust támogatja: törlés helyett a felhasználó deaktiválható (IsActive = 0), így az idegen kulcs hivatkozások épek maradnak.

Mező neve	Típus	Hossz / Korlát	Leírás
UserID	INT	PK, IDENTITY(1,1)	Egyedi azonosító (auto-increment)
Username	NVARCHAR	50, NOT NULL, UNIQUE	Bejelentkezési név (egyedi)
PasswordHash	NVARCHAR	255, NOT NULL	BCrypt hash (cost=11)
Email	NVARCHAR	100, NULL	E-mail cím
FirstName	NVARCHAR	50, NULL	Keresztnév
LastName	NVARCHAR	50, NULL	Vezetéknév
UserRole	NVARCHAR	20, CHECK IN ('Student','Teacher','Admin')	Szerepkör
CreatedAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Regisztráció időpontja
IsActive	BIT	NOT NULL, DEFAULT 1	Aktív fiók (1) vagy letiltott (0)

Subjects tábla

Az elérhető tantárgyakat tárolja. A tantárgy lehet globális (bárki által felvehető) vagy privát (IsPrivate = 1), amelyet a diák maga hozott létre az sp_EnrollCustomSubject tárolt eljárással.

Mező neve	Típus	Hossz / Korlát	Leírás
SubjectID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
SubjectName	NVARCHAR	100, NOT NULL	Tantárgy neve
SubjectCode	NVARCHAR	20, NULL	Tantárgykód (pl. MATE-101)
TeacherID	INT	FK → Users, NULL	Felelős tanár (opcionális)
IsActive	BIT	NOT NULL, DEFAULT 1	Aktív tantárgy-e
CurrentEnrollment	INT	NOT NULL, DEFAULT 0	Jelenlegi beiratkozók
CreatedBy	INT	NULL	Létrehozó UserID (privát esetén)
IsPrivate	BIT	NOT NULL, DEFAULT 0	Privát tantárgy (1 = csak a létrehozó látja)

Enrollments tábla

Tárolja, hogy melyik diák melyik tantárgyat vette fel. Az sp_EnrollSubject tárolt eljárás a duplikációt a StudentSubjects táblán ellenőrzi, majd növeli a Subjects tábla CurrentEnrollment értékét.Soha nem töltődik fel a C# kódból, a StudentSubjects a valódi beiratkozási tábla.

Mező neve	Típus	Hossz / Korlát	Leírás
EnrollmentID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
StudentID	INT	NOT NULL	Diák azonosítója
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy azonosítója
EnrolledAt	DATETIME	DEFAULT GETDATE()	Beiratkozás időpontja

StudentSubjects tábla

Kapcsolótábla a diákok és tantárgyak között. Az sp_EnrollSubject és sp_DropSubject tárolt eljárások kezelik a felvett tantárgyak nyilvántartását. Ez a tábla különáll az Enrollments táblától: míg az Enrollments a beiratkozási eseményt rögzíti, a StudentSubjects az aktuálisan felvett tantárgy-kapcsolatokat tartja nyilván.

Mező neve	Típus	Hossz / Korlát	Leírás
StudentSubjectID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
StudentID	INT	NOT NULL, FK → Users	Diák azonosítója
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy azonosítója

ScheduleEntries tábla

A diákok egyéni órarendjének bejegyzéseit tárolja. Támogatja az ismétlődési mintákat (heti, kéthetente, havi). Az sp_AddScheduleEntry tárolt eljárás ütközés-ellenőrzést végez: ha az adott napon és időpontban már létezik bejegyzés, hibaüzenetet ad vissza.

Mező neve	Típus	Hossz / Korlát	Leírás
EntryID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
StudentID	INT	NOT NULL, FK → Users	Diák azonosítója
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy azonosítója
DayOfWeek	INT	NOT NULL	1=Hétfő, 2=Kedd, … 7=Vasárnap
StartTime	TIME(7)	NOT NULL	Óra kezdete (pl. 08:00)
EndTime	TIME(7)	NOT NULL	Óra vége (pl. 09:30)
Location	NVARCHAR	100, NULL	Terem / helyszín
RecurrenceType	NVARCHAR	20, NOT NULL, DEFAULT 'None'	None / Weekly / Biweekly / Monthly
RecurrenceEndDate	DATE	NULL	Ismétlődés vége
StartDate	DATE	NULL	Kéthetente/Havonta: ettől a dátumtól számítja az ismétlődést.
CreatedAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Létrehozás időpontja

Groups tábla

A tanulói és tanári csoportokat tárolja. Minden csoportnak van egy tulajdonosa (CreatedBy), aki automatikusan taggá is válik az sp_CreateGroup tárolt eljárás futtatásakor. A csoport típusa (diák- vagy tanári csoport) a tulajdonos UserRole mezőjéből adódik implicit módon, külön GroupType mező nem létezik.

Mező neve	Típus	Hossz / Korlát	Leírás
GroupID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupName	NVARCHAR	100, NOT NULL	Csoport neve
CreatedBy	INT	NOT NULL, FK → Users	Tulajdonos (létrehozó) UserID
CreatedAt	DATETIME	DEFAULT GETDATE()	Létrehozás időpontja

Tasks tábla

A diákok egyéni feladatait tárolja. Minden feladat egy felhasználóhoz (CreatedBy) és opcionálisan egy tantárgyhoz (SubjectID) kötődik. A TaskType mező szabadon adható meg. A ParentTaskID self-referencing FK az ismétlődő feladatok sorozat-összerendelésére szolgál.

Mező neve	Típus	Hossz / Korlát	Leírás
TaskID	INT	PK, IDENTITY(1,1)	Egyedi azonosító (auto-increment)
CreatedBy	INT	NOT NULL, FK → Users	Létrehozó felhasználó
SubjectID	INT	NULL, FK → Subjects	Tantárgy (opcionális)
Title	NVARCHAR	200, NOT NULL	Feladat neve
Description	NVARCHAR	MAX, NULL	Részletes leírás
DueDate	DATETIME	NULL	Határidő
TaskType	NVARCHAR	50, NULL	Feladat típusa
IsCompleted	BIT	NOT NULL, DEFAULT 0	0 = elvégzendő, 1 = kész
CreatedAt	DATETIME	NULL	Létrehozás időpontja
RecurrenceType	NVARCHAR	20, NOT NULL, DEFAULT 'None'	None / Weekly / Biweekly / Monthly
RecurrenceEndDate	DATE	NULL	Ismétlődés végének dátuma
ParentTaskID	INT	NULL, FK → Tasks	Ismétlődő sorozat gyökere (self-ref)

GroupMembers tábla

Tárolja a csoport tagjait. Az elsődleges kulcs összetett: (GroupID, StudentID), így egy felhasználó csak egyszer lehet tagja egy csoportnak. A StudentID elnevezés eltérő a várttól: valójában bármilyen UserID-t elfogad (tanár esetén is), az elnevezés történelmi okokból maradt.

Mező neve	Típus	Hossz / Korlát	Leírás
GroupID	INT	PK (rész), NOT NULL, FK → Groups	Csoport azonosítója
StudentID	INT	PK (rész), NOT NULL, FK → Users	Tag azonosítója
JoinedAt	DATETIME	DEFAULT GETDATE()	Csatlakozás időpontja

GroupInvites tábla

A csoportba való meghívásokat tárolja. A meghívás e-mail címre történik (InvitedEmail), nem UserID-ra. Az IsAccepted mező BIT típusú: 0 = függőben, 1 = elfogadva/elutasítva. A Status mező NVARCHAR(20) típusú (alapértelmezett értéke ‘Pending’), amelyet az sp_AcceptGroupInvite (‘Accepted’) és sp_DeclineGroupInvite (‘Declined’) tárolt eljárások frissítenek. Elutasításkor a sor nem törlődik, hanem IsAccepted = 1 és Status = ‘Declined’ értékre frissül, így az előzmény megmarad.

Mező neve	Típus	Hossz / Korlát	Leírás
InviteID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupID	INT	NOT NULL, FK → Groups	Melyik csoportba szól
InvitedEmail	NVARCHAR	200, NOT NULL	Meghívott e-mail címe
InvitedBy	INT	NOT NULL, FK → Users	Meghívó felhasználó azonosítója
IsAccepted	BIT	DEFAULT 0	0 = függőben, 1 = elfogadva
Status	NVARCHAR(20)	NOT NULL, DEFAULT (‘Pending’)	Meghívás státusza (sp_InviteToGroup, sp_AcceptGroupInvite használja)
CreatedAt	DATETIME	DEFAULT GETDATE()	Meghívás időpontja

GroupScheduleEntries tábla

A csoport közös órarendjének bejegyzéseit tárolja. Tanári csoportban csak a tulajdonos tanár adhat hozzá bejegyzést (ellenőrzés az sp_AddGroupScheduleEntry-ben). Diáki csoportban bármely tag hozzáadhat. A GroupID-ra ON DELETE CASCADE van beállítva.

Mező neve	Típus	Hossz / Korlát	Leírás
EntryID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupID	INT	NOT NULL, FK → Groups (CASCADE)	Csoportazonosító
AddedBy	INT	NOT NULL, FK → Users	Ki adta hozzá
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy azonosító
DayOfWeek	INT	NOT NULL	1=Hétfő … 7=Vasárnap
StartTime	TIME(7)	NOT NULL	Kezdési idő
EndTime	TIME(7)	NOT NULL	Befejezési idő
Location	NVARCHAR	100, NULL	Helyszín
RecurrenceType	NVARCHAR	20, NOT NULL, DEFAULT 'Weekly'	None / Weekly / Biweekly / Monthly
RecurrenceEndDate	DATE	NULL	Ismétlődés vége
StartDate	DATE	NULL	Az első előfordulás pontos dátuma, ettől számítja az ismétlődést
CreatedAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Létrehozás időpontja

GroupTasks tábla

A csoporton belüli feladatokat tárolja. A csoportfeladatok külön táblában vannak az egyéni feladatoktól (Tasks), mert a csoportfeladatokhoz GroupID is szükséges. A TaskType mező alapbeállítása ‘Exam’. A GroupID-ra ON DELETE CASCADE érvényesül.

Mező neve	Típus	Hossz / Korlát	Leírás
TaskID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupID	INT	NOT NULL, FK → Groups (CASCADE)	Csoportazonosító
AddedBy	INT	NOT NULL, FK → Users	Hozzáadó felhasználó
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy azonosítója
Title	NVARCHAR	200, NOT NULL	Feladat neve
Description	NVARCHAR	1000, NULL	Részletes leírás
DueDate	DATETIME	NOT NULL	Határidő
TaskType	NVARCHAR	50, NOT NULL, DEFAULT 'Exam'	Feladat típusa
IsCompleted	BIT	NOT NULL, DEFAULT 0	Teljesítve-e
CreatedAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Létrehozás időpontja

Polls tábla

A csoporton belüli szavazások fejadatait tárolja. Minden szavazáshoz tartozik egy kérdés (Question), egy létrehozó (CreatedBy), egy opcionális lejárati idő (ExpiresAt) és egy aktív állapot (IsActive). A GroupID-ra ON DELETE CASCADE érvényesül.

Mező neve	Típus	Hossz / Korlát	Leírás
PollID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupID	INT	NOT NULL, FK → Groups (CASCADE)	Csoportazonosító
CreatedBy	INT	NOT NULL, FK → Users	Létrehozó
Question	NVARCHAR	500, NOT NULL	Szavazás kérdése
CreatedAt	DATETIME2	DEFAULT GETDATE()	Létrehozás időpontja
ExpiresAt	DATETIME2	NULL	Lejárati idő
IsActive	BIT	DEFAULT 1	Aktív szavazás-e

PollOptions tábla

A szavazási opciókat tárolja. Minden opció egy szavazáshoz (PollID) tartozik. A SortOrder mező a megjelenítési sorrendet határozza meg. ON DELETE CASCADE: ha a szavazás törlődik, az opciók is.

Mező neve	Típus	Hossz / Korlát	Leírás
OptionID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
PollID	INT	NOT NULL, FK → Polls (CASCADE)	Szavazás azonosító
OptionText	NVARCHAR	200, NOT NULL	Választási lehetőség szövege
SortOrder	INT	DEFAULT 0	Sorrend

PollVotes tábla

A leadott szavazatokat tárolja. Az UQ_PollVote UNIQUE constraint biztosítja, hogy egy felhasználó szavazásonként legfeljebb egyszer szavazhasson. Az sp_VotePoll tárolt eljárás upsert logikát alkalmaz: ha már létezik szavazat, UPDATE-tel módosítja, egyébként INSERT-tel hozza létre.

Mező neve	Típus	Hossz / Korlát	Leírás
VoteID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
PollID	INT	NOT NULL, FK → Polls (CASCADE)	Szavazás azonosító
OptionID	INT	NOT NULL, FK → PollOptions	Választott opció
UserID	INT	NOT NULL, FK → Users	Szavazó felhasználó
VotedAt	DATETIME2	DEFAULT GETDATE()	Szavazás időpontja

Exams tábla

Az Exams tábla elavult (DEPRECATED): az alkalmazás nem használja. A csoportos dolgozatkezelést a TeacherExams tábla végzi, amely közvetlenül egy csoporthoz köti a dolgozatokat, így a csoport összes tagja láthatja azokat.

Mező neve	Típus	Hossz / Korlát	Leírás
ExamID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
TeacherID	INT	NOT NULL, FK → Users	Tanár azonosító
SubjectID	INT	NOT NULL, FK → Subjects	Tantárgy
Title	NVARCHAR	200, NOT NULL	Vizsga neve
ExamDate	DATETIME	NOT NULL	Vizsga időpontja
Duration	INT	NULL	Időtartam (perc)
Notes	NVARCHAR	MAX, NULL	Megjegyzések
CreatedAt	DATETIME	DEFAULT GETDATE()	Létrehozás időpontja

TeacherExams tábla

A tanárok által csoportnak meghirdetett dolgozatokat tárolja. A SubjectName szabad szöveg, nem FK hivatkozás, így a tanár tetszőleges tantárgynevet írhat. A GroupID-ra ON DELETE CASCADE érvényes.

Mező neve	Típus	Hossz / Korlát	Leírás
ExamID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
GroupID	INT	NOT NULL, FK → Groups (CASCADE)	Csoportazonosító
CreatedBy	INT	NOT NULL, FK → Users	Tanár UserID-ja
Title	NVARCHAR	200, NOT NULL	Dolgozat neve
Description	NVARCHAR	1000, NULL	Leírás
ExamDate	DATETIME2	NOT NULL	Dolgozat időpontja
SubjectName	NVARCHAR	100, NULL	Tantárgy neve (szabad szöveg)
CreatedAt	DATETIME2	DEFAULT GETDATE()	Létrehozás időpontja

AuditLog tábla

Az admin műveletek naplózására szolgál. Minden admin írási művelet automatikusan rögzítésre kerül az sp_Admin_WriteAuditLog tárolt eljáráson keresztül. A UserID mezőre ON DELETE SET NULL beállítás van, így ha a felhasználó törlődik, a naplóbejegyzés megmarad.

Mező neve	Típus	Hossz / Korlát	Leírás
LogID	INT	PK, IDENTITY(1,1)	Egyedi azonosító
UserID	INT	NULL, FK → Users (SET NULL)	Admin felhasználó
Username	NVARCHAR	50, NULL	Admin neve (pillanatkép)
Action	NVARCHAR	200, NOT NULL	Művelet típusa
Details	NVARCHAR	MAX, NULL	Részletek
CreatedAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Időbélyeg

2.6.3 Táblák kapcsolatai

Az alábbi táblázat összefoglalja a legfontosabb idegen kulcs kapcsolatokat a valós SQL séma alapján, a CASCADE szabályok feltüntetésével:

Tábla	FK mező	Hivatkozott	Kapcsolat / Törlési szabály
Enrollments	SubjectID	Subjects(SubjectID)	N:M közbülső
StudentSubjects	StudentID	Users(UserID)	N:M közbülső
StudentSubjects	SubjectID	Subjects(SubjectID)	N:M közbülső
Subjects	TeacherID	Users(UserID)	N:1, NULL
ScheduleEntries	StudentID	Users(UserID)	1:N
ScheduleEntries	SubjectID	Subjects(SubjectID)	N:1
Tasks	CreatedBy	Users(UserID)	1:N
Tasks	SubjectID	Subjects(SubjectID)	N:1, NULL
Tasks	ParentTaskID	Tasks(TaskID)	Self-referencing
Groups	CreatedBy	Users(UserID)	N:1
GroupMembers	GroupID	Groups(GroupID)	Összetett PK
GroupMembers	StudentID	Users(UserID)	Összetett PK
GroupInvites	GroupID	Groups(GroupID)	1:N
GroupInvites	InvitedBy	Users(UserID)	N:1
GroupScheduleEntries	GroupID	Groups(GroupID)	ON DELETE CASCADE
GroupScheduleEntries	AddedBy	Users(UserID)	N:1
GroupTasks	GroupID	Groups(GroupID)	ON DELETE CASCADE
Polls	GroupID	Groups(GroupID)	ON DELETE CASCADE
Polls	CreatedBy	Users(UserID)	N:1
PollOptions	PollID	Polls(PollID)	ON DELETE CASCADE
PollVotes	PollID	Polls(PollID)	ON DELETE CASCADE
PollVotes	OptionID	PollOptions(OptionID)	N:1
PollVotes	UserID	Users(UserID)	N:1
TeacherExams	GroupID	Groups(GroupID)	ON DELETE CASCADE
TeacherExams	CreatedBy	Users(UserID)	N:1
Exams	TeacherID	Users(UserID)	N:1
Exams	SubjectID	Subjects(SubjectID)	N:1
AuditLog	UserID	Users(UserID)	ON DELETE SET NULL

2.6.4 Tárolt eljárások

A projekt kizárólag tárolt eljárásokon keresztül kezeli az adatokat. Az alábbi táblázat összefoglalja az összes tárolt eljárást és azok szerepét:

Tárolt eljárások nevei	Modul	Leírás
sp_AcceptGroupInvite	Csoport	Meghívás elfogadása
sp_AddGroupScheduleEntry	Csoport	Csoport-órarend bejegyzés hozzáadása; támogatja a RecurrenceType, RecurrenceEndDate, StartDate paramétereket
sp_AddGroupTask	Csoport	Csoportos feladat hozzáadása
sp_AddPollOption	Szavazás	Szavazási opció hozzáadása
sp_AddScheduleEntry	Órarend	Új órarend-bejegyzés hozzáadása; fogadja a @StartDate, @RecurrenceType, @RecurrenceEndDate paramétereket; Biweekly/None típusnál az ütközés-ellenőrzés kihagyásra kerül
sp_AddTask	Feladatok	Új feladat hozzáadása (ismétlődési logiával)
sp_Admin_ChangeUserRole	Admin	Felhasználó szerepkörének módosítása
sp_Admin_DeleteExam	Admin	Dolgozat törlése (admin)
sp_Admin_DeleteGroup	Admin	Csoport törlése (admin)
sp_Admin_DeletePoll	Admin	Szavazás törlése (admin)
sp_Admin_DeleteTask	Admin	Feladat törlése (admin)
sp_Admin_DeleteUser	Admin	Felhasználó törlése
sp_Admin_GetAllGroups	Admin	Összes csoport listája
sp_Admin_GetAllUsers	Admin	Összes felhasználó listája
sp_Admin_GetGroupExams	Admin	Egy csoport összes dolgozatának listázása (admin számára)
sp_Admin_GetAuditLog	Admin	Audit napló lekérése
sp_Admin_GetSystemStats	Admin	Rendszerstatisztikák lekérése
sp_Admin_ToggleUserActive	Admin	Felhasználó aktiválása / deaktiválása
sp_Admin_WriteAuditLog	Admin	Audit napló bejegyzés írása
sp_Admin_RenameUser	Admin	Felhasználó nevének (FirstName, LastName) módosítása
sp_CompleteTask	Feladatok	Feladat megjelölése befejezettként
sp_UncompleteTask	Feladatok	Feladat visszajelölése nem teljesítettre
sp_CreateGroup	Csoport	Új csoport létrehozása
sp_CreatePoll	Szavazás	Szavazás létrehozása csoportban

Tárolt eljárások nevei	Modul	Leírás
sp_CreateTeacherExam	Dolgozat	Tanári dolgozat létrehozása csoportnak
sp_DeclineGroupInvite	Csoport	Meghívás elutasítása
sp_DeletePoll	Szavazás	Szavazás törlése (csak létrehozó)
sp_DeleteScheduleEntry	Órarend	Órarend-bejegyzés törlése (csak saját)
sp_DeleteTask	Feladatok	Feladat törlése (csak saját)
sp_DeleteTeacherExam	Dolgozat	Dolgozat törlése (csak létrehozó)
sp_DropSubject	Tantárgy	Tantárgy leadása
sp_EnrollCustomSubject	Tantárgy	Egyedi (nem listából választott) tantárgy felvétele
sp_EnrollSubject	Tantárgy	Tantárgy felvétele
sp_GetAllSubjects	Tantárgy	Összes tantárgy listája
sp_GetAvailableSubjects	Tantárgy	Felvehető tantárgyak listája
sp_GetGroupExams	Dolgozat	Csoport dolgozatainak listája
sp_GetGroupMembers	Csoport	Csoport tagjainak listája
sp_GetGroupPolls	Szavazás	Csoport szavazásainak listája
sp_GetGroupSchedule	Csoport	Csoport közös órarendje; visszaadja a RecurrenceType, RecurrenceEndDate, StartDate, CreatedAt mezőket
sp_GetGroupSubjects	Tantárgy	Csoport tantárgyainak listája (StudentSubjects tábla alapján)
sp_GetGroupTasks	Csoport	Csoport tagjainak feladatai
sp_GetMyEnrollments	Tantárgy	Adott diák felvett tantárgyai
sp_GetMyGroups	Csoport	Felhasználó csoportjainak listája
sp_GetMyInvites	Csoport	Beérkező meghívások listája
sp_GetMySchedule	Órarend	Diák saját órarendjének lekérése; visszaadja a StartDate mezőt az ismétlődési logikához
sp_GetMyTasks	Feladatok	Diák saját feladatainak lekérése határidő-statuszsal
sp_GetUserById	Auth	Felhasználó lekérése ID alapján
sp_InviteToGroup	Csoport	Tag meghívása e-mail alapján
sp_LeaveGroup	Csoport	Csoportból való kilépés
sp_LoginUser	Auth	Felhasználó adatainak lekérése bejelentkezéshez
sp_RegisterUser	Auth	Új felhasználó regisztrálása, duplikáció-ellenőrzéssel
sp_VotePoll	Szavazás	Szavazat leadása

Tárolt eljárások nevei	Modul	Leírás
sp_UpdateUsername	Auth	Felhasználónév módosítása (foglaltság-ellenőrzéssel)
sp_ChangePassword	Auth	Jelszó módosítása (BCrypt hash újragenerálással)
sp_DeleteGroupTask	Csoport	Csoportos feladat törlése (hozzáadó vagy tulajdonos)
sp_DeleteGroupScheduleEntry	Csoport	Csoport órarend-bejegyzés törlése (hozzáadó vagy tulajdonos)

2.7 Autentikáció és jogosultságkezelés

2.7.1 JWT-token alapú autentikáció

A FeladatRadar JWT (JSON Web Token) alapú autentikációt alkalmaz. A bejelentkezés folyamata a következő:

1. A felhasználó a /api/auth/login végpontra POST-kérést küld a felhasználónevével és jelszavával.
2. A backend az sp_LoginUser tárolt eljárással lekéri a felhasználó adatait az adatbázisból.
3. A BCrypt.Net könyvtár Verify() metódusával összehasonlítja a beküldött jelszót a tárolt hash értékkel.
4. Sikeres ellenőrzés esetén a GenerateJwtToken() metódus elkészíti a tokent, amely tartalmazza: a felhasználó ID-ját (sub claim), nevét, e-mailjét, szerepkörét (role claim) és az egyedi Jti azonosítót.
5. A token a frontend oldalán Blazored.LocalStorage segítségével kerül tárolásra (authToken kulcs alatt).
6. Minden következő API-kérés Bearer tokenként csatolja ezt a tokent az Authorization fejlécben.

A token érvényességi ideje az appsettings.json fájlban konfigurálható (ExpirationMinutes). A ClockSkew értéke nulla, tehát a token pontosan a megadott időpontban jár le, nem 5 perc késéssel.

2.7.2 Szerepkör-alapú hozzáférés-szabályozás (RBAC)

A FeladatRadar három szerepkört különböztet meg:

Szerepkör	Leírás	Elérhető végpontok
Student	Diák: az alapfelhasználó	Saját feladatok, órarend, tantárgyak, csoportok
Teacher	Tanár: kibővített jogok	Dolgozat és szavazás létrehozása csoportban
Admin	Adminisztrátor	Felhasználó-kezelés (szerepkör-módosítás, letiltás, törlés), csoportok moderálása, szavazások és dolgozatok törlése, audit napló

2.7.3 Jelszókezelés: BCrypt

A jelszavak tárolása BCrypt.Net könyvtárral történik, amelynek HashPassword() metódusa automatikusan véletlenszerű sót (salt) generál, és azt is beágyazza a hashbe. Ez azt jelenti, hogy ugyanaz a jelszó minden regisztrációnál más hash értéket kap, ami megakadályozza a rainbow-table alapú támadásokat. A jelszó-ellenőrzés a BCrypt.Net.BCrypt.Verify(plainText, hash) metódussal történik, amelynek bemenete a beírt jelszó és a tárolt hash, kimenete egy boolean érték.

```
// Regisztrációkor:
string passwordHash = BCrypt.Net.BCrypt.HashPassword(request.Password);
// Bejelentkezéskor:
bool isValid = BCrypt.Net.BCrypt.Verify(request.Password, storedHash);
```

2.7.4 SQL Injection védelem

A Dapper paraméterezett lekérdezései és a tárolt eljárások használata jelentős SQL injection védelmet nyújtanak. A backend kizárólag DynamicParameters objektumon keresztül továbbítja az adatokat:

```
var parameters = new DynamicParameters();
parameters.Add("@Username", request.Username);
parameters.Add("@PasswordHash", passwordHash);

var results = await connection.QueryAsync<dynamic>(
    "sp_RegisterUser",
    parameters,
    commandType: CommandType.StoredProcedure
);
```

Soha nem fordul elő sztring-összefűzéses SQL utasítás. Ez a minta védelmet nyújt az első és másodrendű SQL injection ellen is.

2.7.5 CORS konfiguráció

Mivel a frontend és a backend két külön porton fut, a Program.cs-ben környezetfüggő CORS policy van konfigurálva (DevPolicy).

Fejlesztői környezetben (IsDevelopment()) az API bármely originről fogad kéréseket (AllowAnyOrigin()), így a helyi fejlesztés akadálymentesen működik (https://localhost:44359 és https://localhost:7189).

Éles környezetben a CORS csak a következő két engedélyezett origint fogadja el:

- https://feladatradar.hu

Ez megakadályozza, hogy illetéktelen domainek közvetlenül hívják az API végpontokat.

2.7.6 Biztonsági megoldások részletes ismertetése

A FeladatRadar több rétegű biztonsági mechanizmust alkalmaz. Ebben a fejezetben összefoglaljuk az egyes biztonsági intézkedéseket.

2.7.6.1 Jelszótárolás: BCrypt

A felhasználók jelszavai soha nem kerülnek nyersben tárolásra. Az AuthService a BCrypt.Net NuGet csomagot használja a hash-eléshez. A BCrypt előnyei:

• Automatikus salt generálás: Minden jelszóhoz egyedi, véletlen salt készül, így két azonos jelszó esetén is különböző hash keletkezik.

• Adaptív cost factor: A cost paraméter (alapértelmezett: 11) meghatározza az iterációk számát. A 11-es érték 2^{11} = 2048 iterációt jelent, ami megfelelő egyensúlyt biztosít a biztonság és a teljesítmény között.

• Időzítés-biztos összehasonlítás: A BCrypt.Verify() metódus konstans időben fut, megelőzve a timing attack támadásokat.

2.7.6.2 JWT token biztonság

A JWT tokenek az alábbi biztonsági jellemzőkkel rendelkeznek:

• HMAC-SHA256 aláírás: A token hitelességét szimmetrikus kulccsal biztosítjuk. A kulcs az appsettings.json-ból származik és nem kerül a verziókövetésbe.

• Lejárati idő (exp claim): A token automatikusan lejár a konfigurált idő után (ExpirationMinutes beállítás).

• Szerepkör-alapú jogosultságkezelés: A ClaimTypes.Role claim a [Authorize(Roles = "...")] attribútummal kombinálva biztosítja, hogy csak a megfelelő szerepkörű felhasználók érhetik el a védett végpontokat.

• Egyedi token ID (jti claim): Minden tokenhez egyedi GUID rendelhető, ami jövőbeli token-visszavonási mechanizmus alapja lehet.

2.7.6.3 Szerep-alapú hozzáférés-szabályozás (RBAC)

A rendszer három szinten alkalmaz RBAC-ot:

1. Adatbázis szint: A Users tábla UserRole mezőjére CHECK constraint van, amely csak a Student, Teacher és Admin értékeket engedélyezi.

2. API szint: A Controller osztályok [Authorize(Roles = "Teacher")] vagy [Authorize(Roles = "Admin")] attribútumokkal védik a végpontokat.

3. Tárolt eljárás szint: Az Admin tárolt eljárások (sp_Admin_*) belső ellenőrzést is végeznek: ha a hívó UserRole nem Admin, hibaüzenetet adnak. Továbbá az sp_AddGroupScheduleEntry ellenőrzi, hogy tanári csoportban csak a tulajdonos tanár adhasson hozzá órarend-bejegyzést.

2.8 Főbb algoritmusok

2.8.1 Ismétlődő feladatok kezelése

A FeladatRadar egyik legösszetettebb funkciója az ismétlődő bejegyzések kezelése. Amikor egy felhasználó heti rendszerességgel ismétlődő feladatot vagy órarend-bejegyzést hoz létre, a backend nem egyetlen rekordot szúr be, hanem a RecurrenceEndDate értékig az összes előfordulást külön-külön. A feladatoknál a ParentTaskID mező megmutatja, hogy melyik az adott sorozat gyökér-rekordja, ez az első (alapfeladat) TaskID-ja lesz minden generált másolatban.

A négyféle ismétlődési típus:

RecurrenceType	Magyar neve	Lépés
None	Egyszeri	Csak 1 rekord kerül be
Weekly	Heti	Minden 7. nap egy új rekord
Biweekly	Kéthetente	Minden 14. nap egy új rekord
Monthly	Havonta	Következő hónap ugyanazon napja

Az sp_AddTask tárolt eljárás egy WHILE-cikluson belül INSERT utasításokat hajtogat a paraméterül kapott RecurrenceEndDate-ig. Az algoritmus lépései:

1. Az alap (szülő) feladat beszúrása a Tasks táblába. A SCOPE_IDENTITY() visszaadja az új TaskID-t, ami a @ParentID változóba kerül.

2. Ha a RecurrenceType értéke nem 'None' ÉS a RecurrenceEndDate meg van adva, egy WHILE ciklus indul:

a) A @NextDate változót a RecurrenceType alapján lépteti: Weekly = +7 nap, Biweekly = +14 nap, Monthly = +1 hónap.

b) Ha @NextDate meghaladja a @RecurrenceEndDate-et, a ciklus leáll.

c) Egyébként beszúr egy új gyermek feladatot, amelynek ParentTaskID = @ParentID.

d) Biztonsági korlát: maximum 52 iteráció (1 év heti ismétlődés esetén).

Az algoritmus SQL kódja:

```
DECLARE @NextDate DATETIME = @DueDate;
DECLARE @EndDate DATETIME = CAST(@RecurrenceEndDate AS DATETIME);
DECLARE @MaxIterations INT = 0;

WHILE @MaxIterations < 52
BEGIN
   SET @NextDate = CASE @RecurrenceType
      WHEN 'Weekly'   THEN DATEADD(WEEK, 1, @NextDate)
      WHEN 'Biweekly' THEN DATEADD(WEEK, 2, @NextDate)
      WHEN 'Monthly'  THEN DATEADD(MONTH, 1, @NextDate)
      ELSE @EndDate
   END;

   IF @NextDate > @EndDate BREAK;

   INSERT INTO Tasks (CreatedBy, SubjectID, Title, Description,
      DueDate, TaskType, IsCompleted, RecurrenceType,
      RecurrenceEndDate, ParentTaskID, CreatedAt)
   VALUES (@StudentID, @SubjectID, @Title, @Description,
      @NextDate, @TaskType, 0, @RecurrenceType,
      @RecurrenceEndDate, @ParentID, GETDATE());

   SET @MaxIterations = @MaxIterations + 1;
END
```

Példa: Ha a felhasználó „Matematika házi” feladatot hoz létre 2026.03.21-i határidővel, heti ismétlődéssel, 2026.04.18-i végdátummal, akkor az tárolt eljárást hoz létre:

1 szülő feladatot (03.21) + 4 gyermek feladatot (03.28, 04.04, 04.11, 04.18) = összesen 5 sor a Tasks táblában.

2.8.2 Határidő-közelség kiszámítása

A határidőhöz kapcsolódó segédmezők nem az sp_GetMyTasks tárolt eljárásban kerülnek kiszámításra, hanem a frontend TaskModels.cs fájlban, C# computed property-ként:

```
public bool IsOverdue => !IsCompleted && DueDate < DateTime.Now;

public bool IsDueSoon => !IsCompleted && !IsOverdue && DueDate <= DateTime.Now.AddDays(3);

public string DaysLeft { get { ... } }  // (DueDate - DateTime.Now).TotalDays alapján
```

Határidő állapotának meghatározása

A feladatok határidejéhez kapcsolódó állapotokat a TaskModels.cs fájlban, számított property-k segítségével állapítjuk meg:

- **Késésben lévő feladat (IsOverdue):** Az a feladat számít késésben lévőnek, amely még nincs befejezve (*IsCompleted* hamis), és a határideje (*DueDate*) már korábban volt, mint a jelenlegi dátum (*DateTime.Now*).
- **Közelgő határidejű feladat (IsDueSoon):** Egy feladat akkor tekinthető hamarosan esedékesnek, ha még nincs befejezve, nem számít késésben lévőnek, és a határideje három napon belül van (azaz *DueDate* ≤ *DateTime.Now.AddDays(3)*).
- **Hátralévő napok (DaysLeft):** A hátralévő idő a határidőig úgy számítható ki, hogy a határidőből (*DueDate*) kivonjuk a jelenlegi dátumot (*DateTime.Now*), majd az eredményt napokban (*TotalDays*) fejezzük ki.

Ezeket a tulajdonságokat a Feladatok.razor közvetlenül felhasználja a vizuális kiemeléshez: az IsOverdue = true esetén piros (fr-tr-danger, fr-due-danger), az IsDueSoon = true esetén narancssárga (fr-due-warn, fr-badge-warn), a befejezett feladatoknál zöld (fr-tr-done) stílust alkalmaz.

2.8.3 E-mail alapú meghívás csoportba

A csoport-meghívás folyamata az e-mail cím alapján azonosítja a meghívandó felhasználót, így a meghívónak nem kell tudnia a pontos felhasználónevet. Az sp_InviteToGroup tárolt eljárás a következő lépéseket hajtja végre:

1. Megkeresi a megadott e-mail alapján az InvitedUserID-t a Users táblában.
2. Ellenőrzi, hogy a meghívott már tagja-e a csoportnak (GroupMembers tábla).
3. Ellenőrzi, hogy van-e már függő (Pending) meghívás.
4. Ha mindkét ellenőrzés átment, INSERT-tel létrehozza a meghívást a GroupInvites táblában.
5. A frontend a GET /api/group/my-invites végponton kérdezi le a beérkező meghívásokat, amelyek elfogadhatók vagy visszautasíthatók.

2.8.4 FocusTimerService

A fókusz időzítő (Pomodoro) egy singleton Blazor service-ként van megvalósítva (FocusTimerService.cs), ami azt jelenti, hogy az állapota megmarad navigáció között, a felhasználó lapot válthat, majd visszatérhet, és a visszaszámlálás pontosan ott folytatódik, ahol abbahagyta.

Az időzítő három állapotot ismer:

Állapot (Mode)	Alapértelmezett időtartam	Leírás
focus	25 perc	Koncentrált tanulási periódus
short	5 perc	Rövid pihenő automatikusan fókusz után, ha AutoSwitch be van kapcsolva
long	15 perc	Hosszú pihenő kézzel választható a mód gombok segítségével

A System.Threading.Timer 1 másodpercenként csökkenti a SecondsLeft értéket, majd az OnTick event meghívásával értesíti a UI-t a frissítésről. Ha SecondsLeft eléri a nullát, az AutoSwitch beállítástól függően automatikusan a következő módra vált, és CompletedSessions értéke nő. A SessionLog lista az elvégzett munkaszakaszokat tárolja, amelyek a Fokusz oldalon listában jelennek meg.

2.8.5 E-mail uniqueness ellenőrzése

Az sp_RegisterUser tárolt eljárás a tényleges INSERT előtt ellenőrzi, hogy a megadott felhasználónév és e-mail cím szabad-e:

```
IF EXISTS (SELECT 1 FROM Users WHERE Username = @Username)
BEGIN
   SELECT 'ERROR' AS Status, 'A felhasználónév már foglalt!' AS Message;
   RETURN;
END

-- Ellenőrzés: létezik-e már az email
IF EXISTS (SELECT 1 FROM Users WHERE Email = @Email)
BEGIN
   SELECT 'ERROR' AS Status, 'Az email cím már használatban van!' AS Message;
   RETURN;
END
```

A backend (AuthService.cs) a tárolt eljárás által visszaadott Status/Message objektumot dolgozza fel, és ha az ERROR, akkor HTTP 400-as választ küld a frontendnek a hibaüzenettel együtt. A frontend Register.razor komponens ezt a hibát közvetlenül megjeleníti a felhasználónak a megfelelő mező alatt.

2.8.6 Szavazási algoritmus (Upsert)

Az sp_VotePoll tárolt eljárás upsert (INSERT or UPDATE) logikát alkalmaz a szavazatkezeléshez:

```
IF EXISTS (SELECT 1 FROM PollVotes
        WHERE PollID = @PollID AND UserID = @UserID)
BEGIN
   -- Szavazat módosítása
   UPDATE PollVotes
   SET OptionID = @OptionID, VotedAt = GETDATE()
   WHERE PollID = @PollID AND UserID = @UserID;
END
ELSE
BEGIN
   INSERT INTO PollVotes (PollID, OptionID, UserID)
   VALUES (@PollID, @OptionID, @UserID);
END
```

Az UQ_PollVote UNIQUE constraint (PollID, UserID) az adatbázis szintjén is garantálja, hogy egy felhasználó csak egyszer szavazhasson per szavazás. Az upsert minta lehetővé teszi, hogy a felhasználó módosítsa a szavazatát (másik opcióra váltson) anélkül, hogy először törölnie kellene a korábbit.

2.8.7 Órarend ütközés-felismerés

Az sp_AddScheduleEntry tárolt eljárás az órarend-ütközéseket az alábbi logika szerint észleli:

```
IF EXISTS (
    SELECT 1 FROM ScheduleEntries
    WHERE StudentID = @StudentID
      AND DayOfWeek = @DayOfWeek
      AND StartTime < @EndTime
      AND EndTime > @StartTime
)
```

Az ütközés-ellenőrzés csak Weekly típusú bejegyzések között fut le. A Biweekly (kéthetente) és None (egyszeri) típusú bejegyzések esetén az ellenőrzés kihagyásra kerül, mivel ezek felváltva vagy csak egyszer jelennek meg — valódi időpontütközés köztük nem állhat fenn. A helyes megjelenítést a frontend IsEntryVisibleOnDate metódusa kezeli.

Az időintervallum-átfedés feltétele Weekly vs Weekly esetén: StartTime < @EndTime AND EndTime > @StartTime. Ez a klasszikus [A,B) és [C,D) intervallum-átfedés képlete.

Példa: Ha már létezik heti bejegyzés hétfőn 08:00–09:30, és a felhasználó 09:00–10:00-t szeretne felvenni szintén heti ismétlődéssel, az ellenőrzés ütközést jelez. Ha viszont kéthetente típust választ, a bejegyzés felvételre kerül, mert felváltva jelenik meg.

Fontos kivétel: tanári felhasználók esetén a beiratkozás-ellenőrzés kimarad, de az ütközés-ellenőrzés rájuk is vonatkozik.

2.9 Backend felépítése

A közös JWT segédmetódusokat (GetCurrentUserId, GetCurrentUserEmail, GetCurrentUserRole) egy BaseController ősosztályba szerveztük ki, amelyből az összes controller örököl. A GetCurrentUserId metódus a JWT tokenből olvassa ki a bejelentkezett felhasználó azonosítóját, és érvénytelen token esetén UnauthorizedAccessException-t dob. Ez elkerüli a kód duplikációt és a clean code elveit követi.

2.9.1 Service osztályok bemutatása

A backend üzleti logikája a Service osztályokban valósul meg. Minden Service osztály implementál egy interfészt (pl. IAuthService, ITaskService), a Dependency Injection (DI) regisztráció a Program.cs-ben AddScoped<> hívással történik. Az adatbázis-elérés minden esetben Dapper micro-ORM-en és tárolt eljárásokon keresztül valósul meg.

2.9.1.1 AuthService (Backend)

Az AuthService felelős a regisztrációért, a bejelentkezésért és a JWT token generálásáért. A főbb műveletek:

Register metódus

1. A felhasználó jelszavát BCrypt.Net.BCrypt.HashPassword() segítségével hash-eli. A BCrypt automatikusan generál salt-ot, és a cost factor alapértelmezett értéke 11. A metódus első lépésként ellenőrzi, hogy a kért szerepkör nem Admin-e, ha igen, azonnal hibát ad vissza, így szerver oldalon sem lehet Admin fiókot regisztrációval létrehozni.

2. Az sp_RegisterUser tárolt eljárást hívja Dapper QueryAsync<dynamic> metódussal. A dynamic típus használata azért szükséges, mert a tárolt eljárás kétféle eredményt adhat vissza: hiba esetén Status + Message mezőket, siker esetén az új felhasználó teljes adatait.

3. A frontend visszairányít a bejelentkezési oldalra, a forceLoad: true frissíti az oldalt. A felhasználónak külön be kell jelentkeznie.

Login metódus

1. Az sp_LoginUser tárolt eljárás a felhasználónevet keresi. Ha talál egyezést, visszaadja a PasswordHash-t és az összes felhasználói adatot.

2. A C# oldalon a BCrypt.Net.BCrypt.Verify(request.Password, storedPasswordHash) metódus hasonlítja össze a megadott jelszót a hash-elt változattal. Ez az összehasonlítás időzítés-biztonságos (timing-safe).

3. Ha a jelszó helyes, JWT token generálódik és LoginResponse-ban visszaküldésre kerül.

GenerateJwtToken metódus

A JWT token az alábbi claim-eket tartalmazza:

```
new Claim(JwtRegisteredClaimNames.Sub, user.UserID.ToString()),
new Claim(JwtRegisteredClaimNames.UniqueName, user.Username),
new Claim(JwtRegisteredClaimNames.Email, user.Email ?? ""),
new Claim(ClaimTypes.Role, user.UserRole),
new Claim("FirstName", user.FirstName ?? ""),
new Claim("LastName", user.LastName ?? ""),
new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
```

A token aláírása HMAC-SHA256 algoritmussal történik. A lejárati időt, az issuer-t és az audience-t az appsettings.json JwtSettings szekciója tartalmazza. A Jti (JWT ID) claim minden tokenhez egyedi GUID-ot rendel, ami a token-visszavonás (token revocation) jövőbeli implementálásához nyújt alapot.

UpdateUsername metódus

Az UpdateUsername metódus a bejelentkezett felhasználó felhasználónevét módosítja. Az sp_UpdateUsername tárolt eljárás ellenőrzi, hogy az új felhasználónév nem foglalt-e más felhasználó által. Sikeres módosítás után a metódus új JWT tokent generál (GenerateJwtToken), amelyet az authorizációs fejlécbe visszacsatol. Így a frontend azonnal frissíteni tudja a localStorage-ban tárolt tokent és a CurrentUser objektumot, kiléptetés nélkül.

ChangePassword metódus

A ChangePassword metódus két lépésben működik: először a BCrypt.Net.BCrypt.Verify() metódussal ellenőrzi a jelenlegi jelszó helyességét az adatbázisban tárolt hash alapján. Csak sikeres ellenőrzés után hívja meg az sp_ChangePassword tárolt eljárást az új BCrypt hash-sel. A jelszó módosítása nem generál új JWT tokent, a meglévő token érvényes marad.

2.9.1.2 TaskService (Backend)

A TaskService az egyéni feladatok Létrehozás (Create) műveleteit valósítja meg, valamint lehetőséget biztosít az adatok Lekérdezésére (Read), Teljesítésére/Visszavonására (Complete/Uncomplete) és Törlésére (Delete). Minden metódushoz szükséges a studentId paraméter, amely a JWT tokenből származik (ClaimTypes.Sub). A főbb metódusok:

```
Task<IEnumerable<TaskItem>> GetMyTasksAsync(int studentId);
Task<SubjectResponse> AddTaskAsync(int studentId, AddTaskRequest request);
Task<SubjectResponse> CompleteTaskAsync(int taskId, int studentId);
Task<SubjectResponse> UncompleteTaskAsync(int taskId, int studentId);
Task<SubjectResponse> DeleteTaskAsync(int taskId, int studentId);
```

A GetMyTasks metódus az sp_GetMyTasks tárolt eljárást hívja, amely LEFT JOIN-nal csatolja a Subjects táblát a tantárgynév megjelenítéséhez. Az AddTask hívás az sp_AddTask-ot használja, amely az ismétlődő feladatokat (RecurrenceType != 'None') automatikusan létrehozza a megadott időszak végéig, maximum 52 iterációval.

2.9.1.3 ScheduleService (Backend)

A ScheduleService az egyéni órarend Létrehozás (Create), Lekérdezés (Read) és Törlés (Delete) műveleteit valósítja meg. A létrehozásnál az sp_AddScheduleEntry két fontos validációt végez:

1. **Beiratkozás-ellenőrzés**: Diák esetén ellenőrzi, hogy a felhasználó beiratkozott-e a tantárgyra (StudentSubjects tábla). Tanár esetén (UserRole = 'Teacher') ezt az ellenőrzést kihagyja.

2. Ütközés-ellenőrzés: Ha az adott napon (DayOfWeek) a megadott idősáv átfed egy már létező Weekly bejegyzéssel, hibaüzenetet ad vissza. Biweekly és None típusnál az ütközés-ellenőrzés nem fut le, mivel ezek felváltva/egyszeri jelleggel jelennek meg.

3. StartDate kezelés: Az sp_AddScheduleEntry fogadja a @StartDate paramétert, amelyet a frontend a kattintott naptári napból tölt ki. Ez az érték határozza meg az ismétlődés kezdőpontját: Havonta típusnál minden hónapban ugyanezen a napon jelenik meg a bejegyzés, Kéthetente típusnál ettől a dátumtól számítódnak a kéthetes ciklusok.

2.9.1.4 GroupService (Backend)

A GroupService a csoportkezelés teljes üzleti logikáját valósítja meg. A GetMyGroupsAsync metódus egyetlen tárolt eljárás-hívással kéri le a csoportokat (beleértve az OwnerRole mezőt is), és hiba esetén InvalidOperationException-t dob ahelyett, hogy csendben üres listát adna vissza. A CanManageGroupAsync dedikált inline query-vel ellenőrzi egyetlen csoport szerkesztési jogát, korábban ez az összes csoport lekérésével járt (N+1 jellegű hívás). Az AddGroupTaskAsync az új sp_AddGroupTask tárolt eljárást hívja. A főbb metódusok:

```
Task<IEnumerable<Group>> GetMyGroups(int userId);
Task<string> CreateGroup(int userId, CreateGroupRequest req);
Task<string> InviteToGroup(int groupId, int invitedBy, InviteToGroupRequest req);
Task<string> AcceptInvite(int inviteId, int studentId);
Task<string> DeclineInvite(int inviteId, int studentId);
Task<IEnumerable<GroupMember>> GetGroupMembers(int groupId, int userId);
Task<IEnumerable<GroupScheduleEntry>> GetGroupSchedule(int groupId, int userId);
Task<IEnumerable<GroupSubject>> GetGroupSubjects(int groupId, int userId);
Task<SubjectResponse> RenameGroupAsync(int groupId, int userId, string newName);
Task<SubjectResponse> DeleteGroupAsync(int groupId, int userId);
Task<SubjectResponse> RemoveGroupMemberAsync(int groupId, int memberuserId, int
requestingUserId);
```

Az sp_CreateGroup tárolt eljárás két műveletet végez egy tranzakcióban: beszúrja a Groups sort, majd a létrehozót automatikusan taggá teszi a GroupMembers táblában. Ez azt biztosítja, hogy egy csoport soha nem üres, mindig legalább a tulajdonos benne van.

Az sp_InviteToGroup három szintű validációt tartalmaz: (a) létezik-e a meghívott e-mail címmel felhasználó, (b) nem tagja-e már a csoportnak, (c) nincs-e már függő meghívása.

2.9.1.5 TeacherService (Backend)

A TeacherService a tanári funkciókat (szavazáskezelés, dolgozat-hozzáadás) valósítja meg. Dolgozatot a csoportban bármely Teacher szerepkörű tag hozhat létre, a TeacherController az IsGroupMemberAsync + szerepkör-ellenőrzést alkalmazza, így nem csak a csoport tulajdonosa meghirdethet dolgozatot. A szavazás (poll) létrehozása két lépéses folyamat:

1. Lépés: sp_CreatePoll: Létrehozza a Polls sort, visszaadja a PollID-t.

2. Lépés: Minden opcióra sp_AddPollOption hívás: beszúrja a PollOptions sorokat.

A C# kód ezeket szekvenciálisan hajtja végre egyetlen SqlConnection-ön belül, így ha az adatbázis-kapcsolat megszakad, nem maradnak árva Polls sorok opciók nélkül.

A dolgozat-létrehozás (sp_CreateTeacherExam) a csoporttagság-ellenőrzést tartalmaz, csak a csoport tagja hozhat létre dolgozatot. A szerepkör-ellenőrzés (Teacher) a controller szintjén történik az [Authorize(Roles = "Teacher")] attribútummal.

2.9.2 API végpontok dokumentációja

2.9.2.1 Autentikáció: /api/auth

Metódus	Végpont	Jogosultság	Leírás
POST	/api/auth/register	Nyilvános	Regisztráció: Username, Email, jelszó, szerep
POST	/api/auth/login	Nyilvános	Bejelentkezés: JWT token visszaadása
GET	/api/auth/me	[Authorize]	Bejelentkezett felhasználó adatai (UserDto)
PUT	/api/auth/update-username	[Authorize]	Felhasználónév módosítása: új JWT token visszaadása
PUT	/api/auth/change-password	[Authorize]	Jelszó módosítása: jelenlegi jelszó ellenőrzéssel

2.9.2.2 Feladatok: /api/task

Metódus	Végpont	Leírás
GET	/api/task/my-tasks	Saját feladatok listája (Student)
POST	/api/task/add	Új feladat hozzáadása (AddTaskRequest)
PUT	/api/task/complete/{taskId}	Feladat teljesítettnek jelölése
PUT	/api/task/uncomplete/{taskId}	Feladat visszaállítása elvégzendőre
DELETE	/api/task/delete/{taskId}	Feladat törlése (csak saját)

2.9.2.3 Órarend: /api/schedule

Metódus	Végpont	Leírás
GET	/api/schedule/my-schedule	Saját órarend bejegyzések (Student, Teacher)
POST	/api/schedule/add	Új bejegyzés (AddScheduleRequest)
DELETE	/api/schedule/delete/{entryId}	Bejegyzés törlése

2.9.2.4 Csoportok: /api/group

Metódus	Végpont	Leírás
GET	/api/group/my-groups	Csoportjaim listája
POST	/api/group/create	Új csoport létrehozása
POST	/api/group/{groupId}/invite	Tag meghívása e-mail alapján
POST	/api/group/invite/{inviteId}/accept	Meghívás elfogadása
POST	/api/group/invite/{inviteId}/decline	Meghívás elutasítása
POST	/api/group/{groupId}/leave	Csoportból kilépés
GET	/api/group/my-invites	Beérkező meghívások
GET	/api/group/{groupId}/members	Csoport tagjai
GET	/api/group/{groupId}/subjects	Csoport tagjainak tantárgyai
GET	/api/group/{groupId}/schedule	Csoport közös órarendje
GET	/api/group/{groupId}/tasks	Csoport tagjainak feladatai
POST	/api/group/{groupId}/schedule/add	Órarend bejegyzés hozzáadása csoporthoz
POST	/api/group/{groupId}/tasks/add	Csoportos feladat hozzáadása
DELETE	/api/group/{groupId}/tasks/{taskId}	Csoportos feladat törlése
DELETE	/api/group/{groupId}/schedule/{entryId}	Csoport órarend-bejegyzés törlése
PUT	/api/group/{groupId}/rename	Csoport átnevezése (csak tulajdonos)
DELETE	/api/group/{groupId}	Csoport törlése (csak tulajdonos)
DELETE	/api/group/{groupId}/members/{memberId}	Tag eltávolítása (csak tulajdonos)

2.9.2.5 Tanári funkciók: /api/teacher

Metódus	Végpont	Leírás
POST	/api/teacher/poll/create	Szavazás létrehozása csoportban
GET	/api/teacher/{groupId}/polls	Csoport szavazásainak listája
POST	/api/teacher/poll/{pollId}/vote	Szavazat leadása
DELETE	/api/teacher/poll/{pollId}	Szavazás törlése (létrehozó)
POST	/api/teacher/exam/create	Dolgozat meghirdetése csoportnak (bármely tanár tag, nem csak a tulajdonos)
GET	/api/teacher/{groupId}/exams	Csoport dolgozatainak listája
DELETE	/api/teacher/exam/{examId}	Dolgozat törlése (létrehozó)

2.9.2.6 Tantárgyak: /api/subject

Metódus	Végpont	Leírás
GET	/api/subject/available	Felvehető tantárgyak (listából választható)
GET	/api/subject/all	Összes tantárgy listázása (admin/rendszerszintű)
GET	/api/subject/my-enrollments	Saját felvett tantárgyak
POST	/api/subject/enroll	Tantárgy felvétele listából
POST	/api/subject/enroll-custom	Egyedi tantárgy felvétele
DELETE	/api/subject/drop/{subjectId}	Tantárgy leadása

2.9.3 DTO-k és adatátviteli modellek

A frontend és backend közötti kommunikáció DTO (Data Transfer Object) osztályokon keresztül történik. A DTO-k elkülönítik az adatbázis entitásokat az API válaszoktól. Az alábbiakban a legfontosabb DTO-kat ismertetjük a C# modellek alapján.

2.9.3.1 Autentikációs DTO-k

RegisterRequest: A regisztrációhoz szükséges adatokat tartalmazza. Validációs attribútumok: [Required], [MinLength(3), MaxLength(50)] a Username-re, [MinLength(8)] és [RegularExpression] a Password-re (legalább 1 nagybetű és 1 szám kötelező), [EmailAddress] az Email-re, és [Required] a UserRole-ra. A ConfirmPassword mező kizárólag a frontend validációhoz létezik, [Compare("Password")] attribútummal biztosítja, hogy a két jelszómező megegyezzen. A backend API-ra a ConfirmPassword nem kerül elküldésre, azt az AuthModel.cs IValidatableObject implementációja helyi szinten ellenőrzi.

LoginRequest: Egyszerű DTO, két [Required] mezővel: Username és Password.

LoginResponse: Az API válasza: Status (SUCCESS/ERROR), Message, Token (JWT string), User (UserDto).

UserDto: A felhasználói adatok biztonságos része: UserID, Username, Email, FirstName, LastName, UserRole. Nem tartalmazza a PasswordHash-t.

UpdateUsernameRequest: Egyetlen mező: NewUsername (string). A backend AuthController a bejelentkezett felhasználó UserID-ját a JWT tokenből olvassa ki, ezért a kérésben csak az új felhasználónév szükséges.

ChangePasswordRequest: Három mező: CurrentPassword, NewPassword és ConfirmNewPassword (string). A ConfirmNewPassword mező kizárólag a frontend validációhoz létezik, a backend API-ra csak CurrentPassword és a NewPassword kerül elküldésre. A frontend Fiok.razor komponense helyi ellenőrzést végez, hogy az új jelszó megfelel-e az erősségi követelményeknek (min. 8 karakter, nagybetű, szám, speciális karakter) és egyezik-e a megerősítő mezővel.

2.9.3.2 Feladat DTO-k

TaskItem: A feladatok teljes modellje, tárolja a SubjectName és SubjectCode mezőket is (JOIN eredménye). A RecurrenceType és ParentTaskID mezők az ismétlődési logikához szükségesek.

AddTaskRequest: A feladat létrehozásához szükséges DTO. A SubjectID nullable (int?), mert személyes feladathoz nem kell tantárgy. A TaskType alapértelmezett értéke "Assignment".

2.9.3.3 Csoport DTO-k

Group: A csoport fejadata, kiegészítve a MemberCount (számított), IsOwner (számított) és OwnerRole mezőkkel. Ezeket az sp_GetMyGroups tárolt eljárás számítja az SQL lekérdezésben.

GroupMember: A csoporttag adatait tartalmazza: UserID, FirstName, LastName, Email, JoinedAt, IsOwner. Az IsOwner mező is számított: Groups.CreatedBy == GroupMembers.StudentID.

GroupInvite: A meghívás adatait tartalmazza a frontend számára: InviteID, GroupID, GroupName, InvitedByName, CreatedAt. A GroupName és InvitedByName JOIN eredmények.

CreateGroupRequest: Egyetlen mező: GroupName (string).

InviteToGroupRequest: Egyetlen mező: InvitedEmail (string).

RenameGroupRequest: Egyetlen mező: NewName (string). A backend az [Authorize] attribútumon keresztül a JWT tokenből azonosítja a kérelmezőt, és csak a csoport tulajdonosa hajthatja végre a műveletet.

2.9.3.4 Szavazási DTO-k

Poll: A szavazás fejadata. Tartalmazza a CreatedByName (JOIN), Options (List<PollOption>) és MyVoteOptionID (nullable int) mezőket. A MyVoteOptionID null, ha a jelenlegi felhasználó még nem szavazott.

PollOption: Opció adatai: OptionID, PollID, OptionText, VoteCount.

CreatePollRequest: GroupID, Question (string), Options (List<string>), ExpiresAt (nullable DateTime).

VoteRequest: Egyetlen mező: OptionID (int).

2.9.3.5 Tanári dolgozat DTO-k

TeacherExam: A dolgozat teljes modellje, tartalmazza a GroupName és CreatedByName JOIN mezőket is.

CreateTeacherExamRequest: GroupID, Title, Description (nullable), ExamDate, SubjectName (nullable, szabad szöveg).

2.9.3.6 Ismétlődési típusok: RecurrenceKind static osztály

A ScheduleEntry.cs fájlban definiált RecurrenceKind static osztály konstansokat tartalmaz az ismétlődési típusokhoz:

```
public static class RecurrenceKind
{
   public const string None = "None";        // Egyszeri
   public const string Weekly = "Weekly";      // Heti
   public const string Biweekly = "Biweekly";   // Kéthetente
   public const string Monthly = "Monthly";     // Havonta
}
```

A konstansok használata („magic string”-ek helyett) csökkenti az elgépelés kockázatát és intellisense támogatást nyújt.

2.9.4 Dapper használati minták

A FeladatRadar backend kizárólag a Dapper micro-ORM-et használja az adatbázis-eléréshez. A Dapper könnyen használható IDbConnection kiterjesztéseket nyújt (QueryAsync, ExecuteAsync stb.), amelyek közvetlenül SQL parancsokat vagy tárolt eljárásokat hívnak.

2.9.4.1 Típusos lekérdezés (QueryAsync<T>)

Amikor a tárolt eljárás kimenetének mezői egyértelműen megegyeznek egy C# osztály property-jeivel, típusos mappingot használunk:

```
var tasks = await connection.QueryAsync<TaskItem>(
   "sp_GetMyTasks",
   new { StudentID = studentId },
   commandType: CommandType.StoredProcedure
);
}
```

A Dapper automatikusan illeszti az SQL oszlopneveket a C# property-nevekhez (case-insensitive). A SubjectName és SubjectCode mezők a LEFT JOIN eredményeként automatikusan kitöltődnek.

2.9.4.2 Dinamikus lekérdezés (QueryAsync<dynamic>)

Az autentikációs tárolt eljárások (sp_LoginUser, sp_RegisterUser) kétféle struktúrát adhatnak vissza (sikeres / hibás), ezért dynamic típust használunk és IDictionary<string, object> cast-ot alkalmazunk:

```
var result = results.FirstOrDefault();
var resultDict = (IDictionary<string, object>)result;

if (resultDict.ContainsKey("Status")
   && resultDict["Status"]?.ToString() == "ERROR")
{
   return new LoginResponse { Status = "ERROR",
      Message = resultDict["Message"]?.ToString() ?? "Hiba" };
}
```

Ez a minta rugalmas, de típusbiztonság nélküli: a propertyneveket stringként kell megadni. A projekt továbbfejlesztésekor érdemes lenne minden tárolt eljárást úgy módosítani, hogy mindig azonos struktúrát adjon vissza.

2.9.4.3 DynamicParameters használata

A Dapper DynamicParameters osztálya lehetővé teszi a paraméterek név-érték párokként való megadását. Az összes backend Service osztály ezt a mintát követi:

```
var parameters = new DynamicParameters();
parameters.Add("@GroupID", groupId);
parameters.Add("@AddedBy", userId);
parameters.Add("@SubjectID", request.SubjectID);
parameters.Add("@DayOfWeek", request.DayOfWeek);
parameters.Add("@StartTime", TimeSpan.Parse(request.StartTime));
parameters.Add("@EndTime", TimeSpan.Parse(request.EndTime));
parameters.Add("@Location", request.Location);
```

A Dapper automatikusan kezeli a NULL értékeket (C# null → SQL NULL), a TimeSpan → TIME konverziót, és a paraméterek típus-illesztését.

2.9.5 Admin funkciók

2.9.5.1 AdminController és AdminService

Az Admin funkciók a backend oldalon az AdminController (Controllers/AdminController.cs) és az AdminService (Services/AdminService.cs) párossal valósulnak meg. A teljes controller az [Authorize(Roles = "Admin")] attribútummal van védve, ami azt jelenti, hogy kizárólag Admin szerepkörű JWT-tokennel érhető el.

Az AdminController a következő végpontokat biztosítja:

- GET /api/Admin/stats: rendszerszintű statisztikák lekérése (sp_Admin_GetSystemStats tárolt eljárás).
- GET /api/Admin/users: összes felhasználó listázása (sp_Admin_GetAllUsers).
- POST /api/Admin/users/change-role: felhasználó szerepkörének módosítása (sp_Admin_ChangeUserRole). A tárolt eljárás megakadályozza az önmódosítást.
- POST /api/Admin/users/toggle-active: felhasználó letiltása vagy aktiválása (sp_Admin_ToggleUserActive). Az IsActive oszlop a Users táblában tárolja az állapotot.
- DELETE /api/Admin/users/{id}: felhasználó törlése (sp_Admin_DeleteUser). Ha FK constraint miatt nem törölhető, automatikusan soft-delete (IsActive=0) történik.
- POST /api/Admin/users/rename: felhasználó nevének (FirstName, LastName) módosítása (sp_Admin_RenameUser). A művelet naplózásra kerül.
- GET /api/Admin/groups: összes csoport listázása tulajdonos adataival (sp_Admin_GetAllGroups). GET /api/Admin/groups/{groupId}/exams: egy csoport összes dolgozatának lekérése admin számára (sp_Admin_GetGroupExams).
- DELETE /api/Admin/groups/{id}: csoport törlése kaszkád módon: tagok, meghívók, szavazások, dolgozatok (sp_Admin_DeleteGroup).
- DELETE /api/Admin/polls/{id}, /exams/{id}, /tasks/{id}: moderációs törlés (sp_Admin_DeletePoll, sp_Admin_DeleteExam, sp_Admin_DeleteTask).
- GET /api/Admin/audit-log: audit napló lekérése (sp_Admin_GetAuditLog, alapértelmezetten az utolsó 100 bejegyzés).

Minden írási művelet automatikusan naplózásra kerül az AuditLog táblába az sp_Admin_WriteAuditLog tárolt eljáráson keresztül. A naplóbejegyzés tartalmazza a végrehajtó admin UserID-ját, nevét, a művelet típusát és az ember által olvasható részleteket (pl. „Felhasználó (ID:5) szerepköre módosítva → Teacher", „Csoport törölve (ID:12)"). A Details mező így közvetlenül értelmezhető az Admin felület Napló fülén, nem szükséges ID-k alapján visszakeresni az érintett rekordokat.

Az AdminService a Dapper micro-ORM-et használja, konzisztensen a projekt többi Service osztályával. A DI regisztráció a Program.cs-ben a builder.Services.AddScoped<IAdminService, AdminService>() sorral történik. A frontend oldalon a FeladatRadar.frontend/Service/AdminService.cs HttpClient-en keresztül hívja az API végpontokat, és a FeladatRadar.frontend /Models/AdminModels.cs tartalmazza a szükséges DTO osztályokat (AdminUserDto, SystemStatsDto, AdminGroupDto, AuditLogEntry). A csoport dolgozatainak listázásához a TeacherExam modell kerül újrafelhasználásra (TeacherModels.cs).

2.9.5.2 AdminService (Frontend)

Az AdminService a rendszergazdai felületet szolgálja ki. Négy funkcionális területre oszlik:

Statisztikák:
GetSystemStatsAsync(): GET api/Admin/stats: rendszerszintű statisztikák (felhasználók, csoportok, feladatok száma).

Felhasználó-kezelés:
GetAllUsersAsync(): GET api/Admin/users. ChangeUserRoleAsync(int targetUserId, string newRole): POST api/Admin/users/change-role. ToggleUserActiveAsync(int targetUserId, bool isActive): POST api/Admin/users/toggle-active. DeleteUserAsync(int userId): DELETE api/Admin/users/{userId}.

Csoport- és moderációs kezelés:
GetAllGroupsAsync(): GET api/Admin/groups. DeleteGroupAsync(int groupId): DELETE api/Admin/groups/{groupId}: kaszkád törlés. GetGroupExamsAsync(int groupId): GET api/Admin/groups/{groupId}/exams: egy adott csoport összes dolgozatának listázása. DeletePollAsync(int pollId): DELETE api/Admin/polls/{pollId}. DeleteExamAsync(int examId): DELETE api/Admin/exams/{examId}. DeleteTaskAsync(int taskId): DELETE api/Admin/tasks/{taskId}. RenameUserAsync(int targetUserId, string firstName, string lastName): POST api/Admin/users/rename: felhasználó nevének módosítása.

Audit napló:
GetAuditLogAsync(int limit = 100): GET api/Admin/audit-log?limit={limit}: az utolsó 100 (vagy megadott számú) admin művelet lekérése.

2.9.5.3 Kaszkád törlés algoritmusa

Az sp_Admin_DeleteGroup tárolt eljárás kaszkád törlést végez, amely az összes kapcsolódó adatot eltávolítja. A törlési sorrend kötött az idegen kulcs függőségek miatt:

1. PollVotes: A szavazatokat töröljük először (hivatkoznak PollOptions-ra és Polls-ra).
2. PollOptions: A szavazási opciók (hivatkoznak Polls-ra).
3. Polls: A szavazások maguk.
4. GroupTasks: A csoportfeladatok.
5. GroupScheduleEntries: A csoport órarend-bejegyzései.
6. GroupInvites: A függőben lévő meghívások.
7. GroupMembers: A csoporttagságok.
8. Groups: Végül maga a csoport.

Az egész művelet BEGIN TRY / BEGIN CATCH blokkban fut, így ha bármelyik DELETE utasítás hibára fut, a tárolt eljárás hibaüzenetet ad vissza. Megjegyezzük, hogy bár a GroupTasks, GroupScheduleEntries, Polls és TeacherExams táblákra ON DELETE CASCADE van beállítva a GroupID-ra, a manuális törlés azért szükséges, mert a PollVotes tábla nem közvetlenül hivatkozik a Groups-ra (hanem a Polls-on és PollOptions-ön keresztül), így a CASCADE nem terjed ki rá.

2.9.5.4 Soft-delete mechanizmus

Az sp_Admin_DeleteUser tárolt eljárás megpróbálja fizikailag törölni a felhasználót (DELETE FROM Users). Ha ez FK constraint sértés miatt sikertelen (például a felhasználó csoporttulajdonos vagy feladatokat hozott létre), a CATCH blokkban automatikusan átvált soft-delete-re:

```
BEGIN TRY
   DELETE FROM [dbo].[Users] WHERE UserID = @TargetUserID;
   SELECT 'OK' AS Status, 'Felhasználó sikeresen törölve.' AS Message;
END TRY
BEGIN CATCH
   UPDATE [dbo].[Users] SET IsActive = 0 WHERE UserID = @TargetUserID;
   SELECT 'OK' AS Status,
     'Felhasználó letiltva (soft-delete, FK hivatkozás miatt).' AS Message;
END CATCH
```

Ez a megoldás biztosítja, hogy az admin mindig sikeres eredményt lát: vagy fizikai törlés, vagy letiltás történik. A letiltott felhasználó (IsActive = 0) adatai megmaradnak az integritás megőrzése érdekében, de a fiók nem használható.

2.9.6 Kódminőség és konvenciók

Névkonvenciók

A projekt az alábbi névkonvenciókat követi következetesen:

Elem	Konvenció	Példa
Osztályok	PascalCase	TaskService, AuthController
Interface-ek	IPascalCase előtaggal	ITaskService, IAuthService
Metódusok	PascalCase + Async utótag async esetén	GetMyTasksAsync(), Login()
Privát mezők (Blazor)	_camelCase	_tasks, _isLoading, _menuOpen
Paraméterek	@camelCase (tárolt eljárás)	@StudentID, @GroupID
Razor komponens változók	_camelCase	_hasCheckedAuth, _userRole
CSS osztályok	fr- előtag + kebab-case	fr-sidebar, fr-nav-item, fr-btn-primary

Hibakezelési stratégia

A Service osztályokban minden adatbázis-hívás try-catch blokkban van, és hiba esetén SubjectResponse { Status = "ERROR", Message = ex.Message } típusú objektumot ad vissza. Ez egységes hibaformátumot biztosít, amelyet a Controller réteg HTTP 400-as (regisztrációnál) vagy 401-es (bejelentkezésnél) válaszként továbbít. A Blazor frontend minden service-hívás után ellenőrzi a Status értékét, és hiba esetén egy piros figyelmeztető sávot (fr-alert-danger) jelenít meg a felhasználónak. A GetMyGroupsAsync metódus kivételt dob (InvalidOperationException) adatbázis-hiba esetén, ahelyett hogy csendben üres listát adna vissza, ez biztosítja, hogy az ilyen hibák ne maradjanak észrevétlenül. Az alkalmazás sehol nem enged ellenőrizetlen kivételt a felhasználóig eljutni, minden hiba le van kezelve és felhasználóbarát üzenetté van alakítva.

Dependency Injection

Az alkalmazás az ASP.NET Core beépített DI konténerét használja. A backend Program.cs-ben minden service regisztrálva van:

```
// Backend Program.cs: Service regisztrációk
builder.Services.AddScoped<ISubjectService, SubjectService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IScheduleService, ScheduleService>();
builder.Services.AddScoped<ITaskService, TaskService>();
builder.Services.AddScoped<ITeacherService, TeacherService>();
builder.Services.AddScoped<IAdminService, AdminService>();
builder.Services.AddScoped<IGroupService, GroupService>();

// Frontend Program.cs: Service regisztrációk
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<SubjectService>();
builder.Services.AddScoped<ScheduleService>();
builder.Services.AddScoped<GroupService>();
builder.Services.AddScoped<TaskService>();
builder.Services.AddScoped<TeacherService>();
builder.Services.AddScoped<AdminService>();
```

Fontos különbség: a FocusTimerService AddSingleton-ként van regisztrálva (nem AddScoped-ként), mert az időzítő állapotát meg kell tartani navigáció között. Blazor Server esetén minden felhasználói kapcsolat saját DI scope-ot kap, tehát a singleton minden kapcsolaton belül egy példány, ez pontosan a kívánt viselkedés.

2.9.7 Swagger API dokumentáció

A backend fejlesztés során az API végpontok tesztelésére és dokumentálásra Swashbuckle.AspNetCore könyvtárat használtunk, amely automatikusan generálja a Swagger UI-t. A Swagger a fejlesztői módban elérhető a https://localhost:44359/swagger URL-en.

A Swagger felületen a JWT autentikáció is be van konfigurálva: az "Authorize" gombra kattintva megadható a Bearer token, amellyel ezután az összes védett végpont is tesztelhető a böngészőből. Ez lehetővé tette, hogy az API tesztelése Postman helyett közvetlenül a Swagger UI-ból is elvégezhető legyen.

A Swagger konfiguráció a FeladatRadar.sln Program.cs-ben:

```
builder.Services.AddSwaggerGen(c =>
{
   c.AddSecurityDefinition("Bearer", new
Microsoft.OpenApi.Models.OpenApiSecurityScheme
   {
      Name = "Authorization",
      Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
      Scheme = "bearer",
      BearerFormat = "JWT",
      In = Microsoft.OpenApi.Models.ParameterLocation.Header,
      Description = "Csak a tokent írja be, a Bearer szó nélkül."
   });
```

2.10 Frontend felépítése

2.10.1 Oldal-architektúra és navigáció

A Blazor Server alkalmazásban minden oldal egy .razor fájl a FeladatRadar.frontend/Components/Pages/ mappában. Az App.razor és Routes.razor fájlok kezelik a routing-ot. A MainLayout.razor biztosítja az egységes oldal-elrendezést (sidebar + fő tartalom), amelyet az összes oldal örököl.

Minden oldal elején AuthService.IsAuthenticated ellenőrzéssel indul, ha a felhasználó nincs bejelentkezve, a Navigation.NavigateTo("/login") hívással a bejelentkező oldalra irányítja át. Ez a minta biztosítja, hogy a védett oldalak ne legyenek elérhetők autentikáció nélkül.

2.10.2 Komponensek bemutatása

2.10.2.1 Dashboard.razor

A főoldal összefoglaló nézetet nyújt a bejelentkezett felhasználónak. Öt statisztikai kártyán megjeleníti az aktív feladatok számát, a lejárt feladatok számát, a mai események számát, a felvett tantárgyak számát és a csoportok számát. A kártyák kattinthatók, mindegyik a vonatkozó oldalra navigál (Feladatok, Naptár, Tantárgyak, Csoportok). A kártyák alatt két panel jelenik meg: a „Közelgő feladatok" lista a még nem teljesített, határidő szerint rendezett feladatokat mutatja (szűrhető: ma, holnap, héten), valamint egy „Gyors navigáció" panel, amely az alkalmazás fő oldalaira vezető hivatkozásokat tartalmazza.

A Főoldal topbarjában a korábbi egyszerű e-mail megjelenítő chip (fr-user-chip) helyett a (Fiok.razor) komponens szerepel. Ez egy Blazor önálló komponens (Components/Pages/Fiok.razor), amely a főoldal @code blokkjától teljesen elkülönítve kezeli a profil-legördülő menü állapotát. A komponens a bejelentkezett felhasználó nevét, e-mail-jét és monogramját jeleníti meg egy kattintható gombban; lenyitva lehetővé teszi a felhasználónév és a jelszó módosítását, illetve a kijelentkezést.

A Fiok.razor komponens fő funkciói: felhasználói adatok megjelenítése (teljes név, e-mail, szerepkör badge); felhasználónév módosítása (min. 3 karakter, valós idejű visszajelzés); jelszóváltás valós idejű erősségmérővel (min. 8 karakter, nagybetű, szám, speciális karakter feltételek vizuális jelzőkkel) és szemecske gombokkal a jelszóbeviteli mezőkhöz; valamint kijelentkezés gomb. A komponens a szerepkörtől (data-role attribútum) függően eltérő színsémát alkalmaz: lila/kék (Student), sárga/amber (Teacher), piros (Admin).

2.10.2.2 Login.razor és Register.razor

A bejelentkező és regisztrációs oldalak az egyetlen két nyilvánosan elérhető oldal az alkalmazásban. A Login.razor felhasználónevet és jelszót kér, POST-kéréssel hívja az /api/auth/login végpontot, és siker esetén a tokent localStorage-ban tárolja, majd a /dashboard oldalra navigál. Hiba esetén piros hibaüzenet jelenik meg az űrlap felett.

A Register.razor részletesebb validációt hajt végre: ellenőrzi, hogy a jelszó és a jelszó-megerősítés egyezik-e, a felhasználónév legalább 3 karakteres-e, az e-mail cím formátuma érvényes-e, és hogy a felhasználó kiválasztott-e szerepkört. A validáció C# kódban történik a @code blokkban, nem HTML5 required attribútumokkal, így pontosabb hibaüzenetek adhatók.

2.10.2.3 Naptar.razor

A Naptár oldal a projekt legösszetettebb frontend komponense.Az alábbiakban részletesen bemutatjuk az architektúráját.

Állapotváltozók

A komponens @code blokkja számos privát állapotváltozót tartalmaz, amelyek a naptár különböző aspektusait kezelik. Az alábbi táblázat csoportosítva ismerteti őket:

Adatok

Változó neve	Típus	Alapérték	Leírás
_schedule	List<ScheduleEntryDto>	new()	Felhasználó saját órarend-bejegyzései
_myEnrollments	List<EnrollmentDto>	new()	Felhasználó felvett tantárgyai (új bejegyzéshez)
_tasks	List<TaskDto>	new()	Felhasználó saját feladatai
_myGroups	List<GroupDto>	new()	Felhasználó csoportjai (csoport módhoz)
_groupSchedule	List<GroupScheduleEntryDto>	new()	Kiválasztott csoport órarendje
_groupTasks	List<GroupTaskDto>	new()	Kiválasztott csoport feladatai

Autentikáció és UI állapot

Változó neve	Típus	Alapérték	Leírás
_hasCheckedAuth	bool	false	Jelzi, hogy az auth ellenőrzés lefutott-e
_isAuthenticated	bool	false	Be van-e jelentkezve a felhasználó
_userEmail	string	""	Bejelentkezett felhasználó e-mail címe
_userRole	string	""	Bejelentkezett felhasználó szerepköre
_menuOpen	bool	false	Mobilon nyitva van-e az oldalsáv
_theme	string	"light"	Aktív téma: "light" vagy "dark"
_message	string	""	Visszajelző üzenet szövege (siker/hiba)
_messageIsError	bool	false	Az üzenet hibaüzenet-e (piros megjelenítés)
_isSaving	bool	false	Mentés folyamatban van-e (gomb letiltáshoz)
_formError	string	""	Beviteli form validációs hibaüzenete

Nézet és navigáció

Változó neve	Típus	Alapérték	Leírás
_viewMode	string	"week"	Aktív nézet: "week", "day" vagy "month"
_weekOffset	int	0	Heti nézet eltolása (0=aktuális hét, -1=előző stb.)
_dayViewOffset	int	0	Napi nézet eltolása az aktuális naptól
_monthMonth	int	DateTime.Today.Month	Havi nézetben megjelenített hónap
_monthYear	int	DateTime.Today.Year	Havi nézetben megjelenített év
_groupMode	bool	false	Csoport mód aktív-e (csoport adatait mutatja)
_selectedGroupId	int	0	Csoport módban kiválasztott csoport azonosítója

Új bejegyzés panel (órarend)

Változó neve	Típus	Alapérték	Leírás
_addingToDay	int	0	Melyik napra nyílt meg a beviteli panel (1-7, 0=zárva)
_panelTab	string	"ora"	Aktív panel fül: "ora" (órarend) vagy "beadando"
_selSubjectId	int	0	Kiválasztott tantárgy azonosítója
_selStart	string	"08:00"	Kezdési időpont (formátum: HH:mm)
_selEnd	string	"09:30"	Befejezési időpont (formátum: HH:mm)
_selLocation	string	""	Helyszín/terem megnevezése
_selRecurrence	string	"Weekly"	Ismétlődés típusa: None/Weekly/Biweekly/Monthly
_selRecurrenceEnd	DateTime?	null	Ismétlődés végdátuma (nullable)
_selHasEndDate	bool	false	Meg van-e adva végdátum (csak feladat módban, órarend fülön eltávolítva)

Új feladat panel

Változó neve	Típus	Alapérték	Leírás
_taskSubjectId	int	0	Kiválasztott tantárgy azonosítója
_taskTitle	string	""	Feladat neve
_taskDescription	string	""	Feladat leírása (opcionális)
_taskDueDate	DateTime	Now.AddDays(7)	Határidő (alapértelmezett: +7 nap)
_taskType	string	"Assignment"	Feladat típusa (Assignment, Exam stb.)
_taskRecurrence	string	"None"	Ismétlődés típusa
_taskRecurrenceEnd	DateTime?	null	Ismétlődés végdátuma
_taskHasEndDate	bool	false	Meg van-e adva végdátum

GetMonthCalDays() metódus

A havi nézet egy 6×7-es rácsot (42 cellát) generál, ahol minden cella egy napot reprezentál. A hónap első napjától visszafelé számolva kitölti a rácsot az előző hónap utolsó napjaival, majd előre számolva a következő hónap első napjaival. Így a rács mindig teljes heteket tartalmaz, hétfőtől vasárnapig.

A GetMonthCalDays() metódus kódja:

```
private List<DateTime> GetMonthCalDays(DateTime first)
{
   var startDow = (int)first.DayOfWeek;
   var offset = startDow == 0 ? 6 : startDow - 1;  // hétfő = 0
   var result = new List<DateTime>();
   for (var i = -offset; i < 42 - offset; i++)
      result.Add(first.AddDays(i));
   return result;
}
```

Magyarázat: A DayOfWeek enum vasárnapot 0-ként értelmez, de a magyar naptár hétfővel kezdődik, ezért az offset kiszámítása: ha vasárnap (0) az első nap, akkor 6 napot kell visszaszámolni; egyébként startDow - 1 napot. A ciklus -offset-től indul (előző hónap), és 42 - offset-ig megy (6 teljes hét). Ebből a 42 DateTime értékből a Razor template az aktuális hónaphoz tartozó napokat más CSS osztállyal jeleníti meg (fr-month-day-other), jelezve hogy nem az aktuális hónaphoz tartoznak.

GetWeekStart() és GetWeekEnd() metódusok

A heti nézet a _weekOffset változó alapján számítja ki az aktuálisan megjelenített hétre vonatkozó hétfőt:

```
private DateTime GetWeekStart()
{
   var today = DateTime.Today;
   var diff = (7 + (today.DayOfWeek - DayOfWeek.Monday)) % 7;
   return today.AddDays(-diff + _weekOffset * 7);
}
private DateTime GetWeekEnd() => GetWeekStart().AddDays(6);
private DateTime GetDayDate(int dayKey) => GetWeekStart().AddDays(dayKey - 1);
```

A (7 + (today.DayOfWeek - DayOfWeek.Monday)) % 7 képlet azért szükséges, mert a DayOfWeek.Monday értéke 1, de vasárnap (0) esetén a sima kivonás negatív lenne. A modulo 7 garantálja, hogy az eredmény mindig 0 és 6 közé essen. A _weekOffset * 7 hozzáadásával tetszőleges héttel előre vagy hátra lehet navigálni.

Csoport mód működése

A Naptár oldal egyedi funkciója a csoport mód (_groupMode = true). Ha a felhasználó aktiválja, a naptár nem a saját (_schedule, _tasks), hanem a kiválasztott csoport adatait jeleníti meg (_groupSchedule, _groupTasks). A váltás a fejlécben lévő toggle gombbal történik, az adatok betöltése a LoadGroupData() metódussal. Csoport módban az Új bejegyzés panel mindkét füle elérhető: az Órarend fülön csoport-bejegyzés adható hozzá (AddGroupEntry → GroupService.AddGroupScheduleEntryAsync), a Feladat / Esemény fülön pedig csoportos feladat rögzíthető (AddTask → GroupService.AddGroupTaskAsync). Saját módban az AddTask a TaskService.AddTaskAsync metódust hívja.

```
private async Task LoadGroupData()
{
   if (_selectedGroupId <= 0) return;
   _groupSchedule = await GroupService.GetGroupScheduleAsync(_selectedGroupId);
   _groupTasks = await GroupService.GetGroupTasksAsync(_selectedGroupId);
}
```

A _canManageGroup computed property azt ellenőrzi, hogy a bejelentkezett felhasználó jogosult-e szerkesztésre a csoportban, ez tanári csoportnál csak a tulajdonos tanárnak, diákcsoportnál minden tagnak lehetséges.

2.10.2.4 Feladatok.razor

A Feladatok oldal az egyik leggyakrabban használt felület az alkalmazásban. Fő funkciói: a feladatok listázása, szűrése, keresése, rendezése, teljesítettnek jelölése és törlése, valamint új feladatok hozzáadása modális ablakból. Az új esemény hozzáadó form tartalmaz ismétlődési típus-választót (Egyszeri, Heti, Kéthetente, Havonta) és ismétlődő típusnál egy opcionális „Vége dátum" mezőt is, amelyek az AddTaskRequest RecurrenceType és RecurrenceEndDate mezőibe kerülnek.

Állapotváltozók

Változó neve	Típus	Leírás
_tasks	List<TaskDto>	Az összes betöltött feladat
_enrollments	List<EnrollmentDto>	Felvett tantárgyak (az új feladat formhoz)
_filter	string	Aktív szűrő: all / active / soon / overdue / done / típusnév
_search	string	Keresési kulcsszó (cím, tantárgy, leírás alapján)
_sort	string	Rendezési szempont: due / type / subject / name
_theme	string	Aktív téma: light vagy dark
_showAdd	bool	Nyitva van-e az új feladat modális ablak
_newTitle	string	Új feladat neve (beviteli mező)
_newType	string	Új feladat típusa (alapértelmezett: Assignment)
_newDescription	string	Új feladat leírása (opcionális)
_newSubjectId	int	Kiválasztott tantárgy azonosítója
_newDueDate	DateTime	Határidő (alapértelmezett: +7 nap)
_isSaving	bool	Mentés folyamatban (gomb letiltásához)
_isLoading	bool	Adatok betöltése folyamatban
_formError	string	Validációs hibaüzenet a formban

FilteredTasks computed property

A FilteredTasks egy IEnumerable<TaskDto> típusú computed property, amely minden egyes rendereléskor újraszámítódik a _tasks, _filter, _search és _sort változók aktuális értékei alapján. Nem tárolja az eredményt, mindig a friss állapotot tükrözi.

A szűrési logika három lépésben fut:

1. lépés: Státusz/típus szűrés (_filter alapján):

_filter értéke	Szűrési feltétel
"all"	Minden feladat (nincs szűrés)
"active"	Nem teljesített és nem lejárt feladatok (!IsCompleted && !IsOverdue)
"soon"	Hamarosan esedékes, nem teljesített feladatok (IsDueSoon && !IsCompleted)
"overdue"	Lejárt feladatok (IsOverdue)
"done"	Teljesített feladatok (IsCompleted)
Típusnév (pl. "Exam")	Az adott típusú, nem teljesített feladatok

2. lépés: Szöveges keresés (_search alapján): Ha a keresőmező nem üres, csak azok a feladatok maradnak, amelyek neve, tantárgya vagy leírása tartalmazza a keresett szöveget (kis-nagybetű érzéketlenül).

3. lépés: Rendezés (_sort alapján):

_sort értéke	Rendezési logika
"due" (alapértelmezett)	Teljesítés státusz szerint (elvégzendők előre), majd határidő szerint
"type"	Típus szerint ábécésorrendben, azonos típuson belül határidő szerint
"subject"	Tantárgynév szerint ábécésorrendben, majd határidő szerint
"name"	Feladat neve szerint ábécésorrendben

Vizuális kiemelések

A feladatsorok CSS osztálya dinamikusan változik a TaskDto computed property-jei alapján:

Állapot	CSS osztály	Megjelenés
IsOverdue = true	fr-tr-danger	Piros háttér: lejárt feladat
IsDueSoon = true	fr-tr-warn	Sárga/narancs háttér: hamarosan esedékes
IsCompleted = true	fr-tr-done	Áthúzott szöveg, halvány megjelenés
Normál állapot	(alapértelmezett)	Fehér/surface háttér

2.10.2.5 Focus.razor

A Fokusz oldal a FocusTimerService singleton service-re épül. A kördiagram-szerű vizuális visszajelzőt SVG strokeDashoffset animációval valósítja meg, ahol a DashOffset értéke kb:(0..553) a SecondsLeft / TotalSeconds arányából számítódik. A beállítások (fókusz-, rövid szünet-, hosszú szünetidő percei és az AutoSwitch) azonnal érvénybe lépnek, ha a felhasználó módosítja őket, és a timer újraindul. A munkamenet-napló az oldal alján látható listában mutatja az eltelt fókusz-szakaszokat, amelyek az oldalon töltött ideig perzisztálnak (a singleton állapota megtartódik Blazor Server esetén a kapcsolat fenntartásáig). Az oldal szerepkörtől függő CSS-témát alkalmaz: Admin esetén a topbar és a lejátszó gomb piros; a szünet módok (Rövid szünet, Hosszú szünet) gombjain ez a piros téma nem érvényesül, megőrizve a zöld és lila megkülönböztetést.

2.10.2.6 Tantargyak.razor

A Tantárgyak oldal a felvett tantárgyakat kártya nézetben jeleníti meg, színkódolással. Minden kártyán látható a tantárgykód, a felvétel dátuma, az aktív és kész feladatok száma, valamint a heti óraszám. A felhasználó kezdetben 0 tantárggyal rendelkezik, az oldal betöltésekor a Sablon tantárgyak panelből kell manuálisan felvenni a tantárgyakat.

Ezen felül az „Egyéni tantárgy" gomb segítségével a felhasználó saját tantárgyat is létrehozhat szabad szöveges név és kód megadásával, amennyiben a központi listában nem szerepel az adott kurzus. Az egyéni tantárgy privátként jön létre (IsPrivate = 1), kizárólag a létrehozó felhasználó számára látható, és az sp_EnrollCustomSubject tárolt eljáráson keresztül kerül az adatbázisba.

A felhasználó a már felvett tantárgyakat törölni is tudja, így lehetősége van a lista folyamatos karbantartására és a már nem szükséges tantárgyak eltávolítására.

2.10.2.7 Csoport.razor

A Csoport.razor a projekt egyik legösszetettebb oldalkomponense, öt különböző fület kezelve egyetlen komponensben (Tagok, Feladatok, Szavazás, Dolgozatok, Meghívó, a korábbi Tantárgyak fül eltávolításra került). A lista mutatja a felhasználó csoportjait, a részlet panel a kiválasztott csoport tartalmát.

Állapotváltozók

Változó neve	Típus	Leírás
_groups	List<GroupDto>	Felhasználó csoportjainak listája
_invites	List<GroupInviteDto>	Beérkező, elfogadásra váró meghívások
_selectedGroup	GroupDto?	Jelenleg kiválasztott csoport (null ha nincs)
_detailTab	string	Aktív fül: members/tasks/polls/exams/invite
_members	List<GroupMemberDto>	Kiválasztott csoport tagjainak listája
_groupSubjects	List<GroupSubjectDto>	Csoport tagjainak tantárgyai
_groupTasks	List<GroupTaskDto>	Csoport feladatai
_polls	List<PollDto>	Csoport aktív szavazásai
_exams	List<TeacherExamDto>	Csoport dolgozatai
_isLoadingDetail	bool	Csoport részletek töltése folyamatban
_isTeacher	bool	A bejelentkezett felhasználó Tanár szerepkörű-e
_currentUserId	int	Bejelentkezett felhasználó UserID-ja
_canManage	bool (computed)	Jogosult-e a felhasználó szerkesztésre ebben a csoportban
_showRename	bool	Nyitva van-e az átnevező szerkesztőmező
_renameText	string	Az átnevező beviteli mező tartalma
_isRenaming	bool	Átnevezés folyamatban (gomb letiltáshoz)
_confirmOpen	bool	Nyitva van-e a megerősítő modal ablak
_confirmDanger	bool	Veszélyes művelet-e (piros stílus)
_confirmMessage	string	A confirm modal szövege

Csoport létrehozás állapotváltozói

Változó neve	Típus	Leírás
_showCreate	bool	Nyitva van-e a csoport-létrehozó panel
_newGroupName	string	Új csoport neve (beviteli mező)
_createError	string	Hibaüzenet a létrehozáshoz
_isCreating	bool	Létrehozás folyamatban (gomb letiltásához)

Meghívás állapotváltozói

Változó neve	Típus	Leírás
_inviteEmail	string	Meghívandó e-mail cím (beviteli mező)
_inviteError	string	Hibaüzenet a meghíváshoz
_isInviting	bool	Meghívás küldése folyamatban

Szavazás állapotváltozói

Változó neve	Típus	Leírás
_pollQuestion	string	Szavazás kérdése
_pollOptionsText	string	Opciók szövege (soronként egy, alapértelmezett: Igen/Nem/Talán)
_pollExpires	DateTime?	Szavazás lejárati ideje (opcionális)
_pollError	string	Hibaüzenet a szavazás létrehozásához
_isSavingPoll	bool	Szavazás mentése folyamatban

Dolgozat állapotváltozói

Változó neve	Típus	Leírás
_examTitle	string	Dolgozat neve
_examSubject	string	Tantárgy neve (szabad szöveg)
_examDate	DateTime	Dolgozat dátuma (alapértelmezett: +7 nap)
_examDesc	string	Megjegyzés (opcionális)
_examError	string	Hibaüzenet
_isSavingExam	bool	Mentés folyamatban

Az öt fül működése

Fül (_detailTab értéke)	Látható kinek	Tartalom
members	Minden tag	Csoporttagok listája névvel, e-maillel, csatlakozás dátumával. A tulajdonosnál korona ikon jelzi az IsOwner státuszt.
subjects	Minden tag	A csoport tagjainak felvett tantárgyai (DISTINCT lista). Forrás: sp_GetGroupSubjects, csak a Student szerepkörű tagok tantárgyai jelennek meg.
tasks	Minden tag	Csoporton belüli feladatok határidővel, típussal, elvégzési státusszal. Forrás: sp_GetGroupTasks.
polls	Minden tag	Aktív szavazások listája. Szavazás leadása opcióra kattintással. Szavazat módosítható. Új szavazás csak tanárnak vagy tulajdonosnak érhető el.
exams	Minden tag	Tanár által meghirdetett dolgozatok dátummal és tantárggyal. Forrás: sp_GetGroupExams.
invite	Csak tulajdonos / tanár	E-mail cím alapján új tag meghívása. A _canManage computed property szabályozza a láthatóságot.

Csoport kiválasztásának folyamata

Amikor a felhasználó a bal oldali listából rákattint egy csoportra, a SelectGroup() metódus fut le:

```
private async Task SelectGroup(GroupDto group)
{
   _selectedGroup = group;
   _detailTab = "members";
   _isLoadingDetail = true;
   StateHasChanged();
   _members = await GroupService.GetMembersAsync(group.GroupID);
   _groupSubjects = await GroupService.GetGroupSubjectsAsync(group.GroupID);
   _groupTasks = await GroupService.GetGroupTasksAsync(group.GroupID);
   _polls = await GroupService.GetGroupPollsAsync(group.GroupID);
   _exams = await GroupService.GetGroupExamsAsync(group.GroupID);
   _isLoadingDetail = false;
   StateHasChanged();
         }
```

A StateHasChanged() kétszeres hívása szándékos: az első hívás azonnal megjeleníti a betöltési indikátort (_isLoadingDetail = true), a második hívás a betöltött adatokat rendereli ki. E nélkül a felhasználó csak az összes API hívás után látná a frissülést.

2.10.2.8 Home.razor

A Home.razor az egyetlen bejelentkezés nélkül is elérhető nyitóoldal. Az oldal építőelemei: egy hero szekció a „Kezdés most” és „Már van fiókom” gombokkal, egy Funkciók szekció diák/tanár tab-váltóval, egy „Hogyan működik?” lépés-kártya szekció, egy CTA „Készen állsz?” szekció Regisztráció és Bejelentkezni gombokkal, valamint az oldal legalján egy elérhetőségi sáv.

Az elérhetőségi sáv (.contact-bar) a page-bg-wrapper div lezárása után helyezkedik el a Home.razor fájlban, így nem része a rácsos háttérnek. A sáv háttere egy lila átmenet.Tartalma: egy boríték SVG ikon, a „Kérdésed van? Írj nekünk:” szöveg, és egy mailto: hivatkozás **info@feladatradar.hu** címre. Hover állapotban a link amber színre vált (#fbbf24) az aláhúzás színével együtt, hogy a lila háttéren jól kiemelkedjen. Mobilon (480px alatt) az elemek egymás alá kerülnek, középre igazítva.

A Home.razor CSS-je teljesen izoláltan, a fájl alján lévő <style> blokkban található, nem érint más oldalakat. A navigációs sáv (.site-nav) fix pozíción helyezkedik el, háttér-blur effekttel (backdrop-filter: blur(20px)). Az oldalak közötti átmenetek sima görgötéssel valósulnak meg a ScrollTo() C# metóduson keresztül, amely a JavaScript scrollIntoView API-t hívja IJSRuntime-on keresztül.

2.10.3 Frontend Service réteg

A Blazor Server frontend Service osztályai a backend API végpontjait hívják HttpClient segítségével. Minden frontend Service osztály a konstruktorban megkapja a HttpClient és az AuthService példányt Dependency Injection-ön keresztül. Az API hívások előtt minden metódus meghívja az _authService.InitializeAsync() metódust, amely a JWT tokent a localStorage-ból betölti és a HttpClient Authorization fejlécébe csatolja.

2.10.3.1 AuthService (Frontend)

Az AuthService a frontend központi szolgáltatása: kezeli a bejelentkezést, regisztrációt, kijelentkezést és a token-állapotot. A Blazored.LocalStorage NuGet csomag segítségével tárolja a JWT tokent és a felhasználói adatokat a böngésző localStorage-ában.

Az InitializeAsync() metódus:

```
public async Task InitializeAsync()
{
    if (_isInitialized) return;
    try
    {
        _token = await _localStorage.GetItemAsync<string>("authToken");
        _currentUser = await _localStorage.GetItemAsync<UserDto>("currentUser");
        if (!string.IsNullOrEmpty(_token))
            _httpClient.DefaultRequestHeaders.Authorization =
                new AuthenticationHeaderValue("Bearer", _token);
        _isInitialized = true;
    }
    catch { }
}
```

Az _isInitialized flag biztosítja, hogy a localStorage olvasás csak egyszer történik meg. Az IsAuthenticated property egyszerűen ellenőrzi, hogy a _token nem üres-e.

A Login metódus a api/Auth/login végpontra küld POST kérést. Ha a válasz nem sikeres HTTP státuszkódú (!response.IsSuccessStatusCode), azonnal hibával tér vissza. Sikeres válasz esetén a tokent és a felhasználói adatokat az ApplyLoginResult() privát segédmetóduson keresztül menti memóriába és localStorage-ba. Ha a localStorage mentés nem sikerül (például prerendering alatt), a SaveToLocalStorageAsync() metódussal később pótolható.:

```
public async Task<LoginResponse?> Login(LoginRequest request)
{
   try
   {
      var response = await _httpClient.PostAsJsonAsync("api/Auth/login", request);
      if (response.IsSuccessStatusCode)
      {
         var result = await response.Content.ReadFromJsonAsync<LoginResponse>();
         if (result != null && result.Status == "SUCCESS"
            && !string.IsNullOrEmpty(result.Token))
         {
            _token = result.Token;
            _currentUser = result.User;
            _httpClient.DefaultRequestHeaders.Authorization =
               new AuthenticationHeaderValue("Bearer", _token);
            try
            {
               await _localStorage.SetItemAsync("authToken", _token);
               await _localStorage.SetItemAsync("currentUser", _currentUser);
            }
            catch { /* Will be saved later */ }
         }
         return result;
      }
     return new LoginResponse { Status = "ERROR", Message = "Hibás felhasználónév vagy jelszó!" };
   }
   catch (Exception ex)
   {
            return new LoginResponse { Status = "ERROR", Message = $"Kapcsolódási hiba:
{ex.Message}" };
   }
}
```

A Register metódus hasonló mintát követ: a api/Auth/register végpontra küld POST-ot, sikeres regisztráció után azonnal elmenti a tokent, így a felhasználó regisztráció után automatikusan be van jelentkezve.

A Logout metódus törli a memóriából és a localStorage-ból is az adatokat:

```
public async Task Logout()
{
   _token = null;
   _currentUser = null;
   _httpClient.DefaultRequestHeaders.Authorization = null;
   await _localStorage.RemoveItemAsync("authToken");
   await _localStorage.RemoveItemAsync("currentUser");}
```

UpdateUsername metódus

Az UpdateUsername metódus PUT kérést küld az api/Auth/update-username végpontra. Sikeres válasz esetén (Status == “SUCCESS”) frissíti a _currentUser objektumot, kicseréli a tárolt JWT tokent a szerver által visszaadott új tokenre (amely az új felhasználónevet már tartalmazza), és elmenti a localStorage-ba. Ez biztosítja, hogy a Fiok.razor azonnal az új felhasználónevet jelenítse meg.

ChangePassword metódus

A ChangePassword metódus PUT kérést küld az api/Auth/change-password végpontra, CurrentPassword és NewPassword mezőkkel (a ConfirmNewPassword a frontend helyi ellenőrzéséhez kell, az API-ra nem kerül el). Sikertelen jelenlegi jelszó esetén a backend HTTP 400-at ad vissza a hibaüzenettel, amelyet a Fiok.razor piros üzenetként jelenít meg.

2.10.3.2 TaskService (Frontend)

A TaskService az egyéni feladatok kezelését végzi. A ParseResponse segédmetódus minden válasznál egységesen kezeli a SubjectResponse deszerializálást:

```
private async Task<SubjectResponse> ParseResponse(HttpResponseMessage response)
{
   try
   {
      var result = await response.Content.ReadFromJsonAsync<SubjectResponse>();
      return result ?? new SubjectResponse { Status = "ERROR", Message = "Ismeretlen hiba." };
   }
   catch
   {
            return new SubjectResponse { Status = "ERROR", Message = $"Szerver hiba:
{response.StatusCode}" };
   }
}
```

A szolgáltatás öt fő metódust tartalmaz:

GetMyTasksAsync(): lekéri az összes feladatot a api/Task/my-tasks végpontról. Ha a válasz nem sikeres vagy kivétel keletkezik, üres listát ad vissza:

```
public async Task<List<TaskDto>> GetMyTasksAsync()
{
   await _authService.InitializeAsync();
   try
   {
      var response = await _httpClient.GetAsync("api/Task/my-tasks");
      if (!response.IsSuccessStatusCode) return new();
      return await response.Content.ReadFromJsonAsync<List<TaskDto>>() ?? new();
   }
   catch { return new(); }
}
```

AddTaskAsync(AddTaskRequest request), POST kérést küld az api/Task/add végpontra. CompleteTaskAsync(int taskId), PUT kérést küld az api/Task/complete/{taskId} végpontra. UncompleteTaskAsync(int taskId), PUT kérést küld az api/Task/uncomplete/{taskId} végpontra a teljesítés visszavonásához. DeleteTaskAsync(int taskId), DELETE kérést küld az api/Task/delete/{taskId} végpontra.

A frontend TaskDto modell tartalmaz számított property-ket is, amelyek kliensoldalon számítódnak:

```
public bool IsOverdue => !IsCompleted && DueDate < DateTime.Now;
public bool IsDueSoon => !IsCompleted && !IsOverdue && DueDate <= DateTime.Now.AddDays(3);
public bool IsRecurring => RecurrenceType != "None";

public string DaysLeft
{
   get
   {
      if (IsCompleted) return "Kész";
      var diff = (DueDate - DateTime.Now).TotalDays;
      if (diff < 0) return $"{(int)Math.Abs(diff)} napja lejárt";
      if (diff < 1) return "Ma jár le!";
      return $"{(int)diff} nap múlva";
   }
}
```

Az IsOverdue, IsDueSoon és DaysLeft property-k a Feladatok.razor oldalon a vizuális jelzésekhez (piros/sárga háttér, lejárati szöveg) szállítanak értéket.

2.10.3.3 SubjectService (Frontend)

A SubjectService a tantárgykezelésért felel. Öt metódusa van:

GetAvailableSubjectsAsync(): GET api/Subject/available: az összes elérhető (publikus) tantárgy listáját kéri le. GetMyEnrollmentsAsync(): GET api/Subject/my-enrollments: a felhasználó által felvett tantárgyakat kéri le. EnrollAsync(int subjectId): POST api/Subject/enroll: tantárgy felvétele. DropAsync(int subjectId): DELETE api/Subject/drop/{subjectId}: tantárgy leadása.

EnrollCustomAsync(string subjectName, string subjectCode): POST api/Subject/enroll-custom: egyedi (privát) tantárgy létrehozása és felvétele:

```
public async Task<SubjectResponse> EnrollCustomAsync(string subjectName, string subjectCode)
{
   await _authService.InitializeAsync();
   var response = await _httpClient.PostAsJsonAsync("api/Subject/enroll-custom", new
   {
      subjectName,
      subjectCode
   });
   return await response.Content.ReadFromJsonAsync<SubjectResponse>()
         ?? new SubjectResponse { Status = "ERROR", Message = "Ismeretlen hiba." };
}
```

2.10.3.4 ScheduleService (Frontend)

A ScheduleService az egyéni órarend kezelését végzi. Három metódusa van:

GetMyScheduleAsync(): GET api/Schedule/my-schedule: a felhasználó órarendjét kéri le. A válasz List<ScheduleEntryDto>, amely tartalmazza a SubjectName és SubjectCode JOIN eredményeket is.

AddEntryAsync(AddScheduleRequest request): POST api/Schedule/add: új órarend bejegyzés hozzáadása. Speciális hibakezelést tartalmaz a 401 Unauthorized válaszra:

```
public async Task<SubjectResponse> AddEntryAsync(AddScheduleRequest request)
{
   await _authService.InitializeAsync();
   try
   {
      var response = await _httpClient.PostAsJsonAsync("api/Schedule/add", request);
      if (response.StatusCode == System.Net.HttpStatusCode.Unauthorized)
         return new SubjectResponse
         {
            Status = "ERROR",
            Message = "Nincs jogosultságod (401). Jelentkezzen be újra!"
         };
      return await response.Content.ReadFromJsonAsync<SubjectResponse>()
            ?? new SubjectResponse { Status = "ERROR", Message = "Ismeretlen hiba." };
   }
   catch (Exception ex)
   {
      return new SubjectResponse { Status = "ERROR", Message = $"Hiba: {ex.Message}" };
   }
}
```

DeleteEntryAsync(int entryId): DELETE api/Schedule/delete/{entryId}: bejegyzés törlése.

A frontend ScheduleEntryDto modell számított property-t is tartalmaz:

```
public string RecurrenceLabel => RecurrenceType switch
{
   "Weekly" => "Heti",
   "Biweekly" => "Kéthetente",
   "Monthly" => "Havonta",
   _ => "Egyszeri"
};
```

2.10.3.5 GroupService (Frontend)

A GroupService a legösszetettebb frontend szolgáltatás, 17 publikus metódussal. Ezek a csoportkezelés teljes funkcionalitását lefedik.

Csoport műveletek:

GetMyGroupsAsync(): GET api/Group/my-groups: a felhasználó csoportjainak listája. CreateGroupAsync(string groupName): POST api/Group/create: új csoport létrehozása. LeaveGroupAsync(int groupId): POST api/Group/{groupId}/leave: kilépés csoportból.

RenameGroupAsync(int groupId, string newName): PUT api/Group/{groupId}/rename: csoport átnevezése. DeleteGroupAsync(int groupId): DELETE api/Group/{groupId}: csoport törlése (csak tulajdonos). RemoveGroupMemberAsync(int groupId, int memberId): DELETE api/Group/{groupId}/members/{memberId}: tag eltávolítása (csak tulajdonos).

Meghívás műveletek:

GetMyInvitesAsync(): GET api/Group/my-invites: bejövő meghívások listája. InviteAsync(int groupId, string email): POST api/Group/{groupId}/invite: meghívás küldése e-mail cím alapján. AcceptInviteAsync(int inviteId): POST api/Group/invite/{inviteId}/accept: meghívás elfogadása. DeclineInviteAsync(int inviteId): POST api/Group/invite/{inviteId}/decline: meghívás elutasítása.

Csoportadatok lekérése:

GetMembersAsync(int groupId): GET api/Group/{groupId}/members. GetGroupSubjectsAsync(int groupId): GET api/Group/{groupId}/subjects. GetGroupScheduleAsync(int groupId): GET api/Group/{groupId}/schedule. GetGroupTasksAsync(int groupId): GET api/Group/{groupId}/tasks. AddGroupScheduleEntryAsync(int groupId, AddScheduleRequest request): POST api/Group/{groupId}/schedule/add. AddGroupTaskAsync(int groupId, AddGroupTaskRequest request): POST api/Group/{groupId}/tasks/add. DeleteGroupScheduleEntryAsync(int groupId, int entryId), DELETE api/Group/{groupId}/schedule/{entryId}.

DeleteGroupTaskAsync(int groupId, int taskId): DELETE api/Group/{groupId}/tasks/{taskId}.

Minden lekérdező metódus ugyanazt a mintát követi:

```
public async Task<List<GroupMemberDto>> GetMembersAsync(int groupId)
{
   await _authService.InitializeAsync();
   try
   {
      var response = await _httpClient.GetAsync($"api/Group/{groupId}/members");
      if (!response.IsSuccessStatusCode) return new();
      return await response.Content.ReadFromJsonAsync<List<GroupMemberDto>>() ?? new();
   }
   catch { return new(); }
}
```

2.10.3.6 TeacherService (Frontend)

A TeacherService a tanári funkciókat kezeli, két fő területen: szavazások és dolgozatok.

Szavazás műveletek:

CreatePollAsync(int groupId, string question, List<string> options, DateTime? expiresAt): POST api/Teacher/poll/create. GetGroupPollsAsync(int groupId): GET api/Teacher/{groupId}/polls. VoteAsync(int pollId, int optionId): POST api/Teacher/poll/{pollId}/vote. DeletePollAsync(int pollId): DELETE api/Teacher/poll/{pollId}.

Dolgozat műveletek:

CreateExamAsync(int groupId, string title, DateTime examDate, string? description, string? subjectName), POST api/Teacher/exam/create. GetGroupExamsAsync(int groupId), GET api/Teacher/{groupId}/exams. DeleteExamAsync(int examId), DELETE api/Teacher/exam/{examId}.

A szavazás létrehozása például:

```
public async Task<SubjectResponse> CreatePollAsync(int groupId, string question,
   List<string> options, DateTime? expiresAt = null)
{
   await _authService.InitializeAsync();
   try
   {
      var response = await _httpClient.PostAsJsonAsync("api/Teacher/poll/create", new
      {
         groupID = groupId,
         question,
         options,
         expiresAt
      });
      return await ParseResponse(response);
   }
   catch (Exception ex)
   {
      return new SubjectResponse { Status = "ERROR", Message = ex.Message };
   }
}
```

2.10.3.7 FocusTimerService

A FocusTimerService különleges: AddSingleton élettartamú, így az állapota megőrződik oldalnavigáció közben is. Ez azért szükséges, mert a Pomodoro-időzítőnek akkor is futnia kell, ha a felhasználó átvalt a Feladatok vagy Naptár oldalra. Nem HttpClient-et használ, hanem tisztán kliensoldali állapotgépként működik.

Az időzítő három módban operál: focus (alapértelmezett 25 perc), short szünet (5 perc) és long szünet (15 perc).

A ToggleTimer() metódus a System.Threading.Timer osztályt használja, 1 másodperces intervallummal:

```
public void ToggleTimer()
{
    if (IsRunning)
    {
        _timer?.Change(Timeout.Infinite, Timeout.Infinite);
        IsRunning = false;
        OnTick?.Invoke();
        return;
    }
    IsRunning = true;
    _timer?.Dispose();
    _timer = new System.Threading.Timer(_ => {
        if (SecondsLeft > 0)
        {
            SecondsLeft--;
            OnTick?.Invoke();
        }
        else
        {
            _timer?.Change(Timeout.Infinite, Timeout.Infinite);
            IsRunning = false;
            var finishedMode = Mode;
            var finishedMinutes = TotalSeconds / 60;
            SessionLog.Add(new SessionEntry(finishedMode, finishedMinutes,
                DateTime.Now, CurrentSessionType));
            if (finishedMode == "focus") CompletedSessions++;
            if (AutoSwitch)
            {
                Mode = finishedMode == "focus" ? "short" : "focus";
                SecondsLeft = TotalSeconds;
                ToggleTimer();
            }
            else
            {
                SecondsLeft = TotalSeconds;
                OnTick?.Invoke();
            }
        }
    }, null, TimeSpan.FromSeconds(1), TimeSpan.FromSeconds(1));
}
```

Amikor a SecondsLeft eléri a nullát, az időzítő naplózza a befejezett munkamenetet a SessionLog listába, növeli a CompletedSessions számlálót (ha a mód „focus” volt), majd az AutoSwitch beállítás alapján automatikusan átvált a következő módba.

A DashOffset és StrokeOffset számított property-k az SVG körös animációhoz szállítanak értéket. A képlet: 553.0 * (SecondsLeft / TotalSeconds), ahol 553 a 2πr értéke (r = 88 pixel). Ahogy a hátralévő idő csökken, az SVG kör fokozatos „feltöltődését” eredményezi.

A SessionEntry record tárolja az elvégzett munkamenetek naplóját:

```
public record SessionEntry(string Mode, int Minutes, DateTime FinishedAt,
    string SessionType = "tanulas");
```

2.10.4 Blazor életciklus és állapotkezelés

A Blazor Server modellben a komponensek életciklusa eltér a hagyományos webalkalmazásokétól. Az alábbiakban a FeladatRadar szempontjából legfontosabb életciklus-eseményeket ismertetjük.

2.10.4.1 OnInitializedAsync

Minden .razor komponens az OnInitializedAsync() metódusban tölti be a kezdeti adatokat. Például a Feladatok.razor oldal:

```
protected override async Task OnInitializedAsync()
{
    tasks = (await TaskService.GetMyTasks())?.ToList() ?? new();
    subjects = (await SubjectService.GetMyEnrollments())?.ToList() ?? new();
}
```

Ez a minta biztosítja, hogy az oldal renderelésekor már rendelkezésre állnak az adatok. A null-koaleszcens operátor (?? new()) garantálja, hogy a lista soha nem null, így a Razor template @foreach ciklusa nem dob NullReferenceException-t.

2.10.4.2 StateHasChanged és InvokeAsync

A FocusTimerService OnTick eseménye nem a fő szálon fut (System.Threading.Timer háttérszálat használ), ezért a Blazor komponensnek InvokeAsync-kel kell jelezni a UI frissítési igényt:

```
Timer.OnTick += () => InvokeAsync(StateHasChanged);
```

Az InvokeAsync biztosítja, hogy a StateHasChanged hívás a megfelelő szinkronizációs kontextusban fusson. Ennek hiányában InvalidOperationException keletkezne.

2.10.4.3 IDisposable implementáció

Azok a komponensek, amelyek eseménykezelőt regisztrálnak (pl. a Focus.razor a Timer.OnTick-re), az IDisposable interfészt implementálják és a Dispose() metódusban leirátkoznak az eseményről. Ez megelőzi a memóriaszivárgást és a „fantom UI frissítéseket” azáltal, hogy a komponens megsemmisítése után nem hívódik meg a StateHasChanged.

2.10.5 CSS architektúra és témarendszer

Az összes oldal egyedi CSS-t tartalmaz a .razor fájl <style> szekciójában, nem globális stíluslapot. Ez az önálló (self-contained) megközelítés biztosítja, hogy az egyes oldalak stílusai ne zavarják egymást. A CSS változókon alapuló témarendszer lehetővé teszi a sötét/világos mód váltását:

```
[data-theme="light"] {
   --bg-primary: #f7f5f0;
   --bg-secondary: #ffffff;
   --text-primary: #1a1a2e;
   --text-secondary: #6b6b7b;
   --border: #e0ddd6;
   --accent: #4f46e5;
   --accent-light: #ede9fe;
}
[data-theme="dark"] {
   --bg-primary: #0f1117;
   --bg-secondary: #1a1d26;
   --text-primary: #e8e6e0;
   --text-secondary: #9b99a0;
   --border: #2a2d3a;
   --accent: #6366f1;
   --accent-light: #312e6b;
}
}
```

A JavaScript függőség minimalizálva van: a témaváltó gomb C# @onclick eseménnyel kezelt, és a JS-ben csupán a localStorage olvasás/írás (IJSRuntime segítségével) és az SVG animációk (Fókusz oldal) valósulnak meg. Ez összhangban van a Blazor Server "C#-first" megközelítésével.

2.11 Fejlesztési lehetőségek

2.11.1 Értesítési rendszer

Jelenleg az alkalmazás nem küld automatikus értesítéseket a közeledő határidőkről. Egy jövőbeli fejlesztés lehetne böngésző-push értesítések integrálása (Web Push API + Service Worker), amely 24 vagy 48 órával a határidő előtt figyelmezteti a diákot.

2.11.2 Mobilalkalmazás

A backend RESTful API-ja lehetővé tenné, hogy a jövőben .NET MAUI vagy React Native alapú mobilalkalmazás is fogyassza. Egy dedikált mobilapp push-értesítéseket is tudna küldeni, és offline módban is képes lenne megjeleníteni a korábban letöltött adatokat.

2.11.3 Fájlfeltöltés feladatokhoz

A diákok jelenleg csak szöveges leírást adhatnak feladataikhoz. Egy hasznos fejlesztés lenne fájlmellékletek (PDF, kép) feltöltési lehetősége, amelyeket az Azure Blob Storage vagy egy helyi fájlrendszer tárolna. Ez különösen hasznos lenne csoportos feladatok esetén, ahol a tagok ugyanarra a dokumentumra hivatkoznak.

2.11.4 Valós idejű csevegés csoportokban

A Blazor Server már rendelkezik SignalR-alapú valós idejű kommunikációval (az interaktív UI frissítéshez). Ez a technika kiterjeszthető lenne egy egyszerű csoportos chat funkcióra, ahol a csoport tagjai valós időben kommunikálhatnak egymással az alkalmazáson belül.

Felhasználói dokumentáció

Ez a fejezet a FeladatRadar alkalmazás teljes végfelhasználói dokumentációját tartalmazza. Lépésről lépésre, részletesen bemutatja az összes funkciót, a regisztrációtól kezdve az admin felület használatáig. A dokumentáció mindhárom szerepkörre Diák (Student), Tanár (Teacher) és Adminisztrátor (Admin) kitér.

3.1 Rendszerkövetelmények és hozzáférés

A FeladatRadar egy böngészőalapú webalkalmazás, amelynek használatához nem szükséges semmilyen szoftver telepítése. Az alkalmazás bármilyen modern böngészőből elérhető, asztali számítógépről és mobileszközről egyaránt.

3.1.1 Hardver- és szoftverkövetelmények

Az alkalmazás használatához az alábbi hardver- és szoftverkövetelmények ajánlottak:

Összetevő	Követelmény
Eszköz	Asztali számítógép, laptop, tablet vagy okostelefon
Operációs rendszer	Windows 10/11, macOS 12+, Linux (Ubuntu 20.04+), Android 10+, iOS 15+
Böngésző	Google Chrome 100+, Microsoft Edge 100+, Mozilla Firefox 100+, Safari 15+ (ajánlott: Chrome vagy Edge)
RAM	Minimum 2 GB (ajánlott: 4 GB vagy több)
Internetkapcsolat	Folyamatos kapcsolat szükséges (Blazor Server / SignalR WebSocket)
Képernyőfelbontás	Minimum 360×640 px (mobil); ajánlott asztali gépen: 1280×720 px vagy magasabb
Telepítendő szoftver	Nincs szükség telepítésre: a böngésző elegendő

3.1.2 Hálózati követelmények

Az alkalmazás Blazor Server technológián alapul, ami azt jelenti, hogy folyamatos internetkapcsolat szükséges a használathoz. A SignalR WebSocket kapcsolat biztosítja a valós idejű frissítéseket. Ha az internetkapcsolat megszakad:

- A böngészőben megjelenik a Reconnecting... üzenet.
- Az alkalmazás automatikusan megpróbál újracsatlakozni.
- Ha az újracsatlakozás sikertelen, az oldalt frissíteni kell.

3.1.3 Az alkalmazás elérése

Az alkalmazás az alábbi URL-en érhető el: https://feladatradar.hu

Az oldal megnyitásakor az alkalmazás automatikusan a kezdőoldalra navigál.

3.2 Fiók létrehozása és bejelentkezés

3.2.1 A kezdőoldal

Az alkalmazás megnyitásakor a kezdőoldal jelenik meg, amely bemutatja az alkalmazás főbb funkcióit. A bal felső sarokban a FeladatRadar logó, az oldal közepén a Bejelentkezés és Regisztráció gombok találhatók. Ez az egyetlen oldal, amely bejelentkezés nélkül is elérhető, minden más oldal automatikusan visszairányít a bejelentkezési oldalra, ha a felhasználó nincs hitelesítve.

Fontos megjegyzés a hozzáférési jogosultságokról: A FeladatRadar valamennyi funkciója a feladatkezelés, az órarend, a fókusz időzítő, a tantárgy-nyilvántartás, a csoportkezelés és az adminisztrátori felület kizárólag bejelentkezett felhasználók számára érhető el. Az egyetlen kivétel a kezdőoldal (Home.razor), amely nyilvánosan elérhető, és az alkalmazás bemutatóoldaláként szolgál. Amennyiben egy nem hitelesített felhasználó megpróbál bármely más oldalt közvetlenül elérni a rendszer automatikusan visszairányítja a bejelentkezési oldalra. Ez a viselkedés a Blazor Server routingszintjén van megvalósítva, és minden védett oldal egyformán érvényesíti.

3.2.2 Regisztráció

Új felhasználói fiók létrehozásához kattintson a Regisztráció gombra. A regisztrációs oldal az alábbi adatokat kéri be:

Mező neve	Kötelező?	Elvárások	Példa
Keresztnév	Igen	Nem üres	János
Vezetéknév	Igen	Nem üres	Kovács
Felhasználónév	Igen	Min. 3, max. 50 karakter, egyedi	kovacs.janos
Email cím	Igen	Érvényes email formátum, egyedi	janos@iskola.hu
Szerepkör	Igen	Diák vagy Tanár kártya kiválasztása	Diák
Jelszó	Igen	Min. 8 karakter, nagybetű, szám, spec. karakter	Pelda123!
Jelszó megerősítése	Igen	Meg kell egyeznie a jelszóval	Pelda123!

A jelszóerősség-ellenőrző

Gépelés közben a jelszó mező alatt egy valós idejű erősség-jelző jelenik meg, amely négy feltételt ellenőriz:

- Min. 8 karakter hosszú legyen
- Tartalmazzon legalább egy nagybetűt
- Tartalmazzon legalább egy számot
- Ajánlott legalább egy speciális karaktert (pl. !, @, #, $)

Minden teljesített feltételnél egy zöld pipa (✅), nem teljesítettnél piros X (❌) jelenik meg. A speciális karakter esetében sárga háromszög ⚠️ jelenik meg. A jelszó mező jobb sarkában egy szem ikonra kattintva a jelszó megjeleníthető vagy elrejthető.

A szerepkör kiválasztása

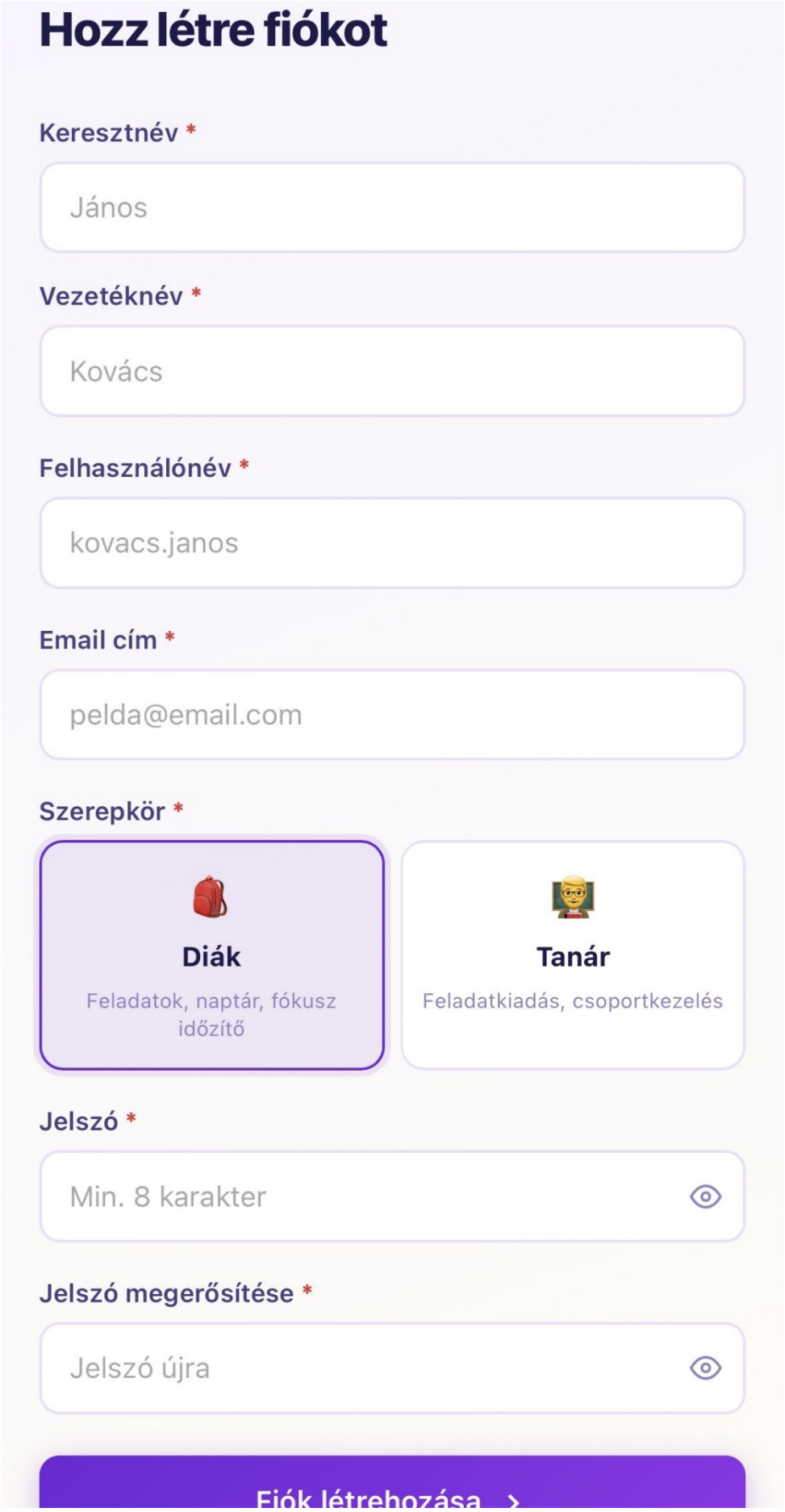
A szerepkör-választó két kártyát jelenít meg:

- Diák (Student) kártya, ikon: 🎒, Feladatok, naptár, fókusz időzítő. Ezt válassza, ha tanulóként szeretné használni az alkalmazást.
- Tanár (Teacher) kártya, ikon: 👨‍🏫, Feladatkiadás, csoportkezelés. Ezt válassza, ha oktatóként, edzőként vagy csoportvezetőként szeretné használni.

A regisztráció folyamata lépésről lépésre

- Nyissa meg a regisztrációs oldalt a kezdőlapról.
- Töltse ki a Keresztnév és Vezetéknév mezőket.
- Adja meg a kívánt felhasználónevet (legalább 3 karakter, pl. kovacs.janos).
- Adja meg az e-mail címét (ezt fogják a csoportos meghívók is használni).
- Kattintson a kívánt szerepkör kártyájára (Diák vagy Tanár).
- Adjon meg egy erős jelszót: figyelje a valós idejű erősség-ellenőrzőt.
- Írja be a jelszót még egyszer a megerősítő mezőbe.
- Kattintson a Regisztráció gombra.

Sikeres regisztráció esetén az alkalmazás a bejelentkezési oldalra irányít át („Sikeres regisztráció! Átirányítás a bejelentkezési oldalra..." üzenettel). A regisztráció után külön be kell jelentkezni a megadott felhasználónévvel és jelszóval.


Hozz létre fiókot
Keresztnév *
János
Vezetéknév *
Kovács
Felhasználónév *
kovacs.janos
Email cím *
pelda@email.com
Szerepkör *
Diák
Feladatok, naptár, fókusz időzítő
Tanár
Feladatkiadás, csoportkezelés
Jelszó *
Min. 8 karakter
Jelszó megerősítése *
Jelszó újra
Fiók létrehozása

Lehetséges hibaüzenetek regisztrációkor

Hibaüzenet	Ok	Megoldás
A felhasználónév már foglalt!	Már létezik fiók ezzel a névvel.	Válasszon másik felhasználónevet.
Az email cím már használatban van!	Már regisztráltak ezzel az email-lel.	Használjon másik email-t, vagy jelentkezzen be.
A felhasználónév legalább 3 karakter legyen!	Túl rövid felhasználónév.	Adjon meg legalább 3 karakteres nevet.
A jelszó legalább 8 karakter legyen!	Túl rövid jelszó.	Adjon meg legalább 8 karakteres jelszót.
A jelszavak nem egyeznek!	A két jelszó mező tartalma eltér.	Írja be ugyanazt a jelszót mindkét mezőbe.
Érvénytelen email formátum!	Hibás email formátum.	Adjon meg érvényes email-t (pl. nev@domain.hu).
Szerepkör kiválasztása kötelező!	Nem kattintott szerepkör kártyára.	Kattintson a Diák vagy Tanár kártyára.

⚠️ Az Admin szerepkör nem elérhető a regisztrációs felületen, és a backend szerver oldalon is blokkolja az Admin szerepkörű regisztrációt. Admin fiókot csak egy már meglévő Admin felhasználó hozhat létre a szerepkör-módosító felületen, vagy közvetlenül az adatbázisba szúrható be.

3.2.3 Bejelentkezés

Visszatérő felhasználóknak a kezdőlapon a Bejelentkezés gombra kell kattintani. A bejelentkezési oldal egyszerű: felhasználónév és jelszó mezőkből, valamint a Bejelentkezés gombból áll.

Bejelentkezési folyamat lépésről lépésre:

- Nyissa meg a bejelentkezési oldalt.
- Adja meg a felhasználónevét (nem az email-t!).
- Adja meg a jelszavát. A szem ikonnal megjelenítheti a beírt jelszót.
- Kattintson a Bejelentkezés gombra.
- Sikeres bejelentkezés esetén az alkalmazás a Főoldalra navigál.

Bejelentkezési hibaüzenetek:

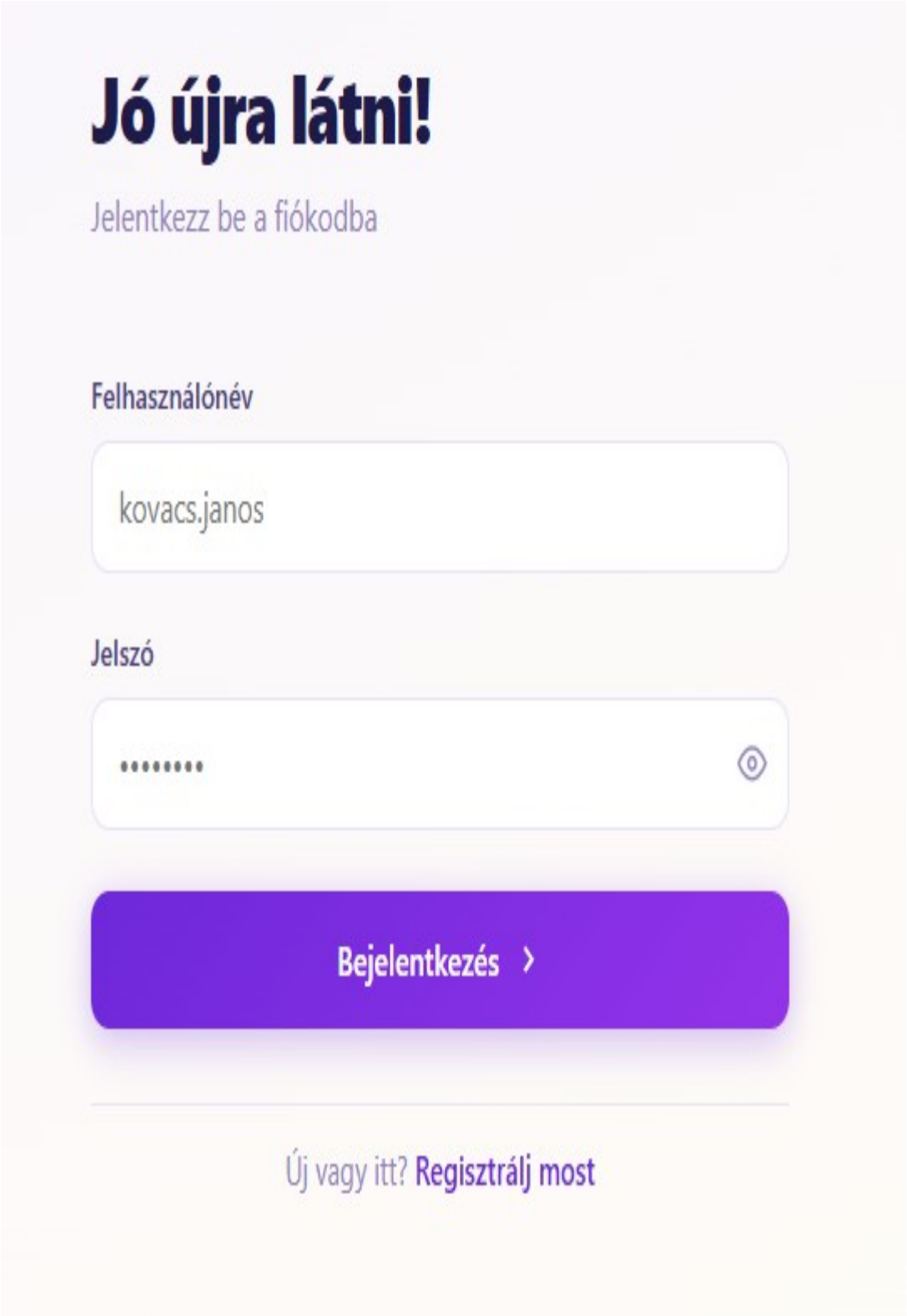
Hibaüzenet	Lehetséges ok
Hibás felhasználónév vagy jelszó!	Nem létező felhasználónév, vagy helytelen jelszó.
Kapcsolódási hiba!	A backend szerver nem elérhető vagy nem fut.

ℹ️ Biztonsági okokból az alkalmazás nem árulja el, hogy a felhasználónév létezik-e, mindig az általános hibaüzenet jelenik meg, hogy megakadályozza a felhasználónév-kitaláló támadást.

A bejelentkezési állapot megőrzése

Sikeres bejelentkezés után az alkalmazás a JWT tokent a böngésző localStorage-ában tárolja. Ez azt jelenti:

- A böngésző bezárása és újbóli megnyitása után a felhasználó bejelentkezve marad.
- A token lejárata az appsettings.json-ban konfigurált ideig tart.
- Explicit kijelentkezéshez a jobb felső sarokban lévő fiók-ikonra kattintva megnyíló Fiok.razor legördülő menüben található Kijelentkezés gombra kell kattintani. (A Naptár, Tantárgyak, Csoportok és Feladatok oldalak sidebarjából a korábbi külön Kijelentkezés gomb eltávolításra került.)


Jó újra látni!
Jelentkezz be a fiókodba
Felhasználónév
kovacs.janos
Jelszó
Bejelentkezés
Új vagy itt? Regisztrálj most

3.3 Az alkalmazás felülete: navigáció és elrendezés

Bejelentkezés után minden oldal egységes elrendezést követ: egy bal oldali navigációs sáv és egy fő tartalom terület.

3.3.1 A navigációs sáv (sidebar)

A bal oldali sidebar az alkalmazás fő navigációs eszköze. Tartalmazza:

- A FeladatRadar logót és nevet a tetején: kattintva a Főoldalra visz.
- A navigációs linkeket középen: az aktív oldal kiemelve jelenik meg.
- A Sötét/Világos mód kapcsolót alul.
- A Kijelentkezés gombot a jobb felső sarokban lévő fiók legördülőben.

Navigációs menüpontok:

Menüpont	URL	Elérhető szerepkörök	Leírás
Naptár	/naptar	Mindenki	Havi/heti/napi órarend és feladatnézet
Fókusz Időzítő	/fokusz	Mindenki	Pomodoro időzítő
Tantárgyak	/tantargyak	Mindenki	Tantárgy felvétel és kezelés
Csoportok	/csoport	Mindenki	Csoportkezelés, szavazás, dolgozatok
Admin Panel	/admin	Csak Admin	Rendszerkezelés

3.3.2 A felső sáv (topbar)

Minden oldal tetején egy vékony felső sáv látható, amely tartalmazza:

- A hamburger menü ikont (bal oldal) ezzel nyílik ki a sidebar.
- Az aktuális oldal nevét (pl. Főoldal, Naptár, Feladatok).
- A bejelentkezett felhasználó email-jét egy kis ponttal jelölve (jobb oldal).
- A profil-legördülő gombot: a felhasználó monogramját, teljes nevét és e-mail-jét mutatja. Rákattintva egy legördülő panel nyílik meg, ahol módosítható a felhasználónév és a jelszó, illetve ki lehet jelentkezni.

ℹ️ Tanár szerepkörű felhasználók esetén a felső sáv és a sidebar kiemelése sárga/amber színezetű, az Admin szerepkörű felhasználók felülete pedig piros/bordó

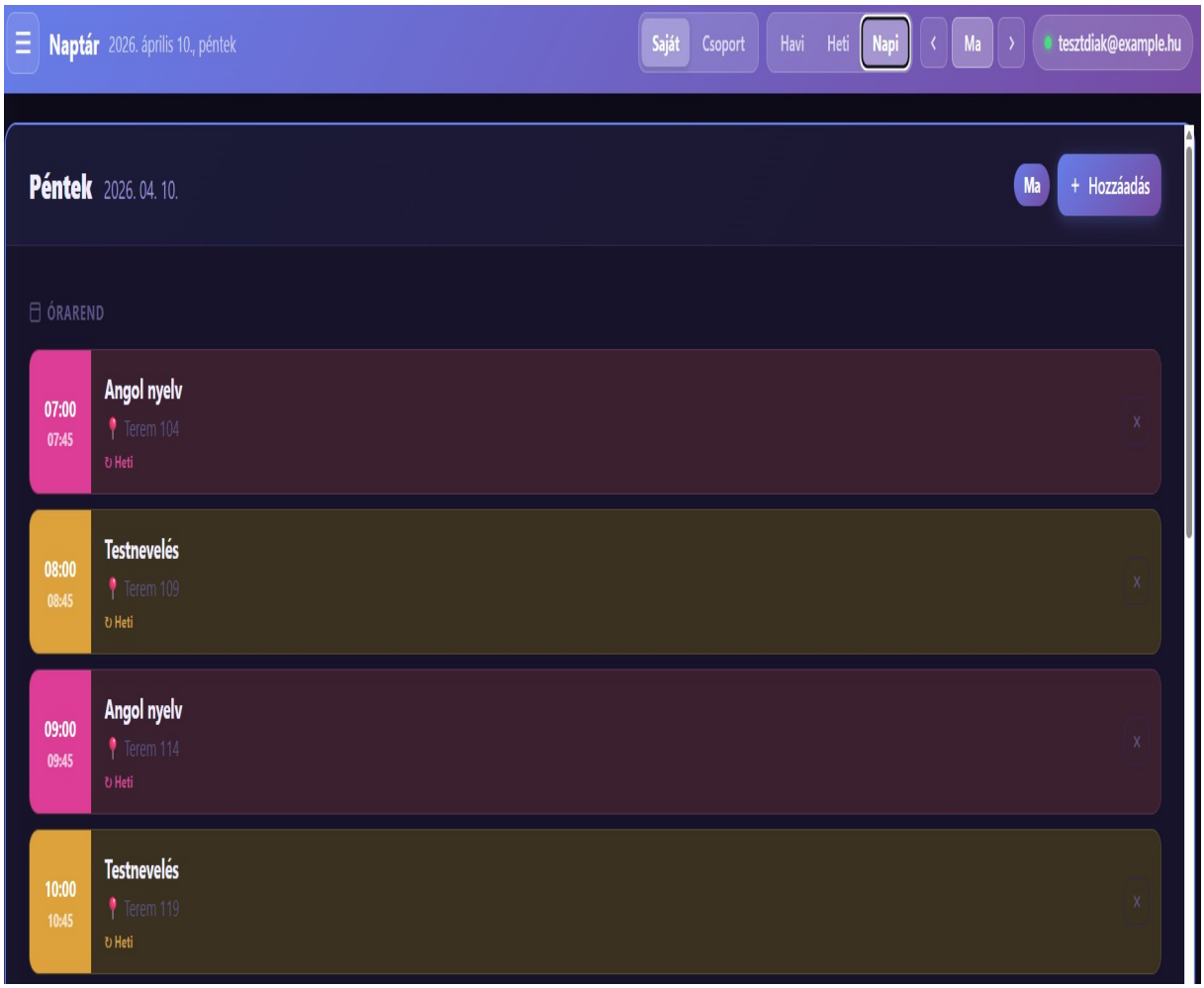
kiemelésű, mindez megkülönbözteti a három szerepkört egymástól: Diák (lila/kék), Tanár (sárga/amber), Admin (piros/bordó).

3.3.3 Sötét és világos mód

Az alkalmazás támogatja a sötét és világos megjelenési módot. A váltógomb a sidebar alsó részén található:

- Világos módban: Hold ikonra kattintva vált sötét módra.
- Sötét módban: Nap ikonra kattintva vált világos módra.

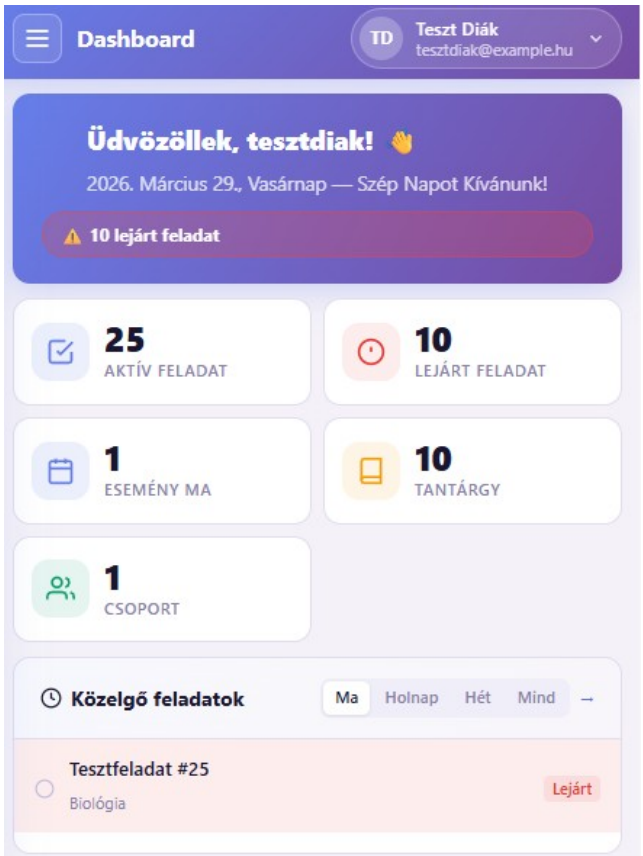
A beállítás a localStorage-ban tárolódik, így az oldal újratöltésekor és böngésző újraindítás után is megmarad. A váltás azonnal, animáció nélkül érvénybe lép.



3.3.4 Mobil nézet

Az alkalmazás reszponzív kialakítású, kisebb képernyőkön is jól használható. 768 képpont alatti képernyőszélességen:

- A sidebar alapértelmezetten rejtve marad.
- A bal felső sarokból elérhető a hamburger menü (három vízszintes vonal ikonja).
- A hamburger menüre kattintva a sidebar előcsúszik bal oldalról.
- A sidebar mögötti elsötétített területre kattintva a menü bezárul.



3.4 Főoldal

A Főoldal az alkalmazás belépési pontja bejelentkezés után. Célja, hogy egyetlen oldalon áttekinthető legyen a legfontosabb információ: hány feladat vár, mi a helyzet a határidőkkel, és gyors hozzáférést biztosítson a többi modulhoz.

3.4.1 Az üdvözlő sáv

Az oldal tetején egy szerepkörtől függő színű gradiens sáv fogadja a felhasználót: Diákok esetén lila/kék, Tanárok esetén sárga/amber, Adminok esetén piros/bordó. A sávban:

- Bal oldalon az üdvözlőszöveg jelenik meg: Üdvözöllek, [Név]! valamint az aktuális dátum és nap neve (pl. 2026. március 8., kedd).
- Jobb oldalon egy dinamikus állapotbadge látható: ha vannak lejárt feladatok: piros háttérrel a lejárt feladatok száma; ha minden feladat kész: zöld háttérrel Minden feladat teljesítve! felirat; ha vannak aktív feladatok: kék háttérrel a szám.

ℹ️ A név az email cím első része alapján kerül kiszámításra: a @ jel előtti rész első szava, nagybetűsítve. Pl. kovacs.janos@iskola.hu esetén Kovacs lesz.

3.4.2 Statisztikai kártyák

Az üdvözlő sáv alatt öt kattintható statisztikai kártya sorakozik. Minden kártyán egy nagy szám és egy felirat látható:

Kártya neve	Szín	Mit mutat	Kattintásra navigál
Aktív feladat	Lila (accent)	El nem végzett, nem lejárt feladatok száma	Feladatok oldal
Lejárt feladat	Piros (danger)	Határidőn túl lévő, el nem végzett feladatok	Feladatok oldal
Esemény ma	Lila (accent)	Aznap esedékes, el nem végzett feladatok száma	Naptár oldal
Tantárgy	Sárga (warning)	Felhasználó felvett tantárgyainak száma	Tantárgyak oldal
Csoport	Zöld (success)	Csoportok száma, ahol a felhasználó tag	Csoportok oldal

3.4.3 Közelgő feladatok panel

A statisztikai kártyák alatt a bal oldali panelben a közelgő feladatok listája látható. A panel tetején négy szűrőgomb található:

Szűrő gomb	Mit mutat
Ma	Csak a mai napra esedékes, el nem végzett feladatokat
Holnap	Csak a holnapi napra esedékes, el nem végzett feladatokat
Hét	A következő 7 napban esedékes, el nem végzett feladatokat
Mind	Az összes el nem végzett feladatot, maximum 10-et mutat

Minden feladatsorban látható a feladat neve, a tantárgy neve (ha van), és egy határidő badge: Lejárt (piros), Ma (narancssárga), Holnap (sárga), vagy a konkrét dátum (távolabbi határidő esetén).

3.4.4 Gyors navigáció panel

A közelgő feladatok paneltől jobbra a Gyors navigáció panel látható. Ez négy nagy ikonos linket tartalmaz a főbb oldalakra: Naptár, Feladatok, Tantárgyak, Csoportok. Mindegyik sor tartalmaz egy ikont, a lap nevét és egy nyíl ikont a jobb oldalon.

3.5 Naptár

A Naptár oldal a FeladatRadar leggazdagabb és legösszetettebb modulja. Lehetővé teszi az órarend és a feladatok integrált, vizuális megjelenítését három különböző nézetben.

3.5.1 A naptár fejléce

Az oldal fejlécében az alábbi elemek találhatók:

- Az oldal neve (Naptár).
- Navigációs nyilak (bal, jobb) és a navigáció gomb (dinamikus felirattal: Ez a hét / Jövő hét / Előző hét / Holnap / Tegnap stb.) az időbeli navigációhoz.
- A nézet-választó gombok: Havi, Heti, Napi.
- Két gomb van egymás mellett: Saját és Csoport.
- A felhasználó email-je jobb oldalon.

3.5.2 Nézetek

Heti nézet (alapértelmezett)

A heti nézet az alapértelmezett megjelenési mód. Hét oszlopot jelenít meg, hétfőtől vasárnapig. Minden oszlop fejlécében a nap neve és dátuma látható, az aktuális nap lila/kék kiemeléssel jelenik meg. Minden oszlopban az aznapi elemek szerepelnek: órarend-bejegyzések (tantárgytól függő dinamikus háttérszínnel, kezdő és befejező időponttal, pl. 08:00, 09:30) és feladatok (típusonként más ikon és szín).

ℹ️ Csak jövőbeli napokra lehet új bejegyzést hozzáadni. A múltbeli napokra kattintás nem nyitja meg a beviteli panelt.

Havi nézet

A havi nézet az adott hónap összes napját rácsszerűen jeleníti meg. Minden napnál kis, névvel ellátott színes sávok jelzik az aznapi eseményeket: órarend-bejegyzéseknél a tantárgy neve jelenik meg tantárgyfüggő dinamikus háttérszínnel, feladatoknál a feladat neve látható (Exam típus piros, többi lila háttérrel). A havi nézetben napra kattintás a napi nézetre vált, a bejegyzések hozzáadásához a heti nézetbe kell váltani.

Napi nézet

A napi nézet az aktuálisan kiválasztott nap összes eseményét jeleníti meg időrendben. A fejlécben a pontos dátum és nap neve olvasható. Leghasznosabb akkor, ha egy adott napnak sok eseménye van.

3.5.3 Navigáció a naptárban

Gomb/elem	Heti nézetben	Havi nézetben	Napi nézetben
Bal nyíl (<)	Előző hét	Előző hónap	Előző nap
Jobb nyíl (>)	Következő hét	Következő hónap	Következő nap
Navigáció gomb (dinamikus felirat: Ez a hét / Jövő hét / Előző hét stb.)	Aktuális hét (visszanavigál az aktuális hétre)	Aktuális hónap (visszanavigál az aktuális hónapra)	Mai nap (visszanavigál a mai napra)
Nézet gombok	Váltás havi/napi nézetre	Váltás heti/napi nézetre	Váltás heti/havi nézetre

3.5.4 Új órarend-bejegyzés hozzáadása

Heti nézetben egy jövőbeli napra kattintva a jobb oldalon megjelenik az Új bejegyzés panel. A panel két fület tartalmaz:

Órarend fül mezői:

Mező neve	Kötelező?	Leírás	Példa
Tantárgy	Igen	Legördülő lista a felvett tantárgyakból	Matematika alapjai
Kezdési időpont	Igen	HH:mm formátum, alapértelmezett: 08:00	09:00
Befejezési időpont	Igen	HH:mm formátum, alapértelmezett: 09:30	10:30
Helyszín / terem	Nem	Szabad szöveges mező	101-es terem
Ismétlődés	Nem	Egyszeri, Heti, Kéthetente, Havonta	Heti

Lehetséges hibaüzenetek bejegyzés hozzáadásakor:

Hibaüzenet	Ok	Megoldás
Időpontütközés: már van bejegyzésed ebben az időpontban.	Az időszakban már van bejegyzés.	Válasszon más időpontot, vagy törölje az ütközőt.
Tanári csoporthoz csak a csoport tulajdonosa adhat hozzá bejegyzést.	Tanári csoportba diákként próbál hozzáadni.	Tanári csoportban csak a tulajdonos tanár adhat hozzá.

ℹ️ A fülváltás nem törli a már beírt adatokat: ha az Órarend fülön már kitöltötte a tantárgyat, majd átváltja a Feladat fülre, az adatok megmaradnak, ha visszavált.

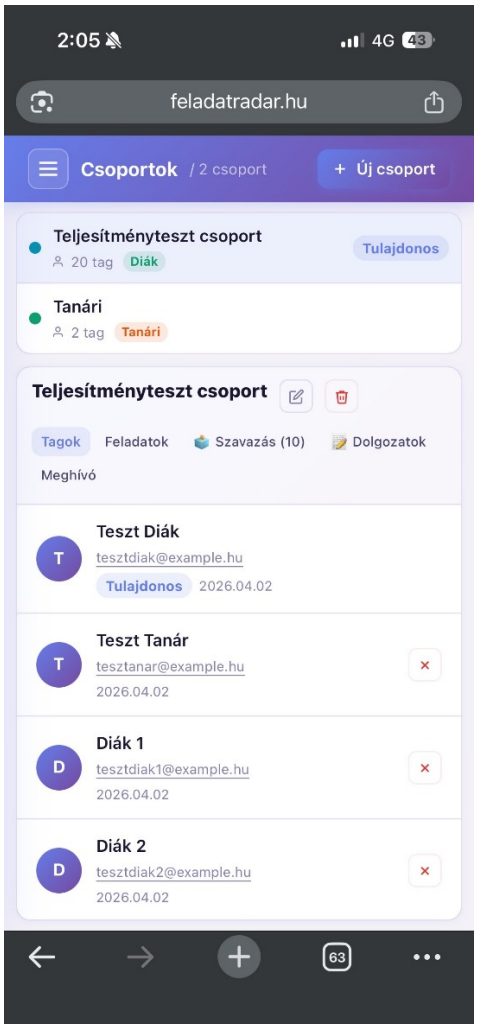
3.5.5 Bejegyzés törlése

Az órarend-bejegyzések és csoportos feladatok törlése közvetlenül a Naptárból is lehetséges, a bejegyzésre/feladatra húzva az egeret megjelenik egy törlés gomb (X ikon). Csoport módban törlési jogosultsággal rendelkezik a bejegyzést/feladatot hozzáadó tag, illetve a csoport tulajdonosa. A naptár jelenleg megjelenítési, hozzáadási és törlési funkcióval rendelkezik, szerkesztési funkcióval nem.

3.5.6 Csoport mód

A fejlécben lévő Csoport mód kapcsolóval aktiválható a csoportos nézet. Amikor be van kapcsolva, a naptár a kiválasztott csoport közös adatait jeleníti meg a személyes adatok helyett. A fejlécben megjelenik egy legördülő lista a csoportjaiból. Csoport módban heti nézetben napra kattintva az Új bejegyzés panel mindkét füle elérhető: az Órarend fülön csoport-órarend bejegyzés, a Feladat / Esemény fülön csoportos feladat adható hozzá.

⚠️ Tanári csoportban csoport módban csak a tulajdonos tanár adhat hozzá bejegyzést. Ha nem Ön a tulajdonos, a beviteli panel nem nyílik meg.



3.6 Feladatok

A Feladatok oldal a személyes teendők kezelésének központi helye. Minden feladat táblázatos formában jelenik meg, és szűrhető, kereshető, rendezhető.

3.6.1 A feladatlista

Oszlop neve	Leírás
Ikon	A feladat típusához tartozó emoji ikon
Feladat neve	A feladat teljes neve
Tantárgy	A feladathoz rendelt tantárgy neve (ha van)
Határidő	A feladat határideje (dátum és idő)
Hátralévő idő	Dinamikus szöveg: pl. 3 nap múlva, Ma jár le!, 2 napja lejárt
Típus	A feladat típusa olvasható formában (pl. Házi feladat, Dolgozat)
Műveletek	Kész/Visszavon gomb és Törlés gomb

Sorok háttérszínezése:

Háttérszín	Állapot	Feltétel
Piros háttér	Lejárt feladat	IsOverdue = true (határidő elmúlt, még nem kész)
Narancs/sárga háttér	Hamarosan esedékes	IsDueSoon = true (3 napon belül jár le)
Halvány zöld/áthúzott	Teljesített feladat	IsCompleted = true
Fehér/surface háttér	Normál állapot	Aktív, nem sürgős feladat

3.6.2 Szűrés

Az oldal tetején egy szűrősáv látható, ahol típus és állapot szerint lehet szűkíteni a listát. A szűrő azonnal érvénybe lép.

Állapot szűrők:

Szűrőgomb	Megjelenített feladatok
Összes	Minden feladat, szűrés nélkül
Aktív	El nem végzett, nem lejárt feladatok
Hamarosan	3 napon belül esedékes, el nem végzett feladatok (IsDueSoon)
Lejárt	Határidőn túl lévő, el nem végzett feladatok (IsOverdue)
Kész	Csak a teljesített feladatok (IsCompleted = true)

Típus szerinti szűrők:

Típus kulcs	Magyar megnevezés	Leírás
Homework	Házi feladat	Iskolai házi feladat
Exam	Dolgozat	Dolgozat vagy vizsga felkészülés
Assignment	Beadandó	Beadandó feladat, projekt
Project	Projekt	Hosszabb projektmunka
Verseny	Verseny	Versenyfelkészülés, versenyek
Sport	Sport	Sporttevékenység, edzés
Olvasas	Olvasás	Olvasnivaló, kötelező olvasmány
Personal	Személyes	Személyes teendő
Other	Egyéb	Bármilyen egyéb feladat

3.6.3 Keresés

A szűrők alatt található a keresőmező. A keresés valós idejű, gépelés közben azonnal frissül a lista. A keresés a feladat nevében a tantárgy nevében és a leírásban keres egyszerre, kis- és nagybetű érzéketlenül.

3.6.4 Rendezés

Rendezési szempont	Leírás
Határidő (alapértelmezett)	Elvégzendők előre, majd határidő szerint növekvő sorrendben
Típus	Típus szerint ábécésorrendben, azonos típuson belül határidő szerint
Tantárgy	Tantárgynév szerint ábécésorrendben, majd határidő szerint
Név	Feladat neve szerint ábécésorrendben

3.6.5 Új feladat hozzáadása

Az oldal jobb felső sarkában az + Új feladat gombra kattintva egy modális ablak nyílik meg:

Mező neve	Kötelező?	Típus	Leírás
Feladat neve	Igen	Szöveg	Min. 1 karakter
Tantárgy	Nem	Legördülő lista	A felvett tantárgyak közül választható.
Feladat típusa	Igen	Legördülő lista	Ld. fenti típuslista. Alapértelmezett: Beadandó.
Határidő	Igen	Dátum + időválasztó	Alapértelmezett: a mai naptól számított 7. nap.
Leírás	Nem	Szövegmező	Részletes leírás, emlékeztető.
Ismétlődés típusa	Nem	Legördülő lista	Egyszeri, Heti, Kéthetente, Havonta.

ℹ️ Az ismétlődő feladatok mind külön sorok lesznek a listában. Mindegyiket egyenként lehet teljesítettnek jelölni vagy törölni. A törlés csak az adott feladatot törli, a sorozat többi tagját nem.

3.6.6 Feladat teljesítése és visszaállítása

Minden feladatsor végén egy Kész gomb található. Erre kattintva a feladat teljesítettnek jelölődik, a sor halvány áthúzott stílusra vált, és a gomb felirata Visszavon-ra változik. A Visszavon gombra kattintva a feladat visszaáll elvégzendő állapotba.

3.6.7 Feladat törlése

A feladatsor végén, a Kész gombtól jobbra található a piros Törlés gomb (kuka ikon). Kattintás után a feladat azonnal törlődik, nincs megerősítő párbeszédablak. A törlés visszavonhatatlan.

💡 Ha véletlenül törölt egy feladatot, nincs visszavonási lehetőség. Érdemes emiatt a fontosabb feladatokat inkább csak teljesítettnek jelölni, nem törölni.

3.7 Fókusz Időzítő: Pomodoro

A Fókusz Időzítő oldal egy Pomodoro-technikán alapuló tanulást segítő eszköz. A Pomodoro-technika lényege: 25 perces koncentrált munkaperiódusok, amelyeket 5 perces szünetek választanak el. Négy Pomodoro után hosszabb, 15 perces szünet következik.

Fókusz Időzítő
/ 0 befejezett munkamenet
Fókusz
Rövid szünet
Hosszú szünet
Típus:
Tanulás
Verseny
Olvasás
Projekt
Edzés
Egyéb
FÓKUSZ
25:00
Kész az indulásra

3.7.1 Az időzítő főablaka

Az oldal közepén egy nagy körös vizuális visszajelző látható (SVG animáció), amelynek szegélye fokozatosan csökken az idő múlásával. A kör közepén a hátralévő idő percben és másodpercben jelenik meg (pl. 24:33).

3.7.2 Módok

Mód neve	Alapértelmezett idő	Leírás	Mikor aktiválódik
Fókusz	25 perc	Koncentrált tanulási/munkaidőszak	Manuálisan, vagy AutoSwitch esetén szünet után
Rövid szünet	5 perc	Rövid pihenő fókusz után	Manuálisan, vagy AutoSwitch esetén fókusz után
Hosszú szünet	15 perc	Hosszabb pihenő	Csak manuálisan, a gombra kattintással

3.7.3 Az időzítő vezérlése

Gomb neve	Funkció
Indít	Elindítja a visszaszámlálást. A gomb felirata Szünet-re változik.
Szünet	Megállítja az időzítőt. A hátralévő idő megmarad. A gomb felirata Indít-ra változik.
Újraindít	A hátralévő időt visszaállítja az aktuális mód teljes idejére.

3.7.4 Automatikus váltás (AutoSwitch)

A Beállítások panelen található az Automatikus váltás kapcsoló. Ha be van kapcsolva, a fókusz időszak végén automatikusan elindul a Rövid szünet, és fordítva. Ha ki van kapcsolva, az időszak végén az időzítő megáll és a felhasználónak kézzel kell elindítani a következőt.

ℹ️ Az AutoSwitch a Hosszú szünet módra nem vonatkozik: azt mindig manuálisan kell aktiválni.

3.7.5 Beállítások panel

Beállítás neve	Alapértelmezett	Leírás
Fókusz ideje (perc)	25	A fókusz időszak hossza percben (1-120 között)
Rövid szünet ideje (perc)	5	A rövid szünet hossza percben (1-60 között)
Hosszú szünet ideje (perc)	15	A hosszú szünet hossza percben (1-60 között)
Automatikus váltás	Ki	Ha be van kapcsolva, automatikusan vált fókusz és szünet között

A beállítások a localStorage-ban tárolódnak, így böngésző bezárása és újbóli megnyitása után is megmaradnak.

3.7.6 Munkamenet-napló

Az oldal alján a SessionLog lista mutatja az aktuális alkalmazás-munkamenet során elvégzett fókusz-szakaszokat. Minden bejegyzés tartalmazza a befejezett mód nevét, az időszak hosszát percben, és a befejezés pontos időpontját.

⚠️ A munkamenet-napló csak a jelenlegi böngészőmunkamenet alatt tartja meg az adatokat. Ha bezárja a böngészőt vagy frissíti az oldalt, a napló törlődik.

3.8 Tantárgyak

A Tantárgyak oldal kezeli a felhasználó felvett tantárgyait. A tantárgyak fontosak, mert az órarend-bejegyzések és feladatok tantárgyhoz köthetők, ez segíti a rendszerezést és a szűrést.

3.8.1 A tantárgyak megjelenítése

A felvett tantárgyak kártya nézetben jelennek meg. Minden kártyán látható a tantárgy neve, tantárgykód, felvétel dátuma, aktív és kész feladatok száma, heti óraszám (ha van), és egy Leadás gomb. Minden kártya egyedi szín-árnyalatot kap a tantárgy azonosítójától függően.

3.8.2 Tantárgy felvétele listából

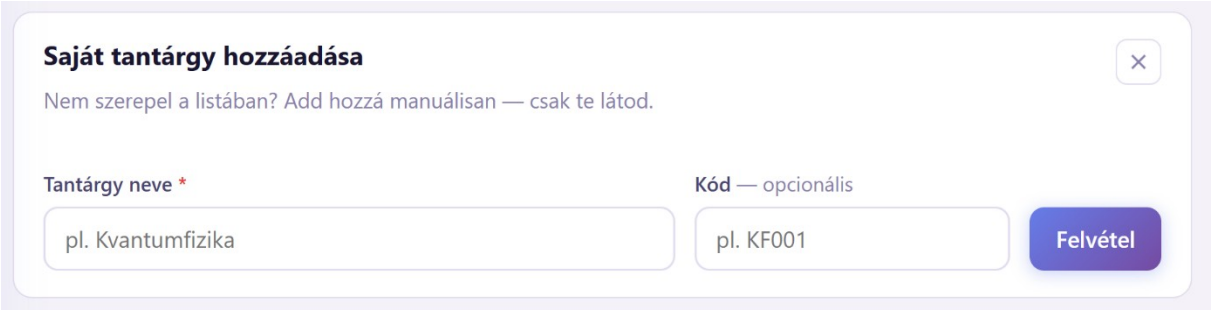
A **+** Tantárgy felvétele gomb a publikusan elérhető tantárgyak listáját jeleníti meg (név, kód, tanár neve). A Felvétel gombra kattintva a tantárgy felkerül a felhasználó listájára.

Hibaüzenet	Ok
Ezt a tantárgyat már felvetted.	Duplán próbálja felvenni ugyanazt a tantárgyat.

3.8.3 Egyedi (privát) tantárgy hozzáadása

Ha a tantárgya nem szerepel a publikus listában (pl. különórák, edzésterv, szabadidős tevékenység), az Egyedi tantárgy gombra kattintva létrehozhatja. A mezők: Tantárgy neve (kötelező) és Tantárgykód (opcionális). Az egyedi tantárgy IsPrivate = 1 értékkel jön létre, csak Ön látja, és nincs létszámkorlátja.

Az egyedi tantárgyak kiválóan alkalmasak személyes tevékenységek nyilvántartásához: sportfoglalkozások, hangszertanulás, hobbik.



3.8.4 Tantárgy leadása

A tantárgy kártyáján a Leadás gombra kattintva a felhasználó leadja a tantárgyat. A kapcsolat törlődik a StudentSubjects táblából, és a CurrentEnrollment számlálója csökken. A tantárgyhoz kötött korábbi feladatok és órarend-bejegyzések megmaradnak, de elveszítik a tantárgy-hivatkozást.

⚠️ A leadás visszavonható. Ha leadja a tantárgyat, majd újra fel szeretné venni, a Tantárgy felvétele listából megteheti.

3.9 Csoportok

A Csoportok oldal az együttműködési funkciók központja. Lehetővé teszi, hogy diákok és tanárok csoportokat hozzanak létre, meghívjanak másokat, közös órarendet és feladatlistát vezessenek, szavazásokat indítsanak, és dolgozatokat hirdessenek meg.

3.9.1 Az oldal felépítése

- Felső/listás panel: a csoportok listája és a beérkező meghívások.
- Jobb oldali panel: a kiválasztott csoport részletei (öt fülön szervezve).

3.9.2 Csoport létrehozása

Az + Új csoport gombra kattintva a csoport nevét kell megadni (min. 1, max. 100 karakter). A Létrehoz gombra kattintva a csoport létrejön, és a létrehozó automatikusan taggá válik.

A csoport típusát a tulajdonos szerepköre határozza meg: ha a létrehozó Diák → Diák csoport (minden tag adhat hozzá bejegyzést); ha Tanár → Tanár csoport (csak a tulajdonos tanár adhat hozzá órarend-bejegyzést).

Csoport átnevezése: A tulajdonos a csoport neve melletti ceruza ikonra kattintva átnevezheti a csoportot. Az átnevezés megerősítéséhez az Enter billentyűt kell lenyomni vagy a Mentés gombra kattintani. Az Escape billentyűvel vagy a Mégse gombbal a módosítás elvethető.

Csoport törlése: A tulajdonos a kuka ikonra kattintva törölheti a csoportot. Törlés előtt egy megerősítő párbeszédablak jelenik meg, amelyben jóvá kell hagyni a műveletet. **Figyelem: a törlés visszavonhatatlan**, minden kapcsolódó adat (tagok, feladatok, szavazások, dolgozatok) véglegesen elvész.

ℹ️ A csoport tulajdonosa nem tud kilépni a csoportból. Ha meg szeretné szüntetni a csoportot, azt maga is törölheti illetve az Admin is elvégezheti.

3.9.3 Tagok meghívása

A csoport kiválasztása után a Meghívó fülön (csak a csoport tulajdonosa látja) meg lehet hívni új tagokat az email cím megadásával.

A meghívás folyamata:

- A meghívó fél megadja a meghívandó email-jét és elküldi a meghívást.
- A meghívott felhasználó a saját Csoportok oldalán látja a beérkező meghívást.
- A meghívott dönthet: Elfogad vagy Elutasít.
- Elfogadás esetén a felhasználó automatikusan taggá válik.
- Elutasítás esetén a meghívás törlődik.

Hibaüzenet	Ok
Nem létezik ilyen email-cimű felhasználó.	Az email nem szerepel a regisztrált felhasználók között.
Ez a felhasználó már tagja a csoportnak.	A meghívandó személy már tag.
Ez a felhasználó már meg van hívva.	Korábban már elküldték a meghívást, de még nem fogadta el.

3.9.4 A csoport fülei

Tagok fül

A csoport összes tagjának listája (névvel, email-lel, csatlakozás dátumával). A tulajdonosnak korona ikon jelzi az IsOwner = true státuszt. A csoport tulajdonosa az egyes tagok sorában megjelenő ✕ gombra kattintva eltávolíthat tagokat (a saját magát kivéve). A törlés előtt egy megerősítő párbeszédablak jelenik meg.

Feladatok fül

A csoport közös feladatlistája: feladat neve, típusa, tantárgy neve, határidő, teljesítési státusz és a hozzáadó neve.A fül tetején — csoport-kezelési joggal rendelkező felhasználóknak (tanár: tulajdonos; diák: bármely tag) megjelenik egy feladat-hozzáadó form is: tantárgy-legördülő, cím, határidő és opcionális leírás mező, amelyen keresztül az sp_AddGroupTask tárolt eljárás hívódik.

Szavazás fül

A csoport aktív szavazásai. Szavazat leadása: az opcióra kattintva azonnal rögzítésre kerül. Szavazat módosítása: egy másik opcióra kattintva az előző szavazat automatikusan frissül.

Új szavazás létrehozásához (tanároknak és tulajdonosoknak elérhető) a következő mezőket kell kitölteni:

Mező neve	Kötelező?	Leírás
Kérdés	Igen	A szavazás kérdése, max. 500 karakter
Opciók	Igen	Soronként egy szavazási lehetőség. Alapértelmezett: Igen, Nem, Talán
Lejárat dátuma	Nem	Meddig legyen aktív a szavazás

Dolgozatok fül

A tanár által a csoportnak meghirdetett közelgő dolgozatok, vizsgák, számonkérések listája. Tanár szerepkörű felhasználók az + Új dolgozat gombra kattintva új számonkérést hirdethetnek:

Mező neve	Kötelező?	Leírás
Cím	Igen	A dolgozat neve, pl. Matematikavizsga: másodfokú egyenletek
Tantárgy neve	Nem	Szabad szöveges mező, pl. Matematika
Dátum és idő	Igen	A dolgozat pontos időpontja
Leírás / megjegyzés	Nem	Részletek, pl. melyik fejezetek szerepelnek

ℹ️ A Dolgozatok fül tartalma megjelenik a Naptár oldalon is, ha a csoport mód aktív.

Meghívó fül

A Meghívó fül kizárólag a csoport tulajdonosának látható. Ezen a fülön lehet meghívni új tagokat email cím alapján (ld. 3.9.3 fejezet).

3.9.5 Kilépés a csoportból

A csoport lista sorában minden csoportnál megjelenik egy Kilépés gomb. Rákattintva a felhasználó elhagyja a csoportot.

⚠️ A csoport tulajdonosa nem léphet ki a csoportból. Ha a tulajdonos el szeretné hagyni, előbb a tulajdonjogát át kellene ruházni, ez jelenleg nem lehetséges a felületen. A csoportot a tulajdonos maga is törölheti, illetve az Admin is elvégezheti a törlést.

3.10 Admin felület

Az Admin felület kizárólag Admin szerepkörű felhasználók számára érhető el. Ha egy nem-admin felhasználó megpróbálja a /admin URL-t közvetlenül megnyitni, az oldal Nincs jogosultságod üzenettel visszairányítja.

3.10.1 Az Admin felület fülei

Az Admin oldal négy fülre tagolódik: Áttekintés, Felhasználók, Csoportok, Napló (Audit Log).

3.10.2 Áttekintés fül

Kártya neve	Mit mutat
Összes felhasználó	A Users táblában lévő összes sor száma
Diák	Student szerepkörű felhasználók száma
Tanár	Teacher szerepkörű felhasználók száma
Admin	Admin szerepkörű felhasználók száma
Csoport	A Groups táblában lévő csoportok száma
Tantárgy	A Subjects táblában lévő tantárgyak száma

A kártyák alatt részletes statisztikák láthatók: összes feladat, aktív feladatok, lejárt feladatok, szavazások, dolgozatok, valamint a mai és elmúlt 7 nap regisztrációinak száma.

3.10.3 Felhasználók fül

Az összes regisztrált felhasználó táblázatban látható. A táblázat felett keresőmező (felhasználónév és e-mail cím alapján keres, névben nem keres) és szerepkör-szűrők (Mind, Diákok, Tanárok, Adminok) találhatók.

Oszlop neve	Leírás
ID	A felhasználó egyedi UserID-ja az adatbázisban
Név	Keresztnév és Vezetéknév
Email	Regisztrált email cím
Username	Bejelentkezési felhasználónév
Szerepkör	Student, Teacher vagy Admin: színkódolt badge
Státusz	Aktív (zöld) vagy Letiltott (piros): az IsActive mező alapján
Regisztráció	A fiók létrehozásának dátuma
Műveletek	Név szerkesztése (ceruza ikon), Szerepkör módosítása, Letiltás/Aktiválás, Törlés

⚠️ Saját szerepkörét az admin nem tudja módosítani, az alkalmazás hibaüzenettel visszautasítja. Más Admin szerepkörű felhasználókat sem lehet törölni, letiltani vagy szerepkörüket módosítani, a műveleti gombok helyett „Védett" felirat jelenik meg. A felhasználó nevét az admin a ceruza ikonra kattintva tudja szerkeszteni.

Felhasználó letiltása: az IsActive értéke 0-ra változik, a felhasználó nem tud bejelentkezni, de adatai megmaradnak. A tiltás feloldható az Aktiválás gombbal. Törléskor, ha FK hivatkozások akadályozzák a fizikai törlést, az alkalmazás automatikusan soft-delete-re vált.

3.10.4 Csoportok fül

Az összes létező csoport listája (név, azonosító, tulajdonos neve és szerepköre, tagok száma, létrehozás dátuma). Minden csoportnál megjelenik egy Törlés gomb, amely kaszkád módon törli az összes kapcsolódó adatot (szavazatok, opciók, szavazások, csoport feladatok, csoport órarend, meghívók, tagság). A csoport sorára kattintva megjelenik egy részlet-panel, amely listázza a csoport összes dolgozatát.Az admin itt egyenként is törölheti a dolgozatokat, szavazásokat és feladatokat a moderációs törlés végpontokon keresztül.

⚠️ A csoport törlése visszavonhatatlan. Az összes kapcsolódó adat — tagok, feladatok, szavazások, dolgozatok véglegesen elvész.

3.10.5 Napló fül (Audit Log)

Az Audit Napló fülön az utolsó 100 admin művelet kronológikus listája látható (legújabbtól a legrégebbi felé rendezve). Minden bejegyzésnél megjelenik: a LogID, a műveletet végrehajtó admin neve és UserID-ja, a művelet típusa a részletek, és a pontos időpont. A napló csak olvasható.

3.11 Hibaelhárítás: Gyakori problémák és megoldások

3.11.1 Bejelentkezési problémák

Probléma	Lehetséges ok	Megoldás
Hibás felhasználónév vagy jelszó üzenet, bár biztos a jelszó.	A jelszó plain-text-ként lett mentve az adatbázisba.	Törölje és hozza létre újra a felhasználót az sp_RegisterUser tárolt eljáráson keresztül.
Az oldal nem tölt be, fehér képernyő.	A backend szerver nem fut.	Ellenőrizze, hogy mindkét projekt fut-e.
Reconnecting felirat jelenik meg.	SignalR kapcsolat megszakadt.	Frissítse az oldalt (F5 vagy Ctrl+R).
A token lejárt, az oldal kijelentkeztet.	A JWT token érvényessége lejárt.	Jelentkezzen be újra.

3.11.2 Naptár problémák

Probléma	Lehetséges ok	Megoldás
Nem jelenik meg az új bejegyzés panel napra kattintáskor.	Múltbeli napra kattintott, vagy nincs szerkesztési joga.	Csak jövőbeli napokra nyílik meg a panel. Tanári csoportban csak a tulajdonos tud hozzáadni.
Időpontütközés hiba, de nincs látható esemény.	Az ütköző bejegyzés más héten van, de ugyanarra az időpontra van beállítva.	Ellenőrizze az összes hetet és törölje az ütköző bejegyzést.
A csoport mód kapcsoló után nem jelennek meg csoportos adatok.	Nincs kiválasztva csoport, vagy nincs bejegyzése.	Ellenőrizze a fejlécben a csoport legördülő listát.

3.11.3 Feladatok problémák

Probléma	Lehetséges ok	Megoldás
Az új feladat mentése után nem jelenik meg a listában.	A szűrő kizárja az új feladatot.	Állítsa a szűrőt Összes értékre.
A tantárgy legördülő üres az új feladat formban.	Nincsenek felvett tantárgyak.	Előbb a Tantárgyak oldalon vegyen fel legalább egy tantárgyat.
Az ismétlődő feladat nem jön létre.	Nincs megadva végdátum.	Jelölje be a Végdátumig kapcsolót és adja meg a végdátumot.

3.11.4 Csoport problémák

Probléma	Lehetséges ok	Megoldás
A meghívás gomb nem látható.	Nem Ön a csoport tulajdonosa.	Csak a tulajdonos hívhat meg tagokat.
Elfogadtam a meghívást, de nem jelenik meg a csoport.	Az oldal nem frissítette magát.	Frissítse az oldalt (F5) vagy navigáljon el és vissza.
A szavazás opcióira kattintva nem változik semmi.	A szavazás lejárt vagy nincs internet kapcsolat.	Ellenőrizze a szavazás lejárati dátumát és az internetkapcsolatot.
A törlés/átnevezés gomb nem látható.	Ön nem a csoport tulajdonosa.	Ezek a gombok csak a csoport tulajdonosának láthatók.

3.11.5 Általános problémák

Probléma	Lehetséges ok	Megoldás
Az alkalmazás lassan tölt be.	Sok adatot kell betölteni, vagy lassú a szerver.	Várjon egy percet. Ha ismétlődik, ellenőrizze a szerver erőforrásait.
A sötét mód beállítás nem marad meg.	A böngésző letiltotta a localStorage-t (inkognitó mód).	Normál böngészőben (nem inkognitóban) használja az alkalmazást.
Hibás szám jelenik meg a Főoldal kártyákon.	Az adatok nem frissültek.	Navigáljon el a Főoldalról, majd vissza.
Az Admin menü nem látható a sidebarban.	A felhasználó nem Admin szerepkörű.	Admin menüt csak Admin lát. Kérjen Admin jogosultságot a rendszergazdától.

3.12 Tippek és trükkök a hatékony használathoz

3.12.1 Tippek diákoknak

- Rendszeresen nyissa meg a Főoldalt: az üdvözlő sáv azonnal megmutatja, hány feladat vár Önre és hány késett le.
- Használja a szűrőket a Feladatok oldalon: a Hamarosan szűrő megmutatja a 3 napon belül esedékes teendőket.
- Az egyedi tantárgy funkciót bátran használja: sportedzések, hobbik, különórák mind felvehetők privát tantárgyként.
- A Pomodoro időzítőt tartsa megnyitva tanulás közben: az AutoSwitch bekapcsolásával automatikusan váltakoznak a fókusz és szünet periódusok.
- A csoport módot a Naptárban érdemes bekapcsolni az első napon, hogy lássa a csoport közös dolgozatait és feladatait.
- Ha elfelejtett valamit beírni, a Naptár Új bejegyzés paneljén a Feladat fülön közvetlenül a megfelelő naphoz adhatja hozzá.

3.12.2 Tippek tanároknak

- A Tanár szerepkör sárga/amber színsémát használ: ez segít megkülönböztetni a tanári fiókot a diákfióktól, ha egyszerre több böngészőfülön dolgozik.
- A dolgozatokat mindig legalább egy héttel korábban hirdesse meg a csoportban, hogy a tagoknak legyen idejük felkészülni.
- A szavazás funkcióval demokratikus módon egyeztetheti a csoport tagjait: pl. melyik időpontra essen a ZH.
- A tanári csoportban csak Ön adhat hozzá órarend-bejegyzést, ezzel garantálhatod, hogy a közös órarend mindig pontos.
- A meghíváshoz szükséges az e-mail cím. Kérje el a tagok regisztrált email-jét előre.

3.12.3 Tippek adminoknak

- Az audit naplót rendszeresen ellenőrizze: a legutóbbi 100 bejegyzés az összes admin tevékenységet rögzíti.
- Felhasználó törlésekor előnyben részesítse a letiltást a fizikai törlés helyett, ha a felhasználónak vannak adatai.
- Ha szerepkört módosít, az érintett felhasználónak ki kell jelentkeznie és újra be kell lépnie, hogy az új szerepkör érvényre jusson.
- A rendszerstatisztikák az Áttekintés fülön valós idejűek, minden alkalommal betöltődnek, amikor az oldalra navigál.

A FeladatRadar felhasználói dokumentációja ezen a ponton véget ér. Ha további kérdések merülnek fel, a fejlesztői dokumentáció (2. fejezet) részletes technikai információkat tartalmaz az alkalmazás működéséről.

3.13 Kapcsolat és elérhetőség

Amennyiben az alkalmazás használatával kapcsolatban kérdése, észrevétele vagy hibajelentése van, az alábbi elérhetőségeken léphet kapcsolatba a fejlesztőcsapattal:

E-mail: FeladatRadar@gmail.com

Összefoglalás és köszönetnyilvánítás

4.1 A projekt értékelése

A FeladatRadar elkészítése komoly, több hónapos munka eredménye, amelynek során nemcsak programozói tudásunkat fejlesztettük, hanem megtanultunk csapatban dolgozni, feladatokat elosztani és verziókezelő rendszerrel (Git/GitHub) együttesen fejleszteni.

A bevezetésben kitűzött szakmai célokat, egy komplex, valóban hasznos iskolai feladatkezelő webalkalmazás elkészítése ASP.NET Core és Blazor Server technológiával, lényegében megvalósítottuk. Az alkalmazás rendelkezik az összes tervezett funkcióval: egyéni feladatkezelés, órarend, fókusz időzítő, tantárgy-nyilvántartás, csoportkezelés, szavazás és tanári modul. A kódminőség tekintetében igyekeztük az iparági ajánlásokat követni: service-réteg elkülönítése, interface-alapú programozás, tárolt eljárások kizárólagos használata adatbázis-hozzáféréshez.

4.2 A program jövőbeli hasznosulása

A FeladatRadar nemcsak vizsgamunkának, hanem hosszú távon is életképes alkalmazásnak készült. Közvetlen következő lépésként megfontolandó az alkalmazás nyilvános szerveren való üzemeltetése, amely lehetővé tenné, hogy valódi diákok és tanárok is használják, nem csupán fejlesztői környezetben. Mivel a backend egy önálló RESTful API-ként üzemel, arra a jövőben .NET MAUI vagy React Native alapú mobilalkalmazás is építhető lenne, amellyel az értesítések és a határidők mobileszközön is elérhetővé válnának. Iskolai szintű bevezetés esetén az adminisztrátori modul már most képes több felhasználó és szerepkör kezelésére, így az intézményi igényekhez való igazítás minimális fejlesztési ráfordítást igényelne. Bízunk benne, hogy a FeladatRadar az iskolai kereteken túl is valódi eszközévé válhat azoknak, akik csapatban szeretnék szervezni közös feladataikat és határidőiket.

4.3 A tanulságok

A fejlesztés során a legnagyobb kihívást a JWT-token tárolása és az autentikált állapot kezelése jelentette Blazor Server és a saját API között, ez eltér a szokásos Blazor WebAssembly vagy MVC megközelítéstől, és komoly hibakeresést igényelt. Szintén tanulságos volt a Blazor Server és a Dapper együttes használata: a Blazor aszinkron renderelési modellje néhány esetben váratlan sorrendben hajtotta végre a komponens inicializálási metódusait, amit explicit StateHasChanged() hívásokkal kellett kezelni.

A legtöbbet az ismétlődő feladatok logikájának tárolt eljárásban való megvalósításából tanultuk, ez kényszerített minket arra, hogy alaposan átgondoljuk az adatmodell tervezését és a szülő-gyermek kapcsolatokat az ismétlődő sorozatoknál.

4.4 Köszönetnyilvánítás

Ezúton szeretnénk kifejezni őszinte hálánkat és mély köszönetünket Szalainé Török Edit, Méhes József és Csukárdi Rajmund tanárainknak, akik szakmai tudásukkal, kitartó támogatásukkal és folyamatos bátorításukkal végig segítették vizsgaremekünk elkészítését. E projekt megvalósítása hosszú hónapokat vett igénybe, és nem egyszer szembesültünk olyan kihívásokkal, amelyek megoldásához elengedhetetlen volt az ő iránymutatásuk és biztatásuk. Munkájuk nélkül ez a webalkalmazás nem jöhetett volna létre abban a formában, amelyre végül büszkék lehetünk.

Szalainé Török Edit tanárnő türelme, alapos útmutatásai és értékes szakmai tanácsai meghatározó szerepet játszottak a fejlesztési folyamat során. Köszönjük, hogy minden kérdésünkre készségesen és kimerítően válaszolt, hogy a nehezebb technikai és dokumentációs feladatoknál is mindig megtalálta a módját, hogy a helyes irányba tereljen minket. Visszajelzései nemcsak a projekt minőségét emelték, hanem szemléletmódunkat is formálták: megtanultuk, hogyan gondolkozzunk pontosabban, hogyan dokumentáljunk igényesebben, és hogyan álljunk hozzá felelősséggel egy összetett fejlesztői feladathoz. Hálásak vagyunk a befektetett idejéért és energiájáért, amellyel mindvégig kísérte munkánkat.

Méhes József tanár úr szakmai felkészültsége és inspiráló, lelkes hozzáállása hatalmas motivációt és segítséget jelentett számunkra a projekt teljes időtartama alatt. Köszönjük a rendszeres és őszinte visszajelzéseket, amelyek mindig pontosan rámutattak arra, min kell még dolgoznunk, miközben az elért eredményeink értékét is mindig éreztette velünk. Az általa közvetített tudás, legyen szó szoftverfejlesztési elvekről, adatbázis-tervezésről vagy a csapatmunka fontosságáról, messziről meghaladja azt, amit egyedül könyvekből és online forrásokból szerezhettünk volna. Csukárdi Rajmund Tanár úrnak köszönhetjük azt, hogy megismerkedhettünk a reszponzív webdesignnal és megtanultuk, hogyan kell úgy tervezni egy felületet, hogy az különböző képernyőméreteken, telefonon, tableten és asztali gépen egyaránt kellemes és használható maradjon. Ez a tudás a FeladatRadar fejlesztése során is napi szinten hasznosult, és olyan szemléletet adott, amelyet a jövőbeli projektjeinkben is magabiztosan alkalmazni tudunk majd.

A Tanár úr megismertette velünk a Git verziókezelő rendszert és a GitHub platformot, amelyek használata eleinte lassabbnak tűnt a közvetlen fájlmegosztásnál, ám hamar megértettük, miért nélkülözhetetlen eszköze ez minden komoly fejlesztési projektnek. A commit történet, a branchek kezelése és a pull requestek világa teljesen új szemléletet adott a csapatban végzett munkához, soha nem veszett el kód, mindig vissza lehetett nézni ki mit és mikor változtatott, és a merge conflictek megoldása valódi csapatmunka-tapasztalatot jelentett. Inspiráló volt látni, hogy mekkora szenvedéllyel és elkötelezettséggel végzi munkáját, és ez a lelkesedés ránk is átragadt a fejlesztés során.

Hálásak vagyunk, hogy ilyen elhivatott, türelmes és szakmailag felkészült tanároktól tanulhattunk az évek során, és hogy a segítségükkel sikeresen megvalósíthattuk vizsgaremekünket. Bízunk benne, hogy a tőlük kapott tudást, a reszponzív tervezés elveit, a verziókezelés kultúráját és a szoftverfejlesztés iránti felelős hozzáállást, a jövőbeli tanulmányaink és munkánk során is kamatoztatni tudjuk majd.

4.5 Záró gondolatok

4.5.1 Le Tuan Anh (Róbert) visszatekintése

A FeladatRadar fejlesztése számomra az eddigi legtanulságosabb programozási projekt volt. A Naptár oldal (Naptar.razor) megírása különösen emlékezetes, a közel 2000 soros komponens három iteráción (havi, heti, napi nézet) ment keresztül, és rengeteg aprólékos részletet kellett megoldani: a hétkezdet kiszámítása, az ismétlődő bejegyzések megjelenítése, a közelgő feladatok panel és az "Új bejegyzés" modal mind-mind külön kihívást jelentett.

A SubjectService és az adatbázis Tárolt eljárások-k megírása során értettem meg igazán a paraméteres lekérdezések fontosságát. Kezdetben direktben összefűztem volna a string-eket az SQL-ben, de amikor az SQL injection-ről tanultunk, azonnal átírtam Dapper DynamicParameters-re. Ez a döntés nemcsak biztonságossá tette a kódot, hanem egyszerűbbé is, a Dapper automatikusan kezeli a típuskonverziókat.

A GitHub-os csapatmunka eleinte lassabb volt, mint a közvetlen fájlmegosztás, de az első merge conflict megoldása után értettük meg, miért éri meg: soha nem veszett el kód, mindig vissza lehetett nézni, ki mit és mikor változtatott.

4.5.2 Tarcsányi Csongor Márk visszatekintése

Az autentikáció és a JWT implementálás volt a legizgalmasabb részem. A JWT token felépítése, aláírása és ellenőrzése először bonyolultnak tűnt, de miután megértettem a struktúrát természetes lett. A BCrypt megértése is fontos pillanat volt: az, hogy a hash nem visszafejthető, és a „salt” automatikusan beépül, ezért minden jelszóhashnak más az értéke, még ha ugyanaz a jelszó is.

A TeacherService szavazás-modulja különösen tetszett: a GroupPolls Tárolt eljárás lapos JOIN eredményét C# dictionary-ba összevonni, majd Poll objektumokba összegyűjteni elegáns megoldás, és jól mutatja, hogy a Dapper és a C# együtt erősebb, mint az ORM automatizmusa.

Összességében büszke vagyok arra, amit alkottam. Amenniyben az iskola használná ezt a jövőben, jó érzés lenne látni, hogy a diákok a mindennapokban is használják.

4.5.3 Lőrincz Antal visszatekintése

A FocusTimerService megírása meglepően összetett feladatnak bizonyult. Első nekifutásra egyszerű JavaScript-es setInterval-nak tűnt, de Blazor Server esetén a timer a szerveren fut, ezért az UI frissítéséhez a komponensben explicit módon kell gondoskodni a Blazor szinkronizációs kontextusáról. A FocusTimerService egy OnTick eseményt süt el minden másodpercben, a Focus.razor komponens pedig erre feliratkozva hívja az InvokeAsync(StateHasChanged) metódust, így a szálkezelés helyesen elkülönül a service és a komponens között.

Ez a különbség a kliensoldalas és szerveroldali renderelés között valós tapasztalattal erősítette meg a tankönyvi tudást.

A Feladatok.razor fejlesztésekor a vizuális visszajelzők tervezésével sokat foglalkoztam: a piros/narancs kiemelések nemcsak szépek, hanem funkcionálisan is fontosak, egy pillantásra megmutatják, mi a sürgős és mi kész. Az IsOverdue és IsDueSoon értékeket a frontend modellrétegében számítjuk, C# computed property-ként a TaskModels.cs fájlban, ez azért praktikus, mert a Dapper által visszaadott adatokon azonnal, az API-hívás után alkalmazható a logika, és nem terheli feleslegesen a tárolt eljárást.

A csapatmunka során a legfontosabb lecke az volt, hogy kommunikálni kell, mielőtt nekiállunk valaminek.

4.5.4 Végszó

A FeladatRadar fejlesztése során mindannyian túlléptünk azon a határon, amelyet az iskolai keretek között megszoktunk. Megtanultuk, hogyan kell egy valódi szoftverfejlesztési projektet tervezni, kivitelezni és dokumentálni, csapatban, verziókövetéssel, teszteléssel és egymás kódjának megértésével. Ez az élmény és tudás értékesebb lesz a jövőbeli munkánkban, mint bármelyik elméleti anyag. Köszönjük a lehetőséget, hogy ezt a projektet elkészíthettük, és reméljük, hogy a FeladatRadar nemcsak vizsgán, hanem a valóságban is hasznos eszközzé válhat.

Irodalomjegyzék

5.1 Felhasznált online források

- Microsoft: ASP.NET Core dokumentáció (2025) https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-10.0
- Microsoft: Blazor Server bevezetés https://learn.microsoft.com/en-us/aspnet/core/blazor/
- Microsoft: JWT autentikáció ASP.NET Core-ban https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn
- Dapper GitHub repository és dokumentáció https://github.com/DapperLib/Dapper
- A program elkészítése közben Tabnine autokomplett funkciót vettünk igénybe a szintaktikai hibák csökkentésére. https://www.tabnine.com/
- BCrypt.Net-Next NuGet csomag dokumentáció https://www.nuget.org/packages/BCrypt.Net-Next
- Blazored.LocalStorage dokumentáció https://github.com/Blazored/LocalStorage
- SQL Server 2022 dokumentáció: tárolt eljárások https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/
- JSON Web Token (JWT) RFC 7519 szabvány https://datatracker.ietf.org/doc/html/rfc7519
- MDN Web Docs: Reszponzív webdesign alapjai https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
- Pomodoro Technique®: Hivatalos módszerleírás https://francescocirillo.com/pages/pomodoro-technique
- GitHub: A projekt forráskódja https://github.com/RobiGanz/FeladatRadar
- dbdesigner.net: Adatbázis-diagram tervező eszköz https://dbdesigner.net
- draw.io: Folyamatábra és architektúradiagram rajzoló https://app.diagrams.net
- Swagger / OpenAPI dokumentáció: Swashbuckle https://github.com/domaindrivendev/Swashbuckle.AspNetCore

5.2 Felhasznált könyvek

- Andrew Lock: ASP.NET Core in Action, 3rd Edition. Manning Publications, 2023.
- Jon P. Smith: Entity Framework Core in Action, 2nd Edition. Manning Publications, 2021.
- Christian Nagel: Professional C# and .NET 2021 Edition. Wiley, 2021.